

SQL 프로그래밍

CONTENTS

목차

CHAPTER 1

Database 개요

CHAPTER 2

MySQL 설치 및 세팅

CHAPTER 3

SQL 실습

CHAPTER 4

Spring Boot 개요

CHAPTER 5

개발환경 세팅

CHAPTER 6

Hello Spring

CHAPTER 7

MVC

CHAPTER 8

IOC

CHAPTER 9

DI

CONTENTS

목차

CHAPTER 10 프로젝트

CHAPTER 11 DB 제어 방법

CHAPTER 12 Mybatis & MySQL 연동

CHAPTER 13 프로젝트

CHAPTER 14 차량용 대시보드 백엔드 설계

CHAPTER 15 차량용 대시보드 백엔드 구현

Chapter 01

Data Base

SQL 프로그래밍

용어 정리

✓ DB(Data Base)

- 데이터의 기지(base) 체계적으로 구조화된 데이터의 집합

✓ DBMS(DB Manegement System)

- Data Base를 관리하기 위한 시스템
- MySQL / Oracle / Maria / SQLite / Mongo DB(No) / Redis(No) ...
- 대표 DB : Oracle사 소유 MySQL (오픈 소스 1위) / Oracle (기업대상 1위)



DB없이 개발 가능 여부

모든 데이터를 Text file로 저장

- DB 없이 웹 서비스 개발 방법
- 웹서버에 id.txt / pass.txt 파일을 만들어서 저장
- 게시판 글은 article.txt 파일을 만들어 저장
- 로그인 요청시 id.txt / pass.txt 를 읽어 확인

개발 필요

- 보안
- 조회,수정 알고리즘 구현
- 에러방지

개발자 실수로 인해 데이터(파일)가 탈취되거나, 고객 데이터가 망가질 확률이 높음
여러 명이 동시에 글을 쓰면 파일이 깨지거나, 나중에 쓴 글이 먼저 쓴 글을 덮어쓰우는 문제가 발생

DB 필요성

✓ 고객 데이터들을 안전하게 관리

- 수십 년간 다듬어진 최적의 조회/수정/삭제 알고리즘 내장
- 알고리즘들이 이미 구현되어 있음
- 무료로 사용 가능한 툴 존재

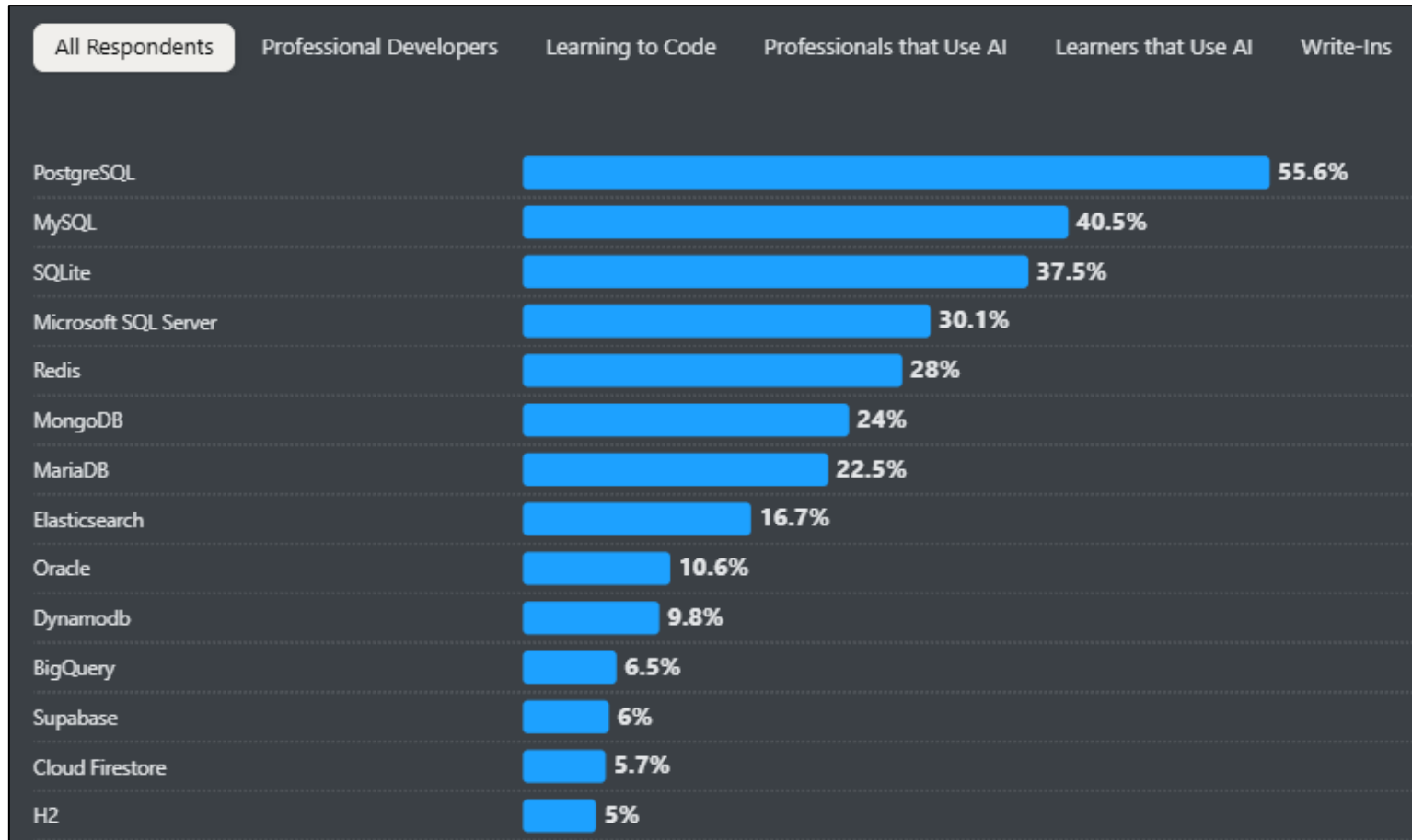
데이터들을 편리하고, 빠르고, 안전하고, 무료로 사용할 수 있다.

- 데이터 관리는 DB가 필수
- DBMS 에서 데이터 관리 방법을 알고,
- DBMS를 제어하는 "SQL" 만 학습하면 데이터 제어를 가능



- ✓ DB는 RDBMS (관계형 DBMS) / NoSQL으로 구분
 - 관계형 DBMS
 - 구조 및 제약조건 (스키마)를 만들고 값을 저장
 - 대표적 RDBMS : MySQL / Oracle / PostgreSQL
 - NoSQL
 - 구조,관계가 없는 DBMS
 - 비관계형 DBMS 마다 구조가 다양하다.
 - 대표적인 비관계형 DBMS
 - Mongo DB
 - Redis

✓ DBMS 순위



MySQL 소개

- ✓ Open Source / 기업도 무료
- ✓ DBMS에 포함된 도구
- ✓ MySQL Server 포함
- ✓ MySQL Client 프로그램
- ✓ GUI 용 : **MySQL Workbench** (DBeaver)
- ✓ CLI 용 : mysql

참고)SQL 코딩테스트

✓ 국내 코딩테스트의 대표 유형 (4문항 3시간)

- 3문항 알고리즘 1문항 SQL
- 빠른 SQL 문제 풀이후, 알고리즘에 집중하는 경우가 많았음
- SQL 문제가 고도화중

The screenshot displays a SQL coding test interface. At the top, there are tabs for '프로그래밍1', '프로그래밍2', '프로그래밍3', and 'SQL1', with 'SQL1' being the active tab. The main area is divided into two columns. The left column, titled '문제 설명' (Problem Description), contains text about the 'GAME_USERS' table and its columns: ID, DISTANCE, TIME_SPENT, and BEST_DATE. Below this text is a table schema for 'GAME_USERS' with columns NAME, TYPE, ID, DISTANCE, TIME_SPENT, and BEST_DATE. The right column, titled 'solution.sql', contains a text editor with a red box highlighting the first two lines of code: '1. -- 코드를 입력하세요' and '2. SELECT'. Below the editor, there is a section for '실행 결과' (Execution Results) with a placeholder text '실행 결과가 여기에 표시됩니다.' (Execution results will be displayed here).

NAME	TYPE
ID	VARCHAR
DISTANCE	INT
TIME_SPENT	DECIMAL
BEST_DATE	DATETIME

NAME	TYPE
NAME	VARCHAR
SPEED	INT
BOOST_SPEED	INT
BOOST_TIME	INT
PRICE	INT

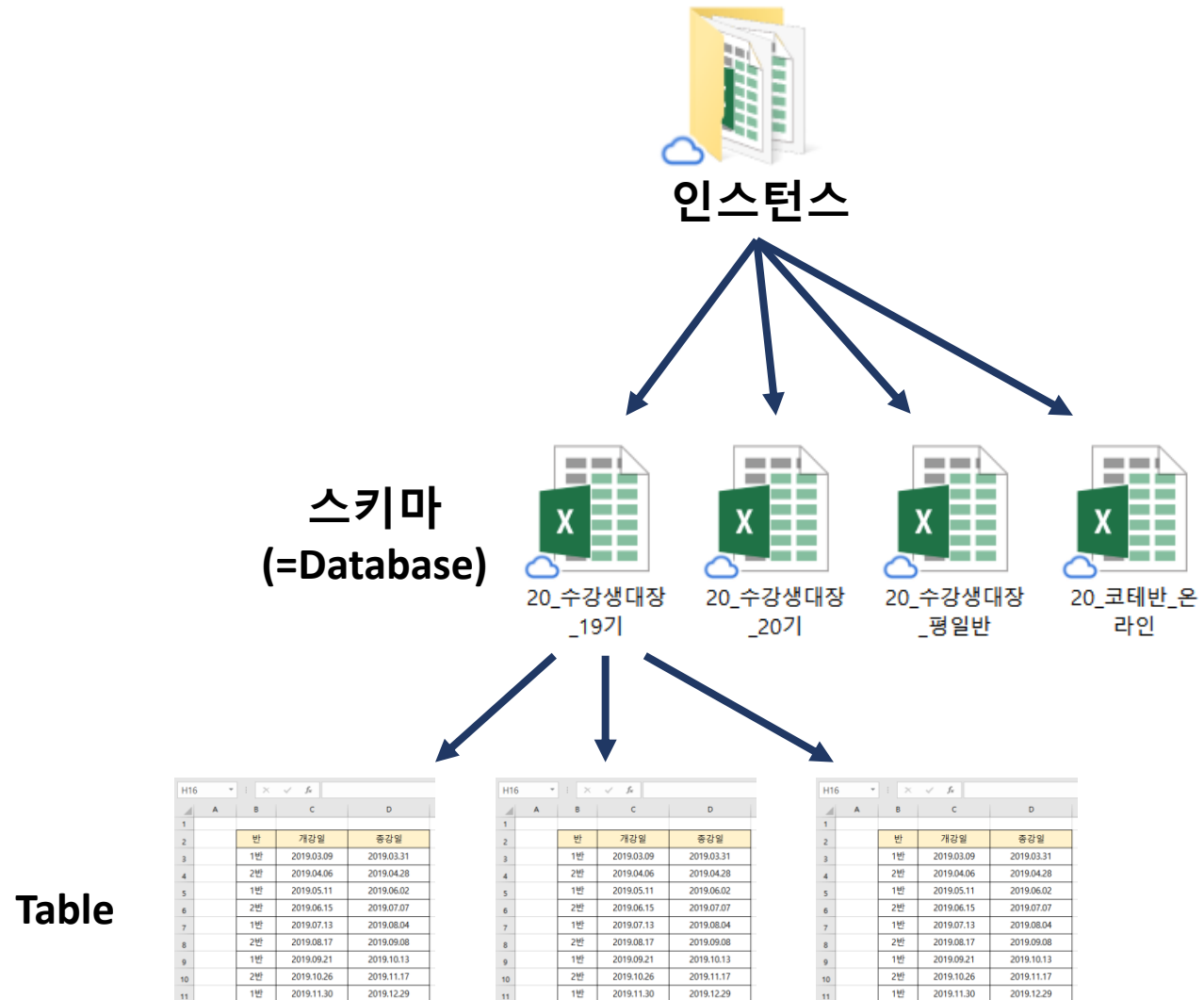
```
1. -- 코드를 입력하세요
2. SELECT
```

mysql / oracle 선택가능

MySQL 구조

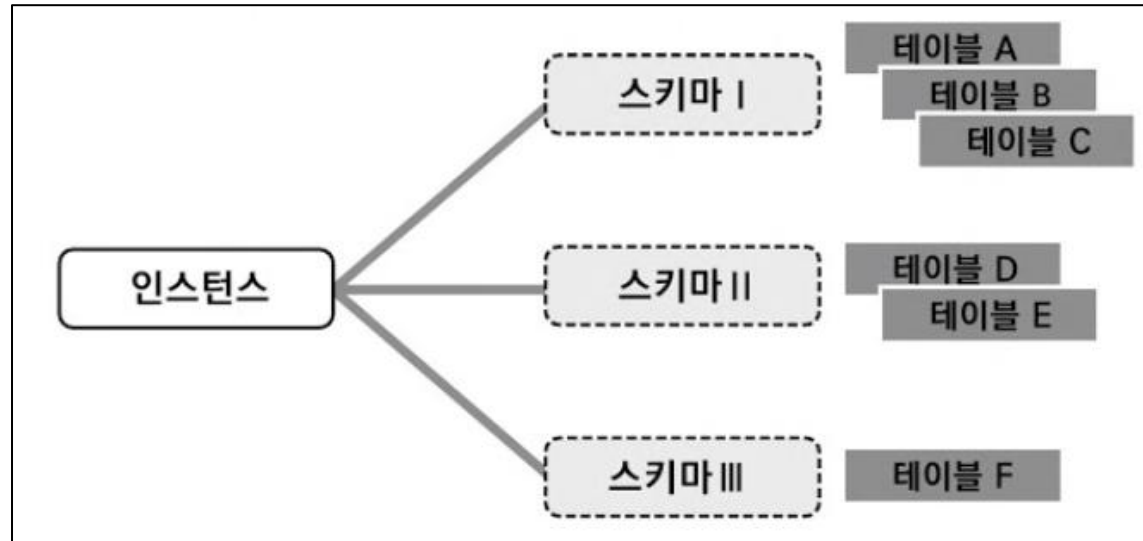
✓ DB는 3 계층 구조

- MySQL 8 기준
 - 인스턴스 = DB 서버
 - 스키마 = Database
 - 테이블
- 유의사항
 - Oracle에서는 Database와 Schema 독립된 4 계층 구조로 되어있어 MySQL 구조와 혼동



Server Instance

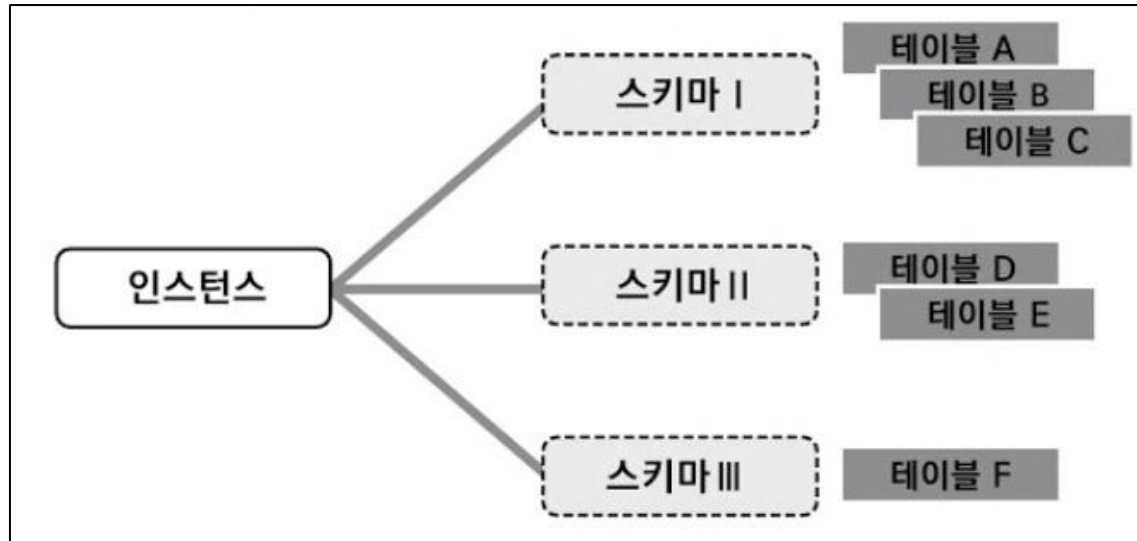
- ✓ 서버 인스턴스는 하나의 DB Server 지칭
 - DBMS를 설치되면 인스턴스가 자동 생성
 - 하나의 DB를 운영하기 위해 내부 Buffer / 내부 저장공간 / 관리 도구들이 동작되어야 한다. 운영이 필요한 모든 도구들을 모아 “서버 인스턴스” 라고 부른다.



Schema

✓ 스키마는 Database와 같은 의미

1. 스키마 생성 SQL : CREATE DATABASE minco;
2. 스키마 생성 SQL : CREATE SCHEMA minco;



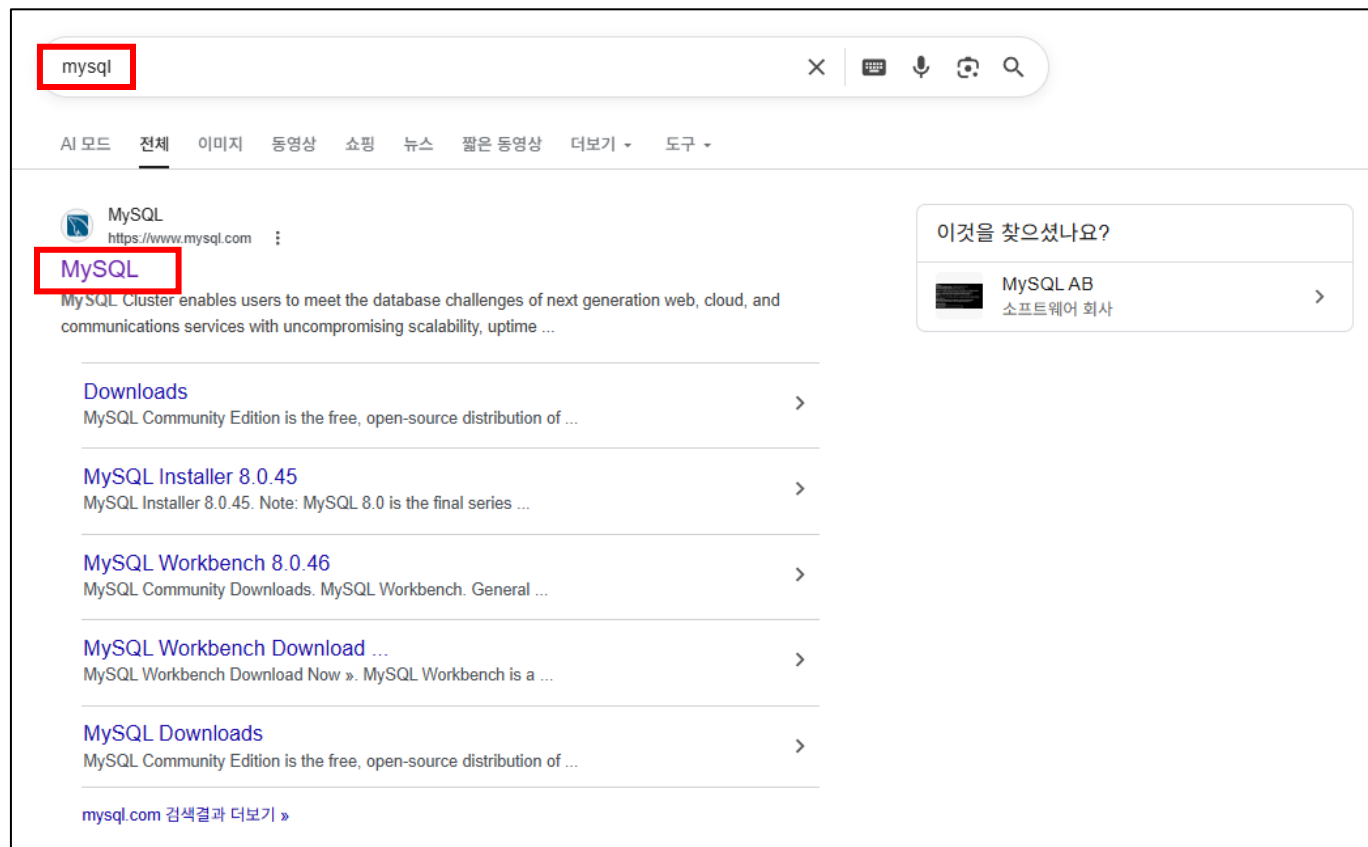
Chapter 02

MySQL 설치 및 세팅

SQL 프로그래밍

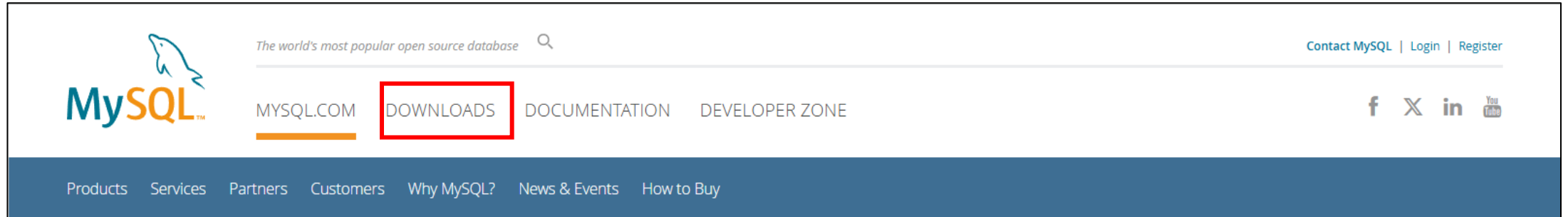
개발 환경 세팅

✓ MySQL 검색 후 방문



개발 환경 세팅

✓ DOWNLOADS 클릭



- ✓ MySQL 웹사이트 접속 **MySQL Community** 다운로드

MySQL Newsletter Subscribe » Archive »  Free Webinars MySQL Global Forum Tuesday, July 22, 2025 Unlocking the Power of JavaScript in MySQL: Creating Stored Programs with Ease On-Demand What's New in MySQL Monitoring with Oracle Enterprise Manager Plugin On-Demand More » Contact Sales USA: +1-866-221-0634 Canada: +1-866-221-0634	MySQL Enterprise Edition MySQL Enterprise Edition includes the most comprehensive set of advanced features, management tools and technical support for MySQL. Learn More » Customer Download from My Oracle Support (MOS) » Trial Download from Oracle edelivery » Developer Download from Oracle OTN » MySQL NDB Cluster CGE MySQL NDB Cluster is a real-time open source transactional database designed for fast, always-on access to data under high throughput conditions. • MySQL NDB Cluster • MySQL NDB Cluster Manager • Plus, everything in MySQL Enterprise Edition Learn More » Customer Download from My Oracle Support (MOS) » Trial Download from Oracle edelivery » MySQL Community (GPL) Downloads »
---	---

✓ MySQL Installer for Windows 클릭

MySQL Community Downloads

- MySQL Yum Repository
- MySQL APT Repository
- MySQL SUSE Repository
- MySQL Community Server
- MySQL NDB Cluster
- MySQL Router
- MySQL Shell
- MySQL Operator
- MySQL NDB Operator
- MySQL Workbench
- C API (libmysqlclient)
- Connector/C++
- Connector/J
- Connector/NET
- Connector/Node.js
- Connector/ODBC
- Connector/Python
- MySQL Native Driver for PHP
- MySQL Benchmark Tool
- Time zone description tables
- Download Archives

MySQL Installer for Windows

MySQL Enterprise Edition for Developers

Free for learning, developing,
and prototyping.



Download Now »

개발 환경 세팅

✓ 버전 체크 : 8.0.20 다운그레이드

MySQL Community Downloads

MySQL Installer

General Availability (GA) Releases

Archives

MySQL Installer 8.0.42

Note: MySQL 8.0 is the final series with MySQL Installer. As of MySQL 8.1, use a MySQL product's MSI or Zip archive for installation. MySQL Server 8.1 and higher also bundle MySQL Configurator, a tool that helps configure MySQL Server.

Select Version:
8.0.42

Select Operating System:
Microsoft Windows

Windows (x86, 32-bit), MSI Installer (mysql-installer-web-community-8.0.42.0.msi)	8.0.42	2.1M	Download
Windows (x86, 32-bit), MSI Installer (mysql-installer-community-8.0.42.0.msi)	8.0.42	353.7M	Download

We suggest that you use the MD5 checksums and GnuPG signatures to verify the integrity of the packages you download.

개발 환경 세팅

✓ 버전 체크 : 8.0.20 다운그레이드

MySQL Product Archives

MySQL Installer (Archived Versions)

⚠ Please note that these are old versions. New releases will have recent bug fixes and features!
To download the latest release of MySQL Installer, please visit [MySQL Downloads](#).

Product Version: **8.0.20** ▼
Operating System: **Microsoft Windows** ▼

Windows (x86, 32-bit), MSI Installer (mysql-installer-web-community-8.0.20.0.msi)	Apr 14, 2020	24.4M	Download
Windows (x86, 32-bit), MSI Installer (mysql-installer-community-8.0.20.0.msi)	Apr 14, 2020	420.6M	Download

! We suggest that you use the [MD5 checksums](#) and [GnuPG signatures](#) to verify the integrity of the packages you download.

MySQL open source software is provided under the [GPL License](#).

✓ 로그인은 무시(나오지 않는 경우도 있음)

MySQL Community Downloads

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

Login »

using my Oracle Web account

Sign Up »

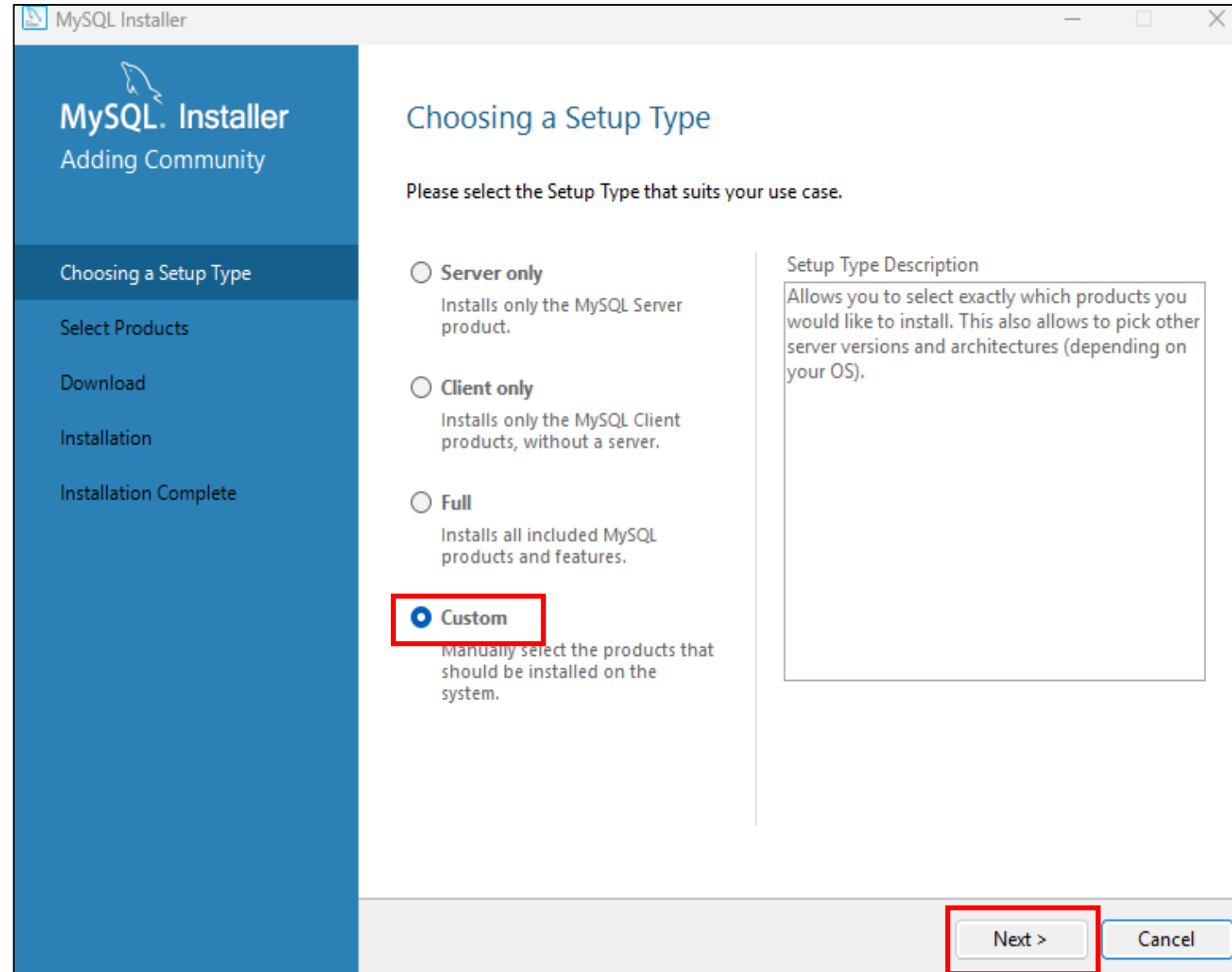
for an Oracle Web account

MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can sign up for a free account by clicking the Sign Up link and following the instructions.

No thanks, just start my download.

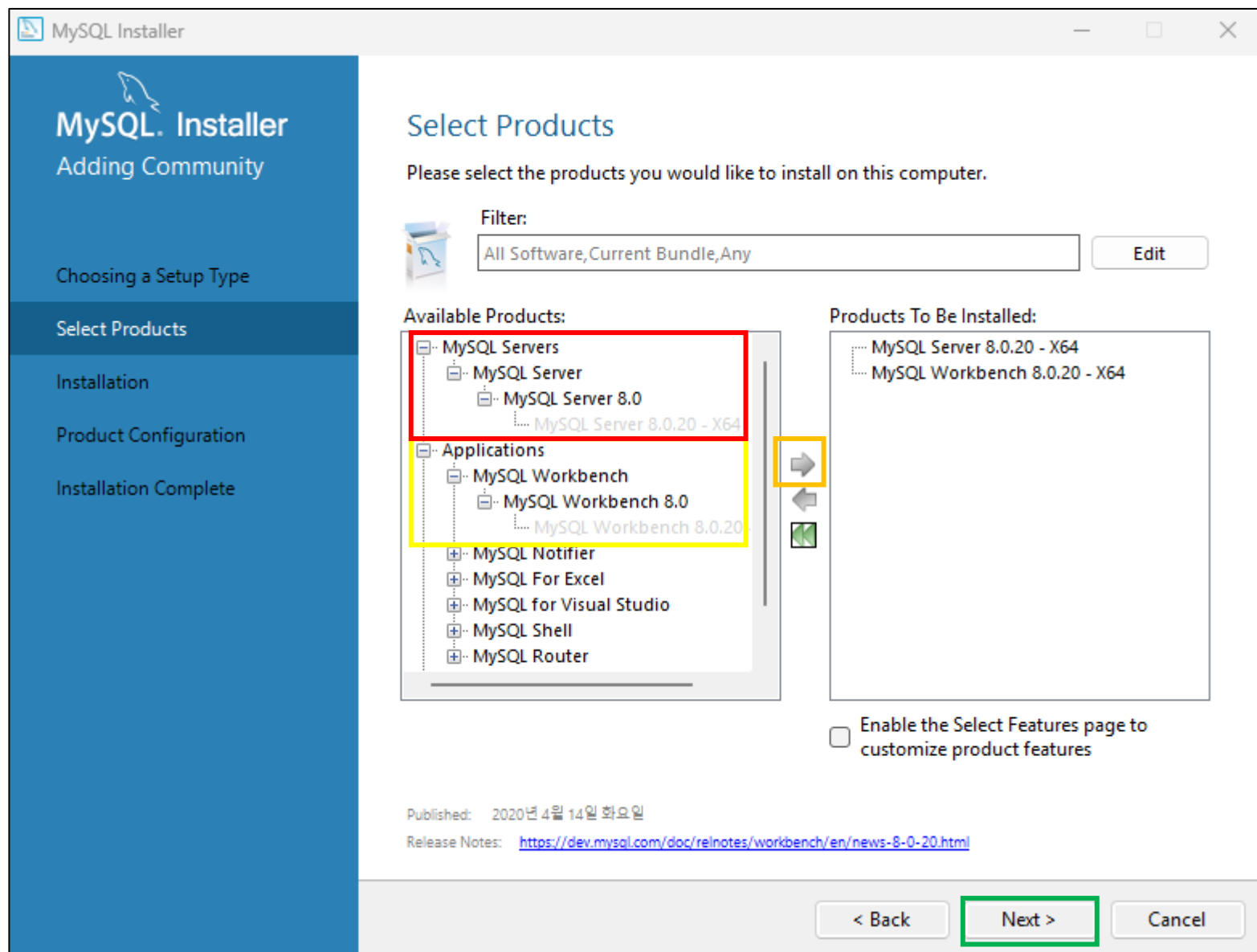
개발 환경 세팅

✓ 나오지 않는 화면이 있다면
Next 버튼 등을 눌러 다음화면으로 이동

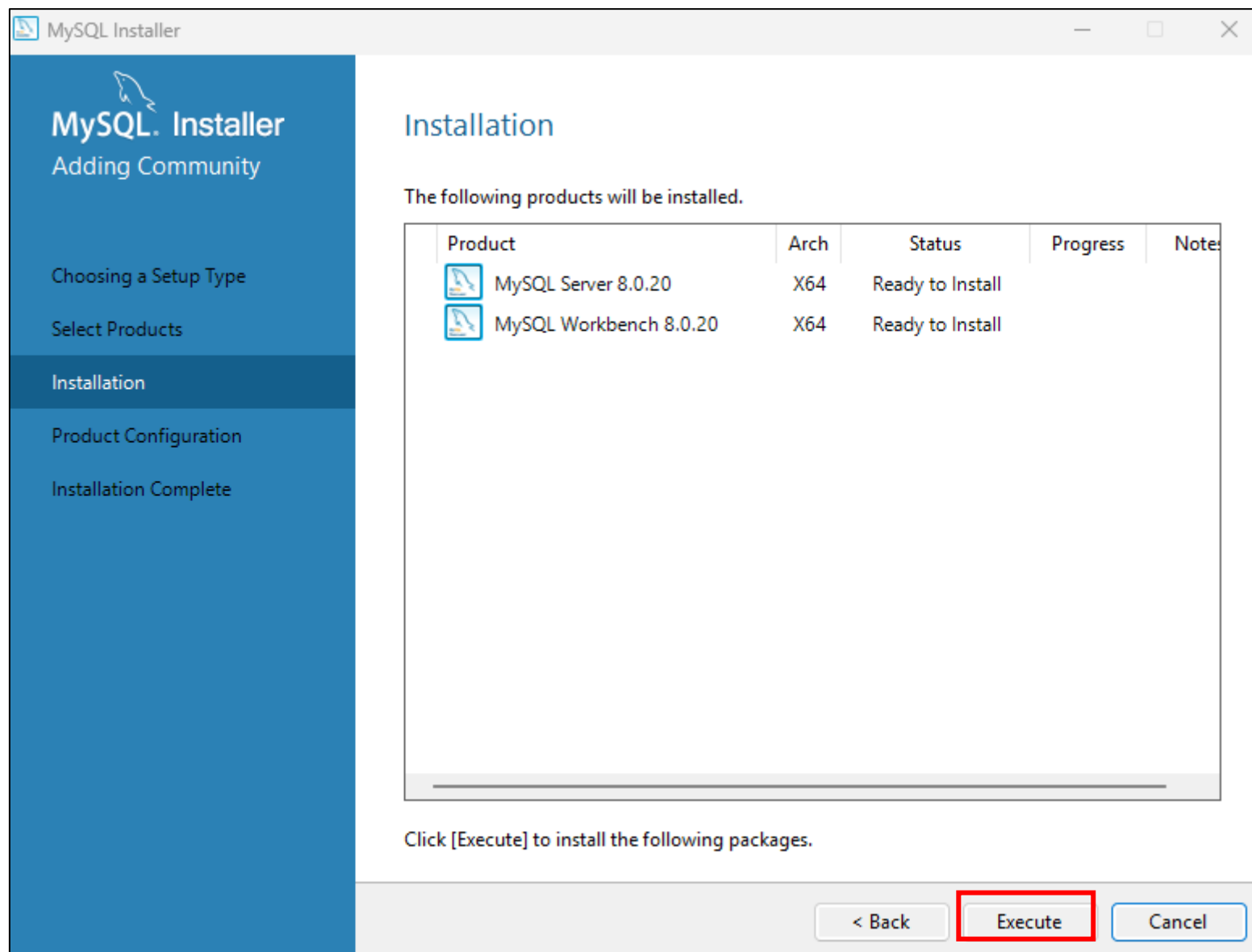


개발 환경 세팅

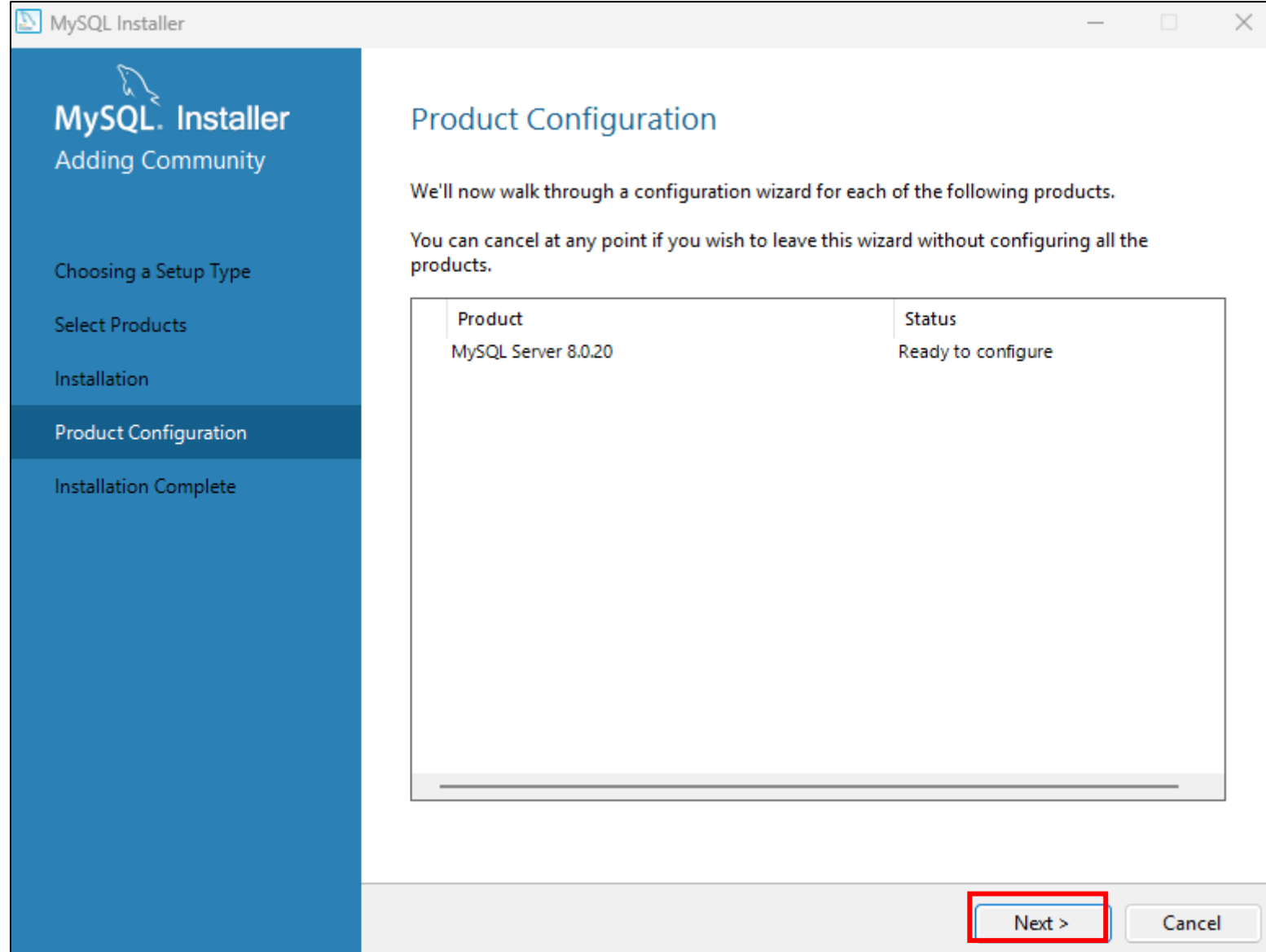
- ✓ Custom 설정으로 설치를 한다.
 - 실제 강의에 필요한 도구들
 - MySQL Server
 - MySQL Workbench

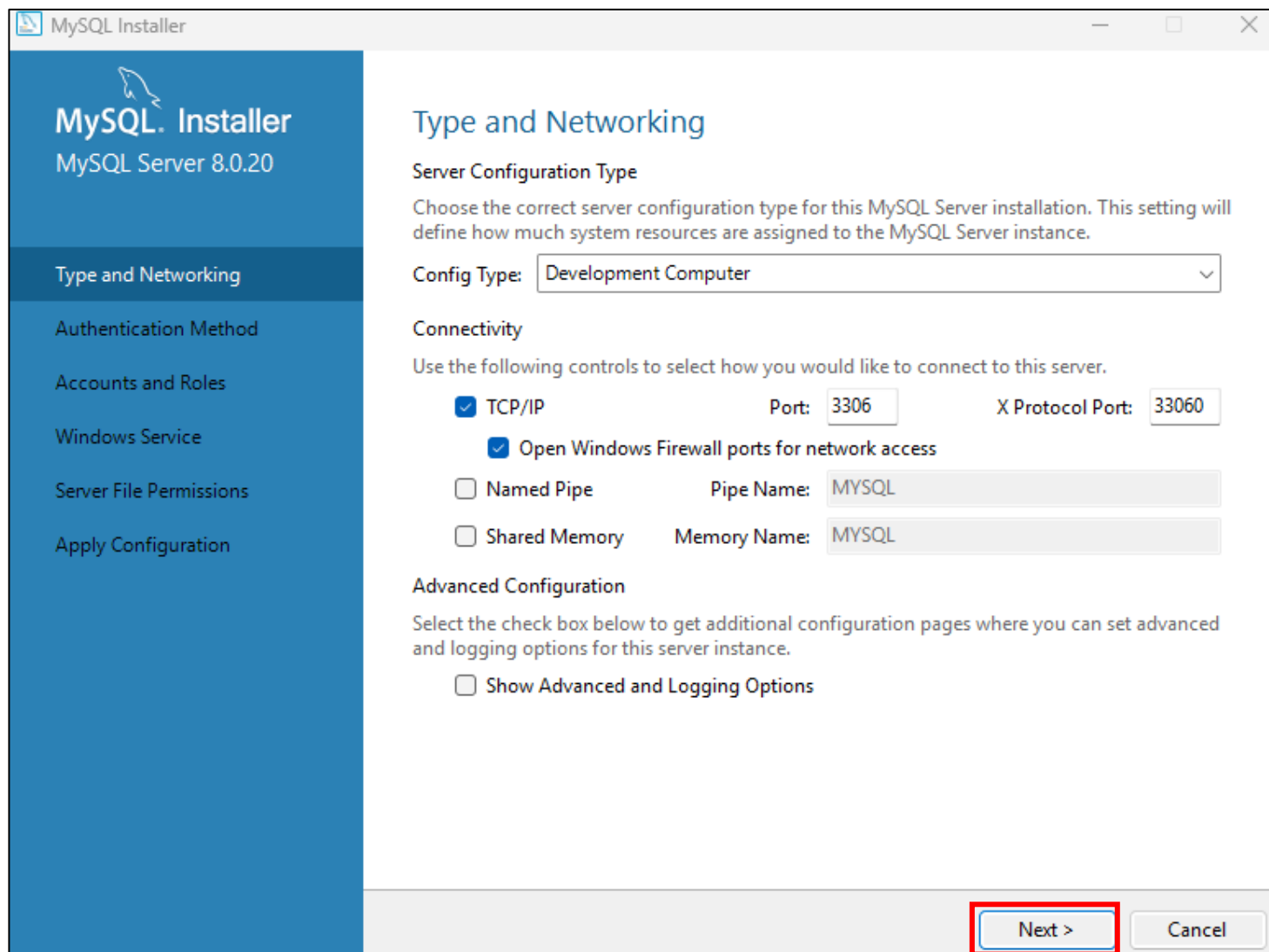


개발 환경 세팅



개발 환경 세팅



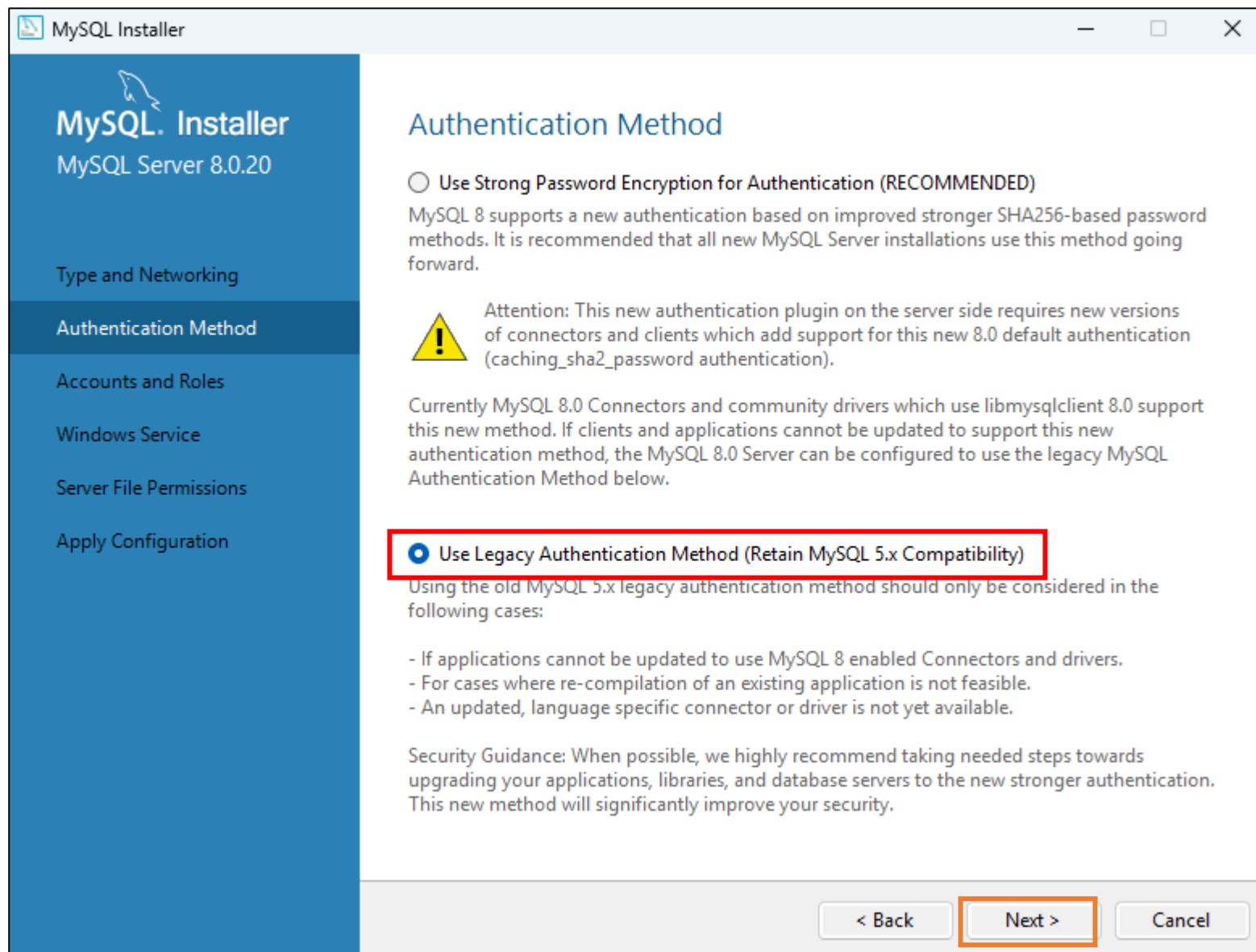


The image shows the MySQL Installer window for MySQL Server 8.0.20. The left sidebar contains the following navigation items: MySQL. Installer, MySQL Server 8.0.20, Type and Networking (selected), Authentication Method, Accounts and Roles, Windows Service, Server File Permissions, and Apply Configuration. The main area is titled 'Type and Networking' and contains the following sections:

- Server Configuration Type**
Choose the correct server configuration type for this MySQL Server installation. This setting will define how much system resources are assigned to the MySQL Server instance.
Config Type: Development Computer
- Connectivity**
Use the following controls to select how you would like to connect to this server.
 - ☒ TCP/IP Port: 3306 X Protocol Port: 33060
 - ☒ Open Windows Firewall ports for network access
 - ☐ Named Pipe Pipe Name: MYSQL
 - ☐ Shared Memory Memory Name: MYSQL
- Advanced Configuration**
Select the check box below to get additional configuration pages where you can set advanced and logging options for this server instance.
 - ☐ Show Advanced and Logging Options

At the bottom right, there are two buttons: 'Next >' (highlighted with a red box) and 'Cancel'.

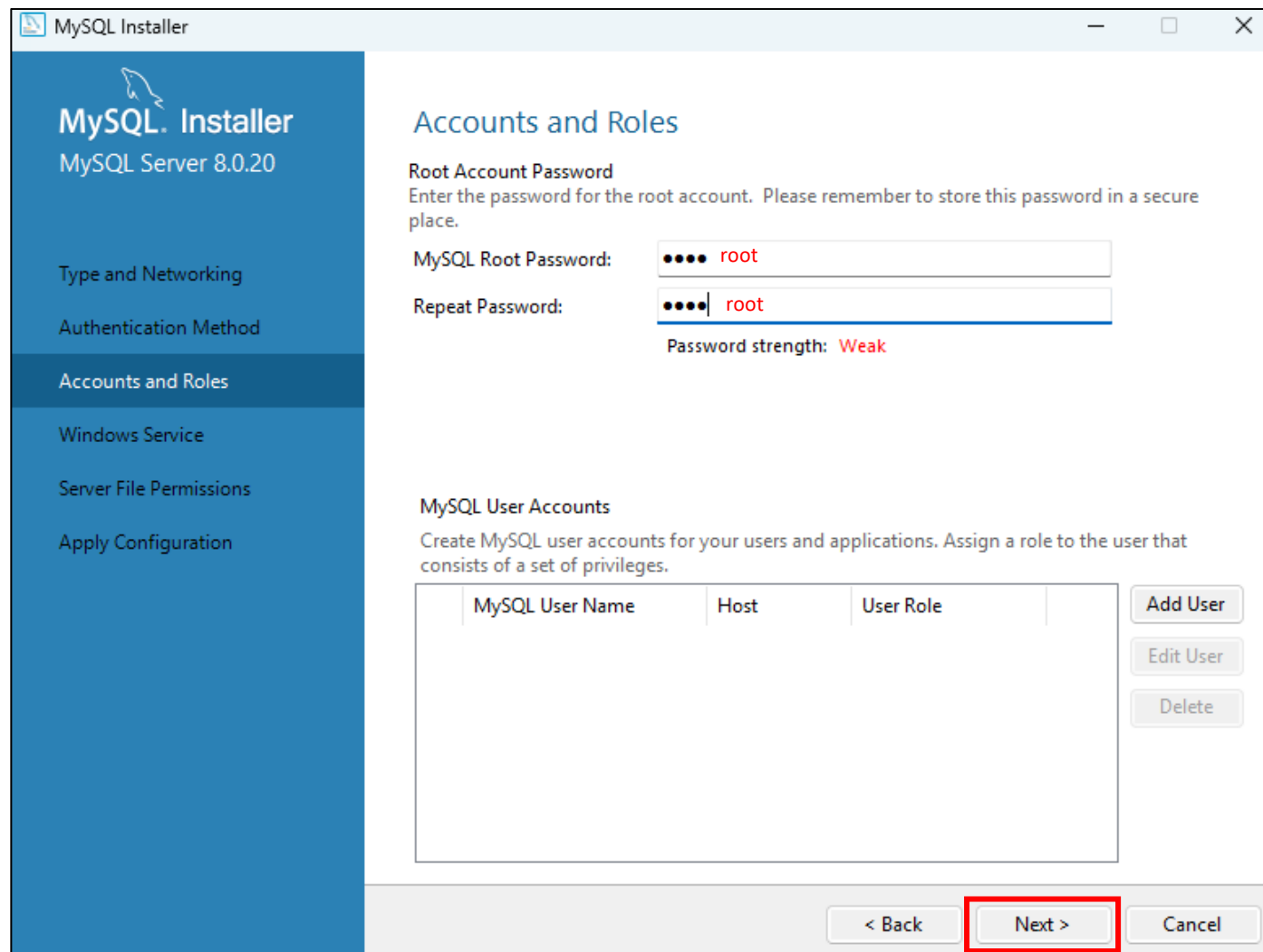
✓ 레거시로 선택



개발 환경 세팅

✓ 비밀번호 설정

- 비밀번호는 root
- 실전에선 말도 안되지만, 실습 목적



The image shows the 'Accounts and Roles' step of the MySQL Installer for MySQL Server 8.0.20. The left sidebar lists the installation steps: Type and Networking, Authentication Method, Accounts and Roles (selected), Windows Service, Server File Permissions, and Apply Configuration. The main area is titled 'Accounts and Roles' and contains the 'Root Account Password' section. It prompts the user to enter a password for the root account, with a note to store it securely. The 'MySQL Root Password' field contains 'root' and the 'Repeat Password' field also contains 'root'. The password strength is indicated as 'Weak'. Below this is the 'MySQL User Accounts' section, which includes a table for creating new users and buttons for 'Add User', 'Edit User', and 'Delete'. The table has columns for 'MySQL User Name', 'Host', and 'User Role'. At the bottom, there are navigation buttons: '< Back', 'Next >' (highlighted with a red box), and 'Cancel'.

MySQL Installer
MySQL Server 8.0.20

Type and Networking
Authentication Method
Accounts and Roles
Windows Service
Server File Permissions
Apply Configuration

Accounts and Roles

Root Account Password
Enter the password for the root account. Please remember to store this password in a secure place.

MySQL Root Password:

Repeat Password:

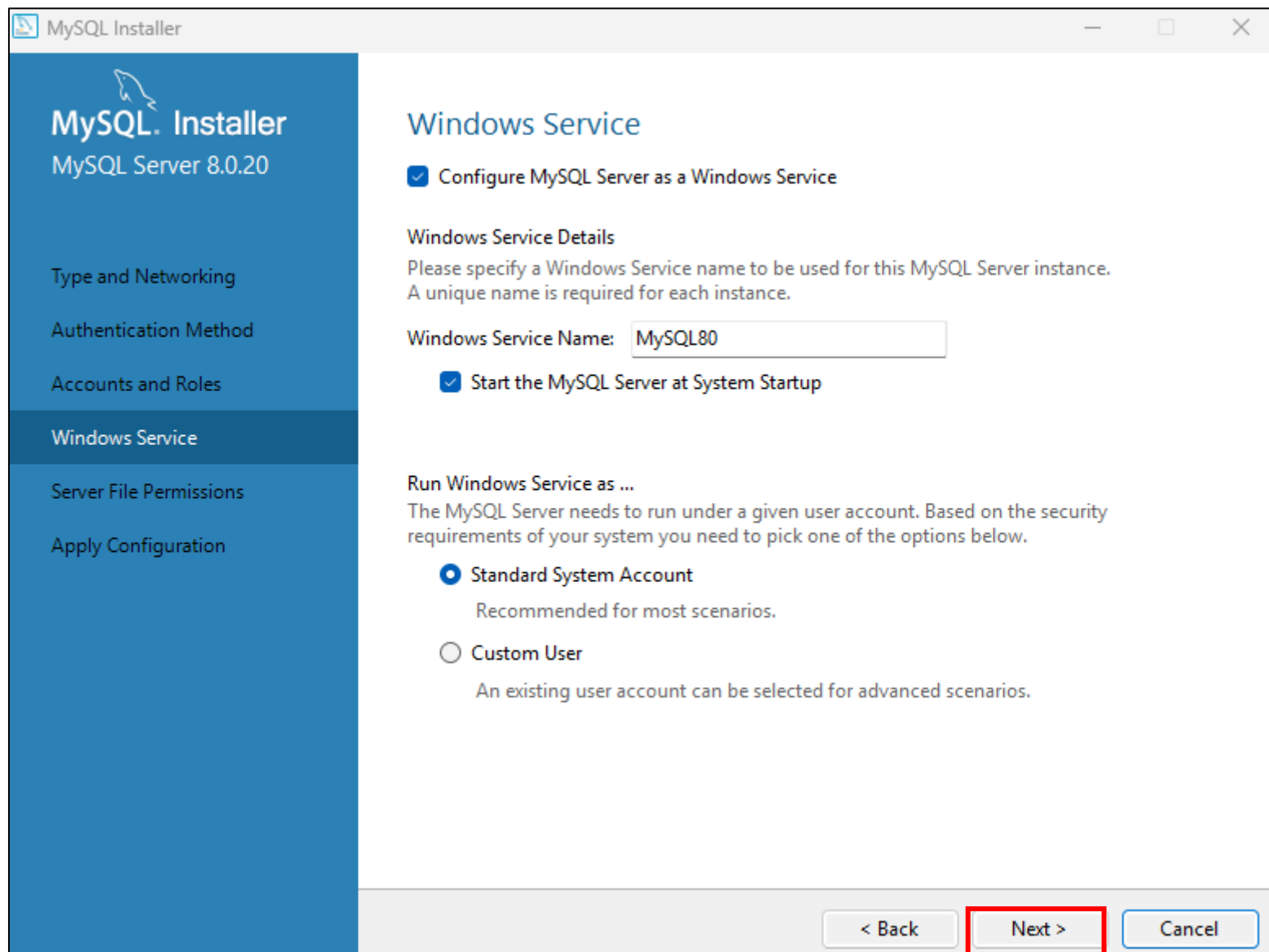
Password strength: Weak

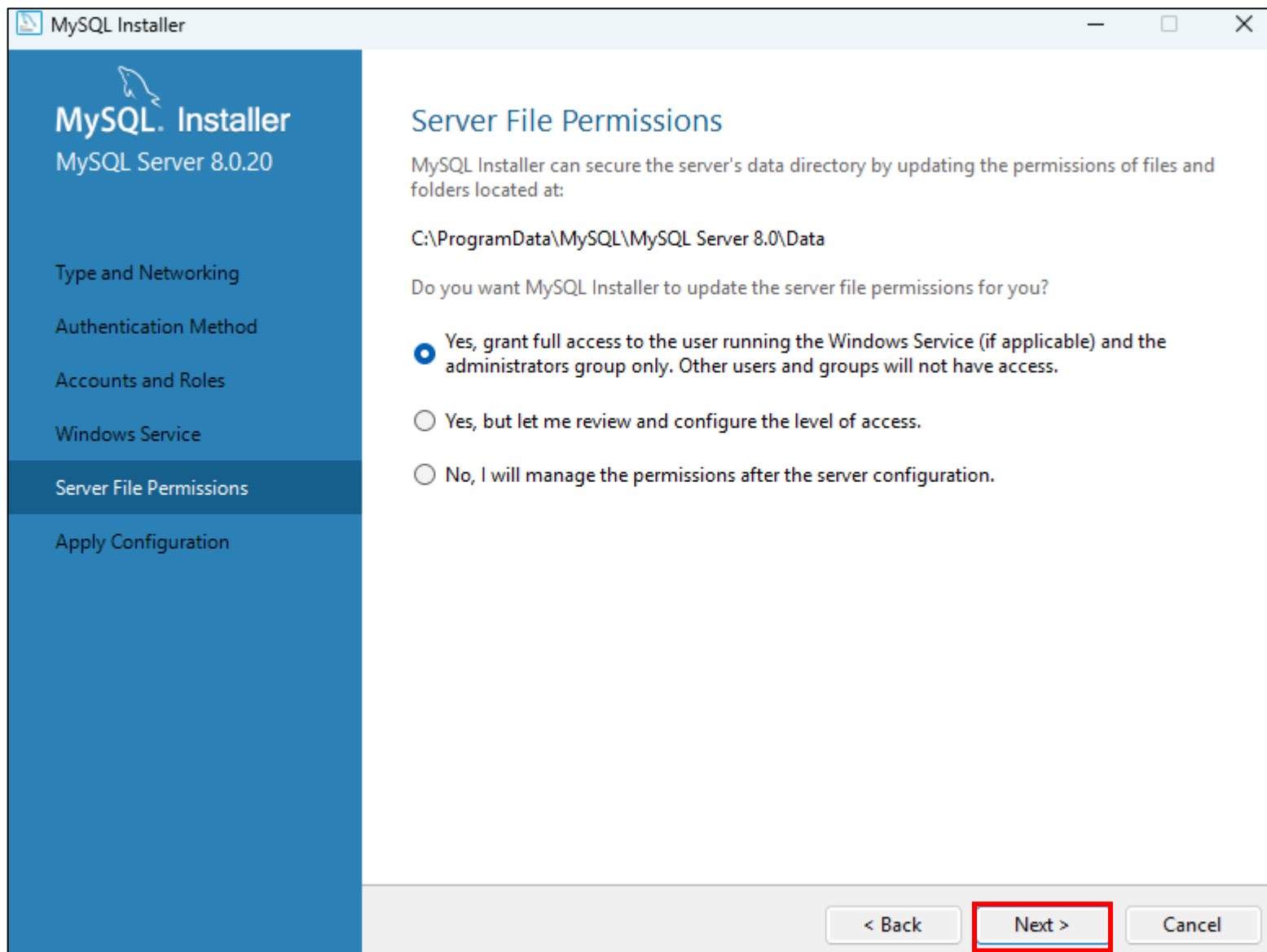
MySQL User Accounts
Create MySQL user accounts for your users and applications. Assign a role to the user that consists of a set of privileges.

MySQL User Name	Host	User Role
-----------------	------	-----------

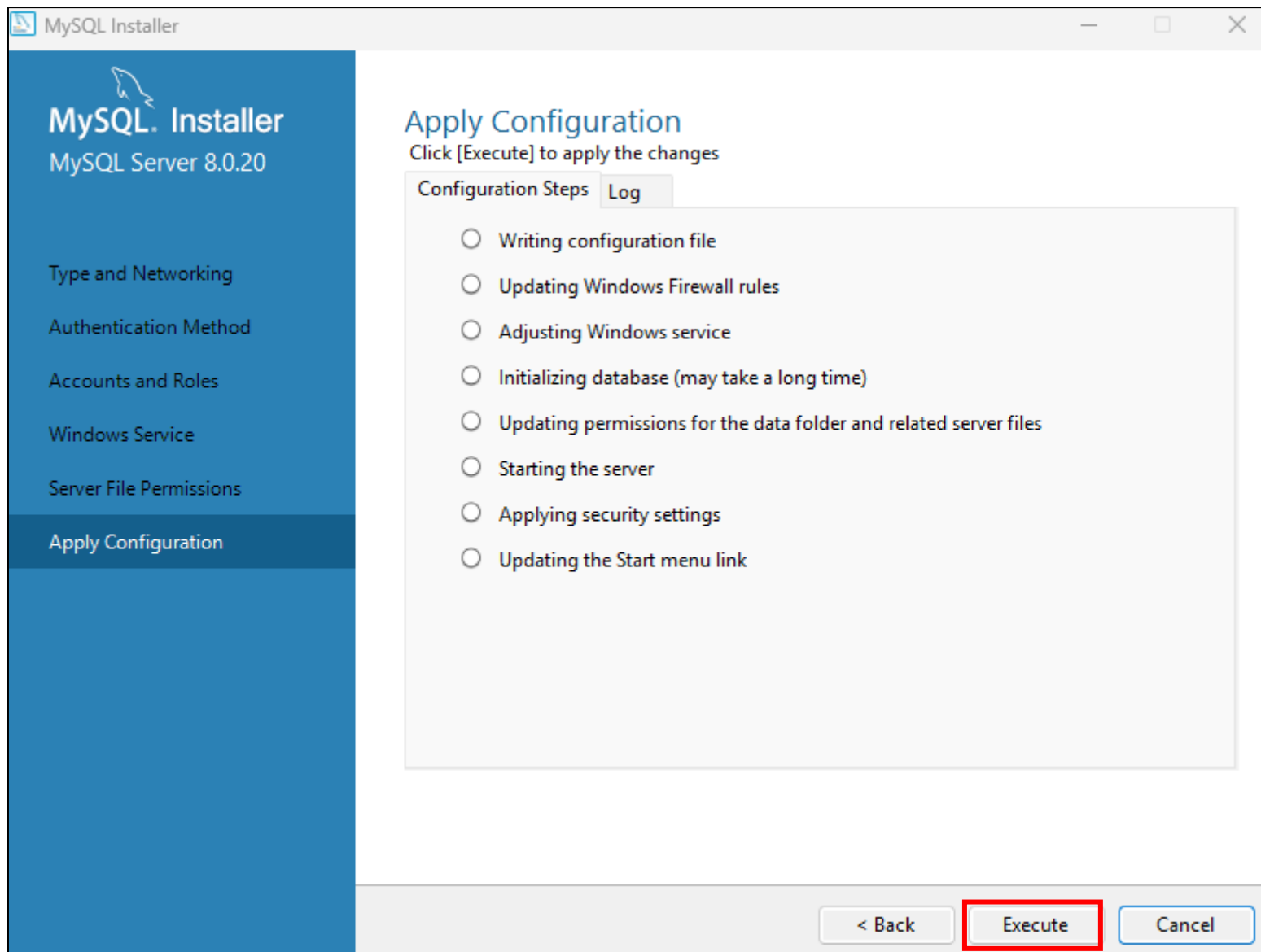
Add User
Edit User
Delete

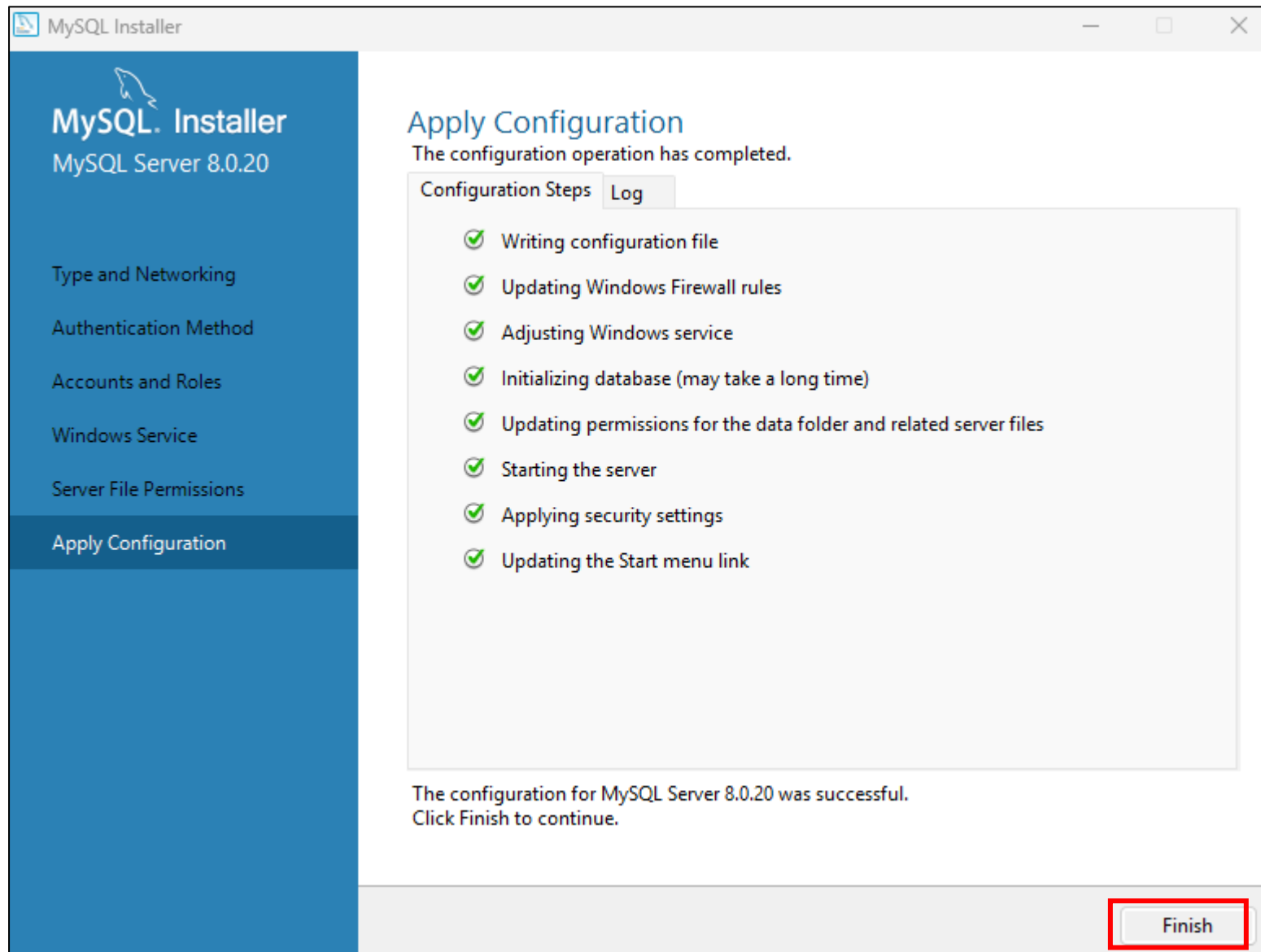
< Back **Next >** Cancel

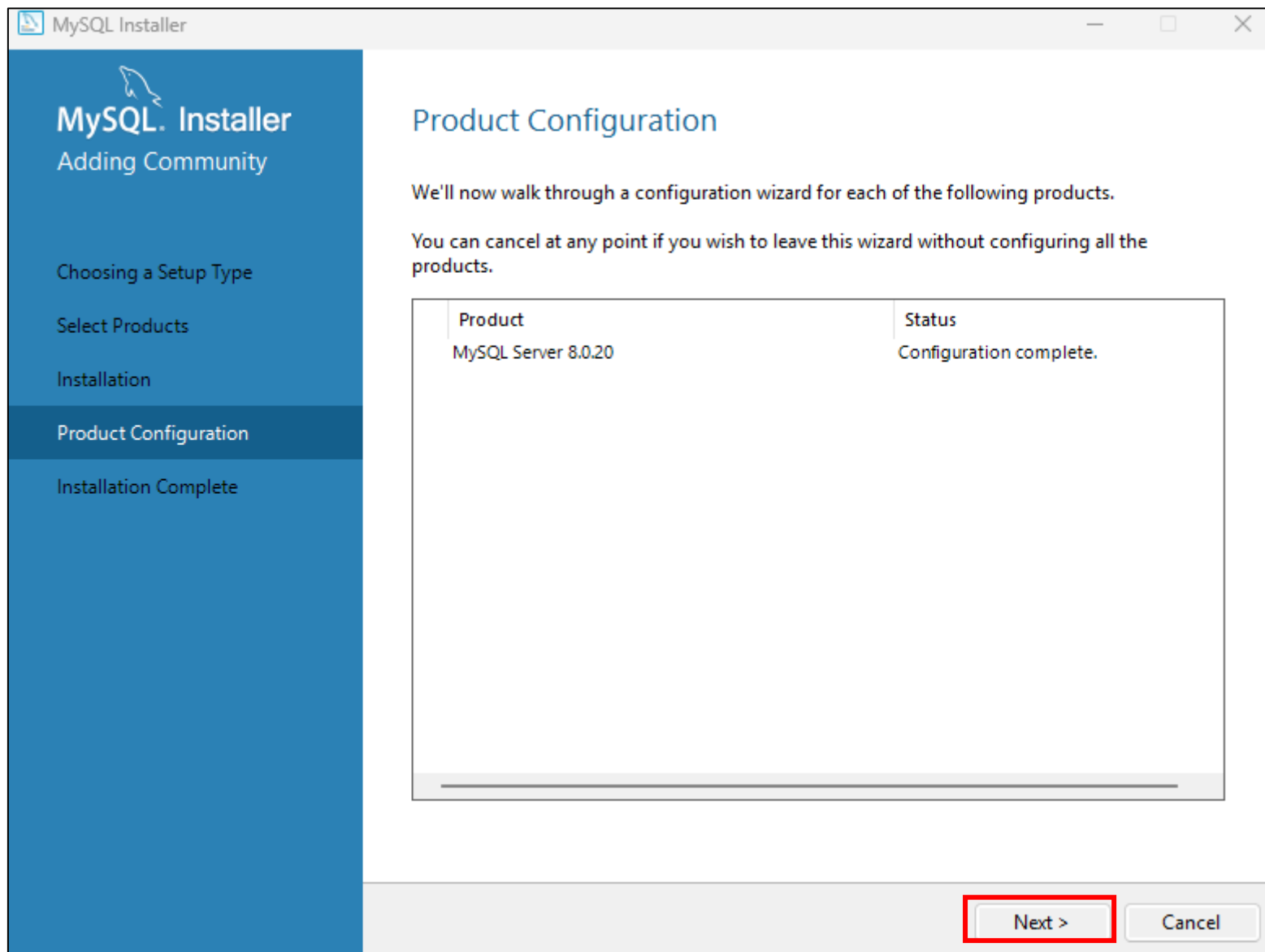




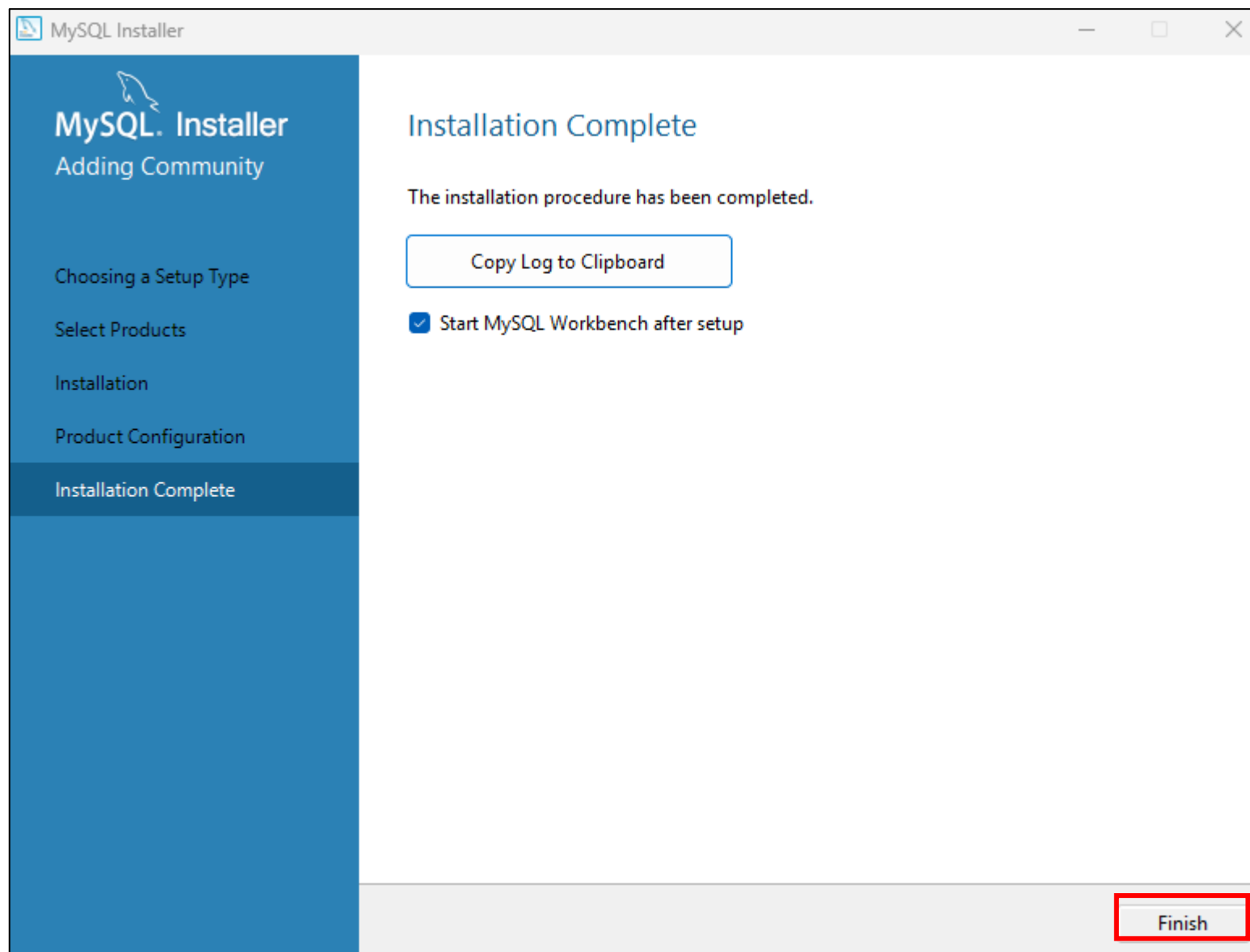
개발 환경 세팅







개발 환경 세팅

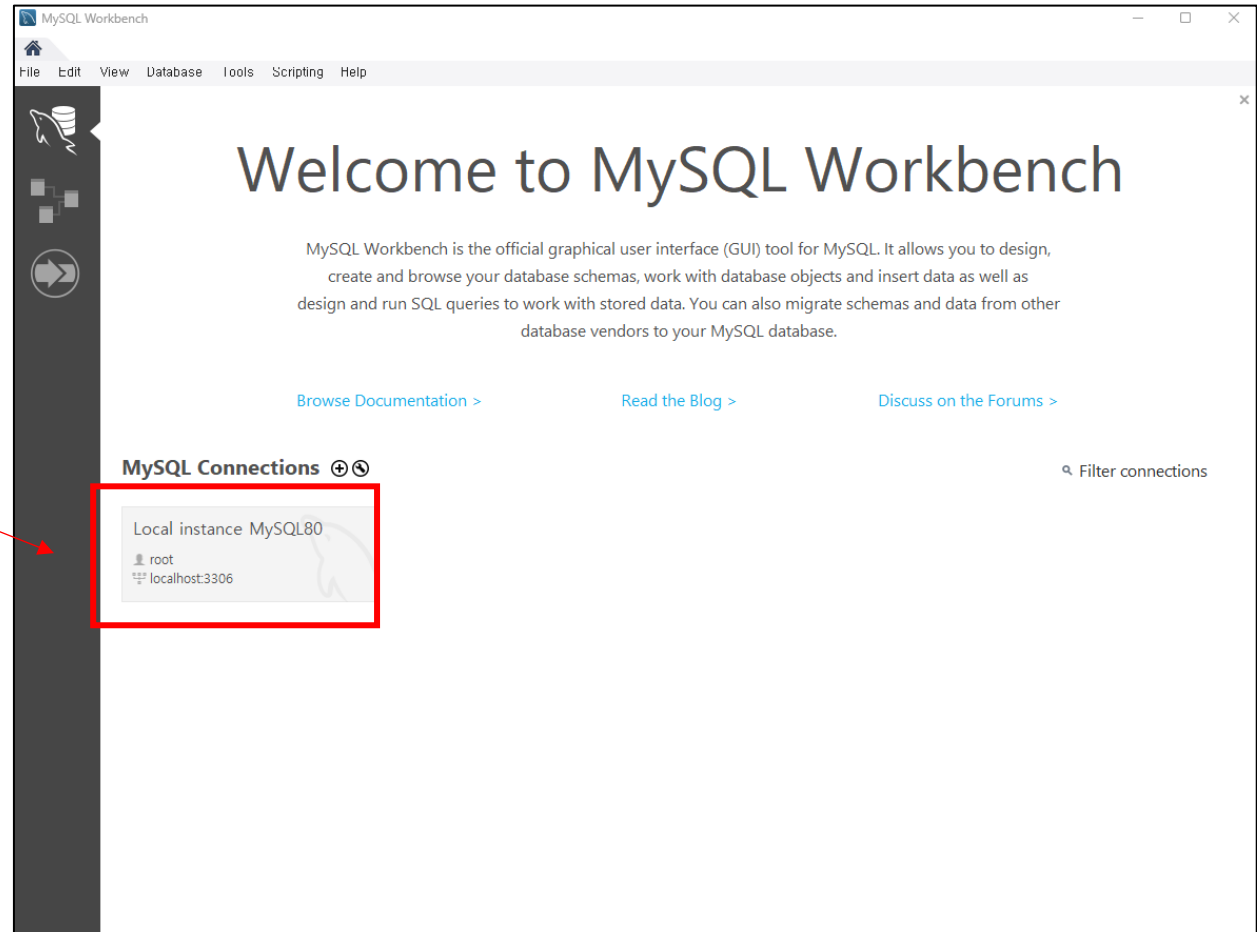


개발 환경 세팅

✓ MySQL Workbench 동작 Test

- 생성된 DB Instance에 접속할 수 있도록, 접속 정보가 등록 되어있음

설치 과정에서 추가된
DB Server (DB Instance)에
접속할 수 있다.

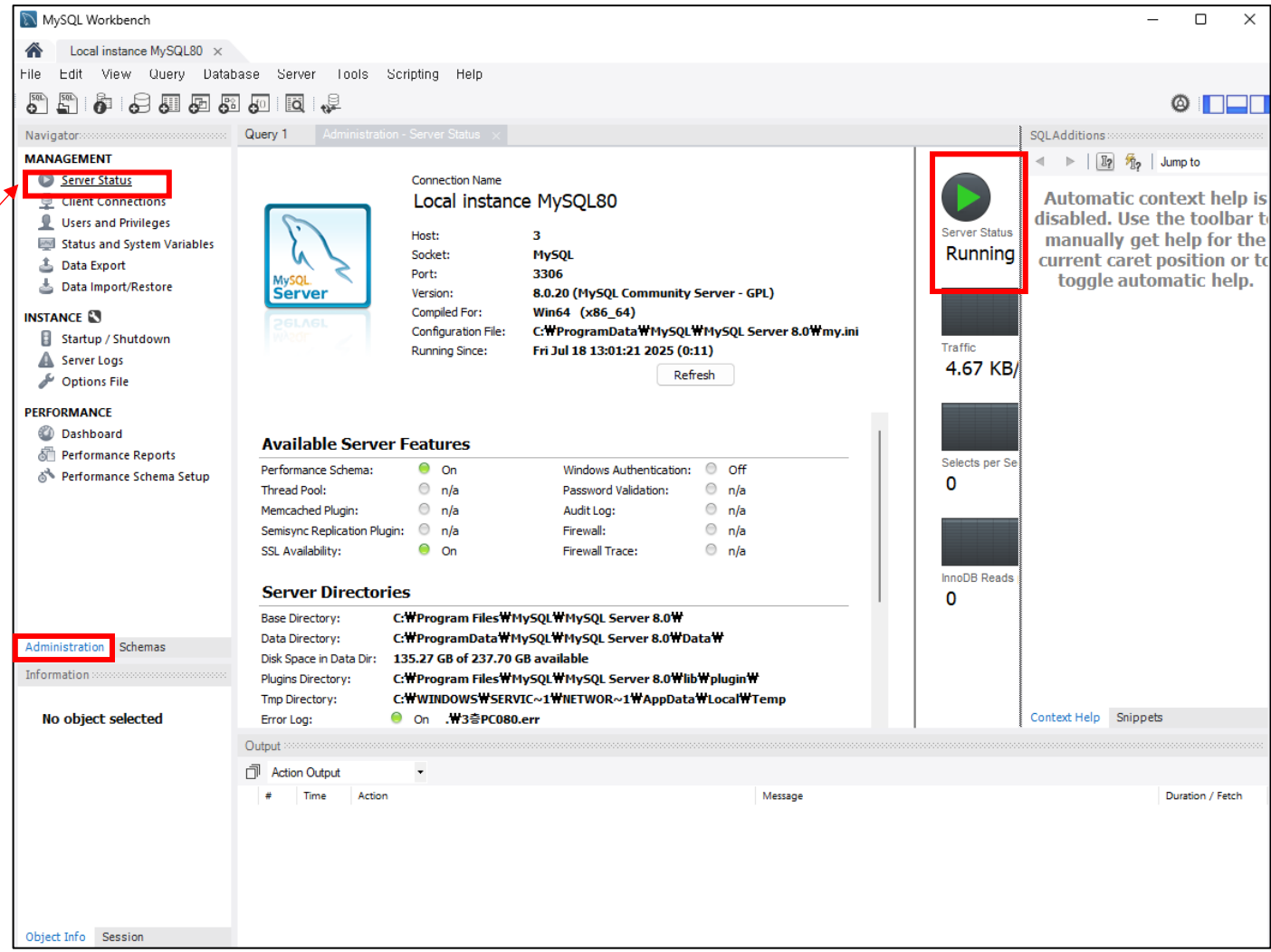


개발 환경 세팅

정상적으로 Local Server가 운영되고 있음을 확인할 수 있다.

- DB로 접속하여 값을 R/W 할 수 있는 상태

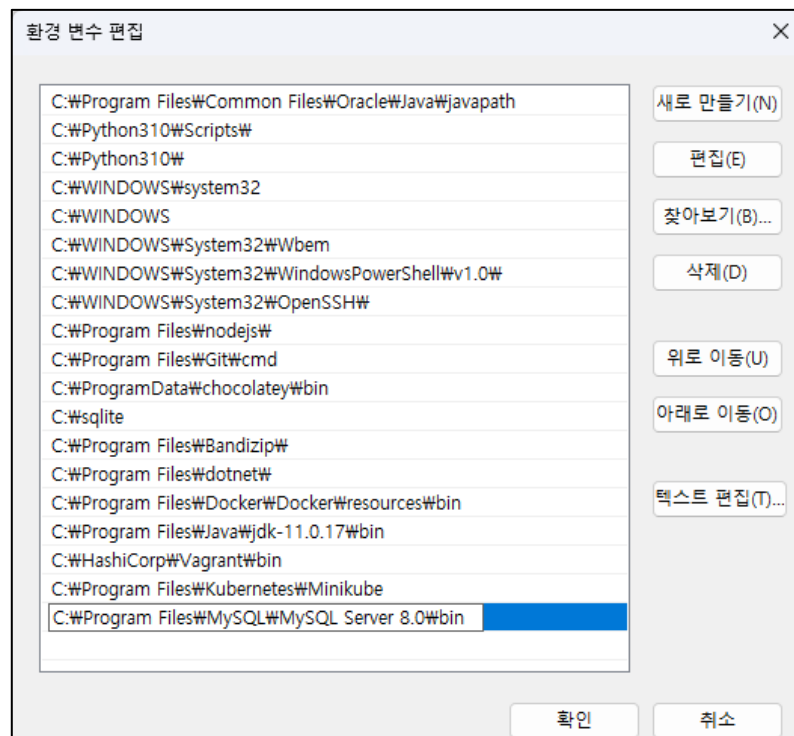
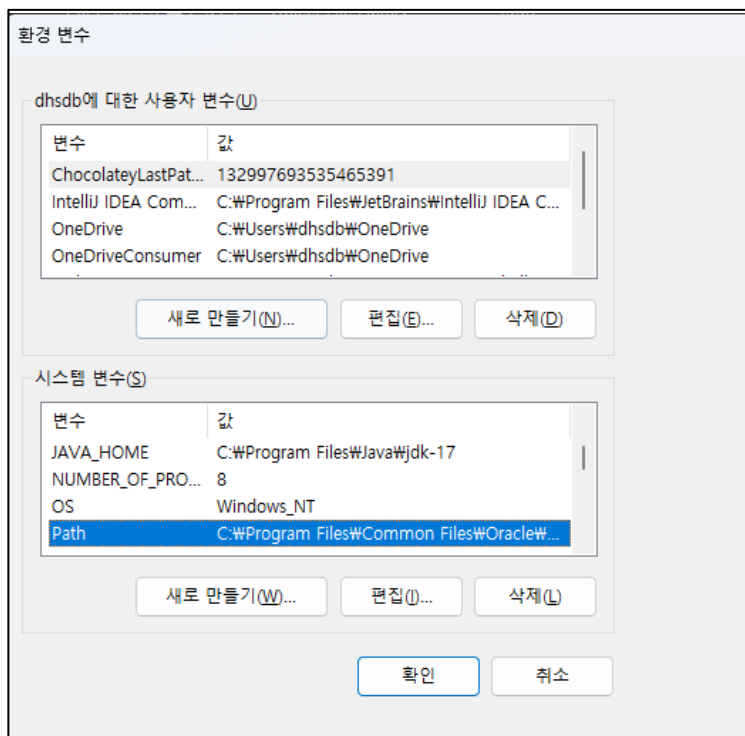
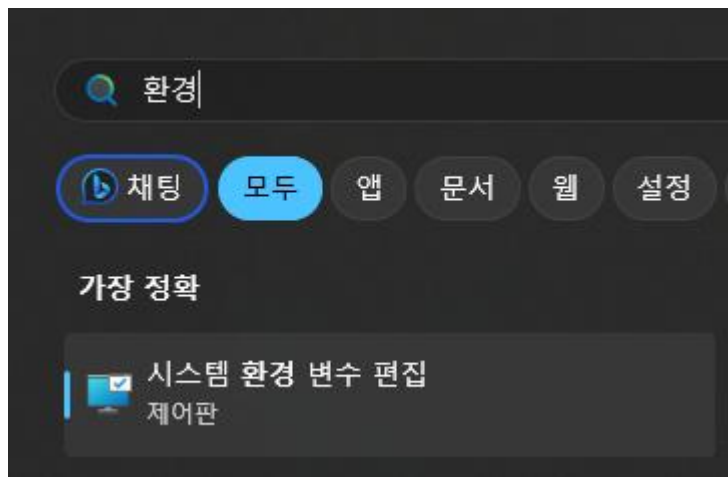
만약 서버가
중지되어있다면
Startup 해주면 된다.



환경 변수 등록

✓ 환경변수 등록

- 환경변수를 등록하면 해당 파일이 존재하는 디렉토리외에서도 사용이 가능하다
- C:\Program Files\MySQL\MySQL Server 8.0\bin 디렉토리 주소 복사
- Path 더블 클릭 -> 환경변수 넣고 저장



환경 변수 등록

✓ 환경변수 등록 체크

- cmd로 터미널을 열고
mysql -u root -p 입력

✓ 명령 프롬프트 창을 띄운 후 mysql 입력

- 해당 에러메세지가 뜨면 path 등록 된것
- 접속방법mysql -u [계정아이디] -p

```
C:\Users\dhsdb>mysql
ERROR 1045 (28000): Access denied for user 'ODBC'@'localhost' (using password: NO)
C:\Users\dhsdb>|
```



```
C:\Users\dhsdb>mysql -u root -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 15
Server version: 8.0.29 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

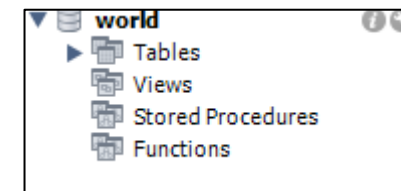
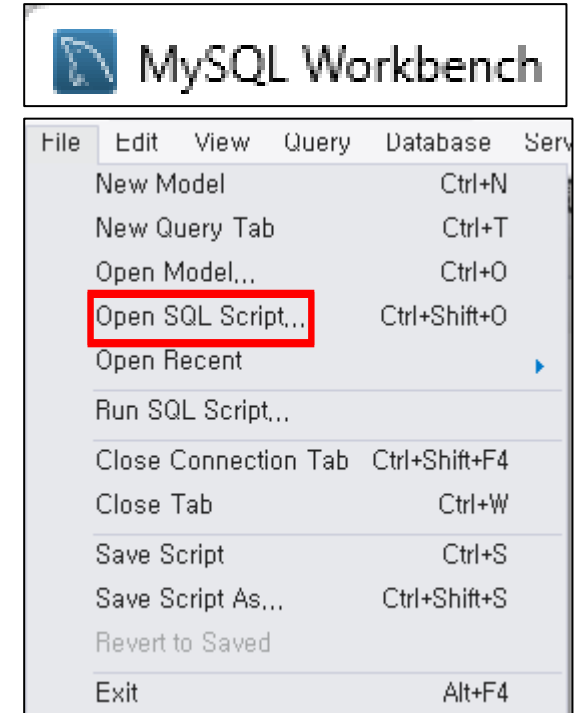
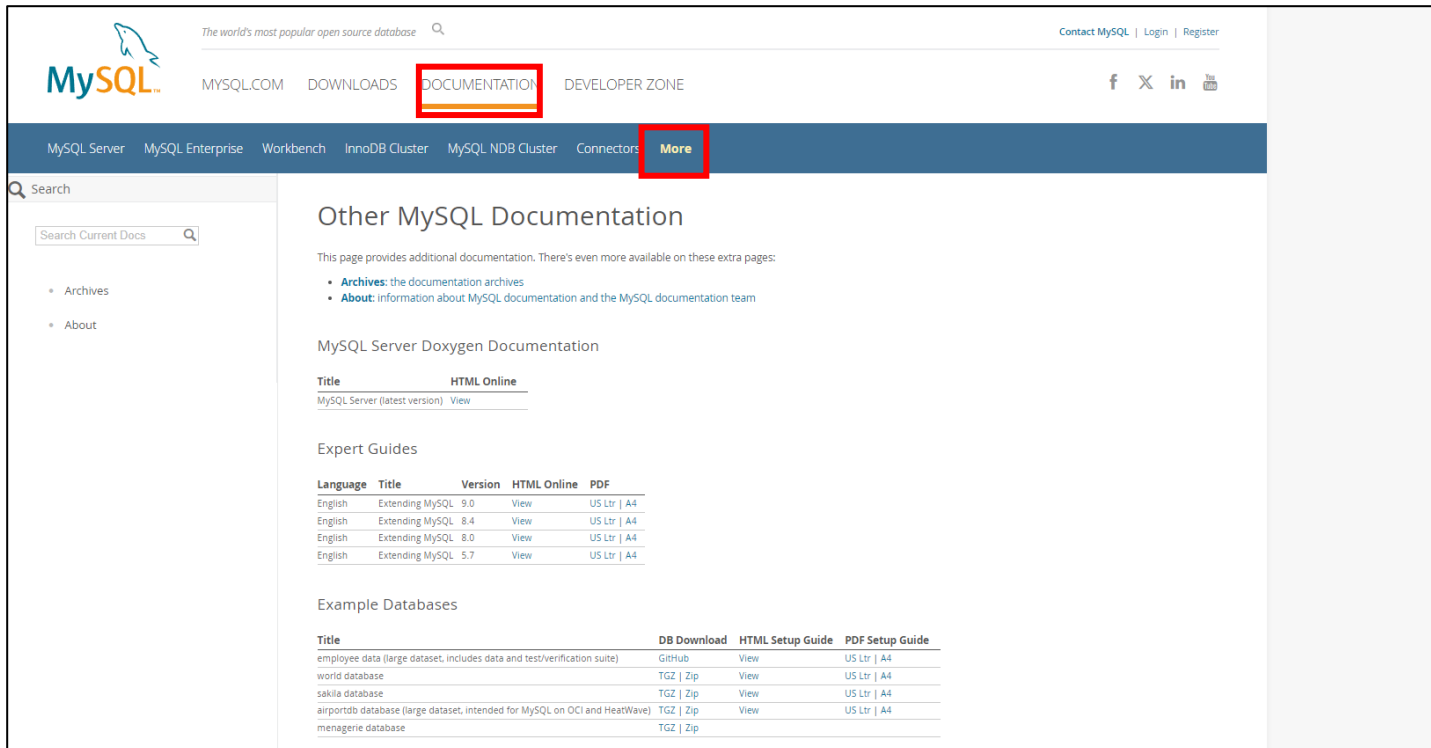
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> |
```

샘플 데이터 다운로드

✓ world 데이터베이스 다운로드

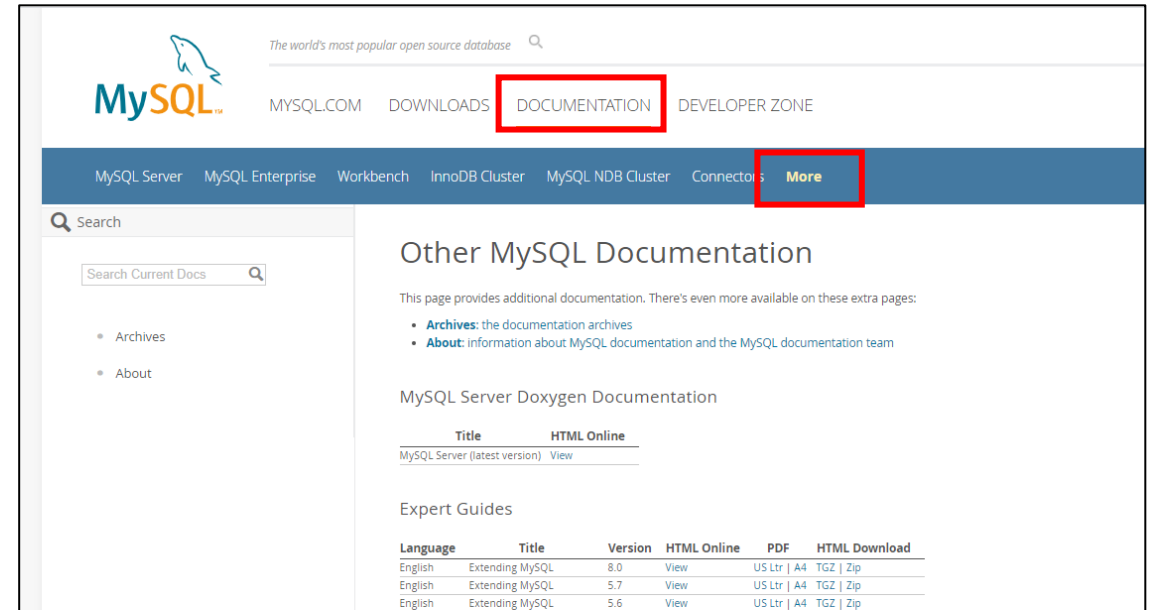
- 쿼리문을 연습하기 좋은 데이터베이스
- <https://www.mysql.com/>
공식사이트 > document > More 클릭
- 다운로드 후 압축 풀기



샘플 데이터 다운로드

employees 데이터베이스 다운로드

- 쿼리문을 연습하기 좋은 데이터베이스
- <https://www.mysql.com/>
공식사이트 > document > More 클릭



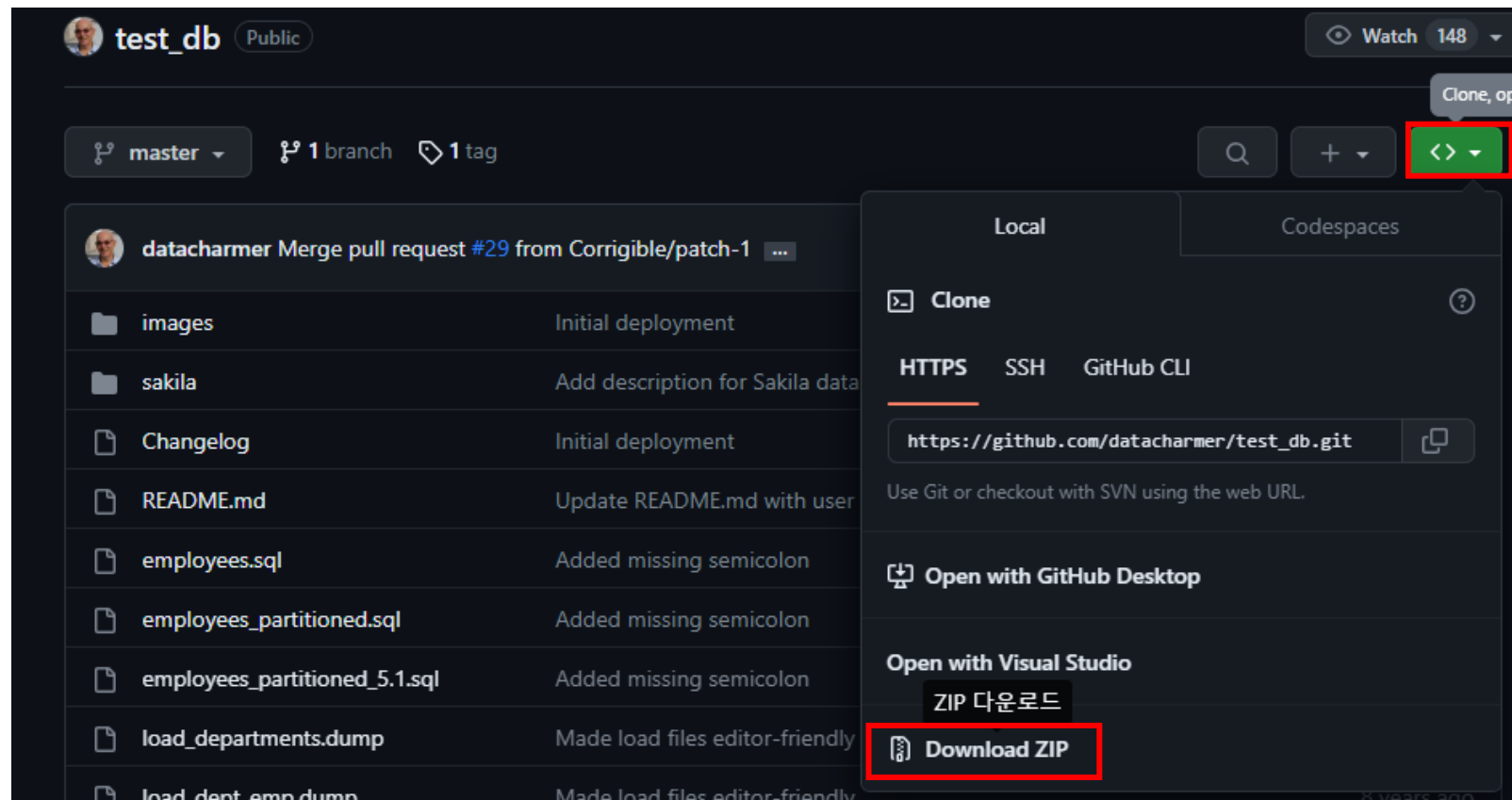
Example Databases

Title	Download DB	HTML Setup Guide	PDF Setup Guide
employee data (large dataset, includes data and test/verification suite)	GitHub	View	US Ltr A4
world database	Gzip Zip	View	US Ltr A4
world_x database	TGZ Zip	View	US Ltr A4
sakila database	TGZ Zip	View	US Ltr A4
menagerie database	TGZ Zip		

샘플 데이터 다운로드

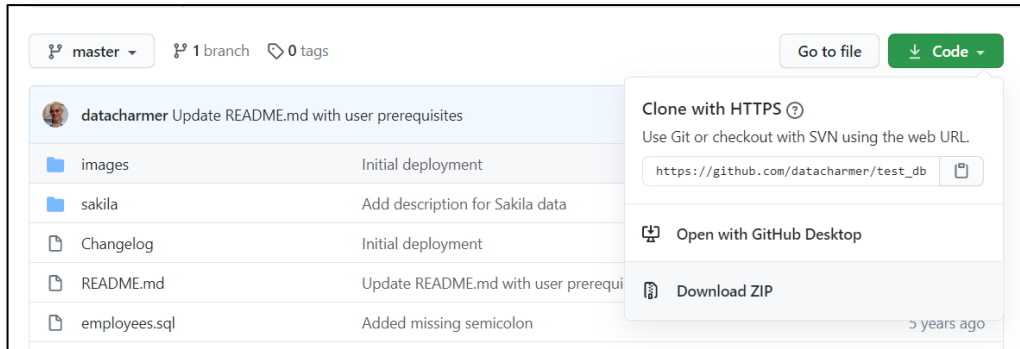
✓ ZIP 다운로드 진행

- 다운로드 후 압축 풀기



다운로드 및 스크립트 실행

- ✓ mysql console에서 Script를 수행한다.
 - git clone을 다운로드 > 압축해제
 - CMD 창 띄우고, 해당 폴더로 이동하기



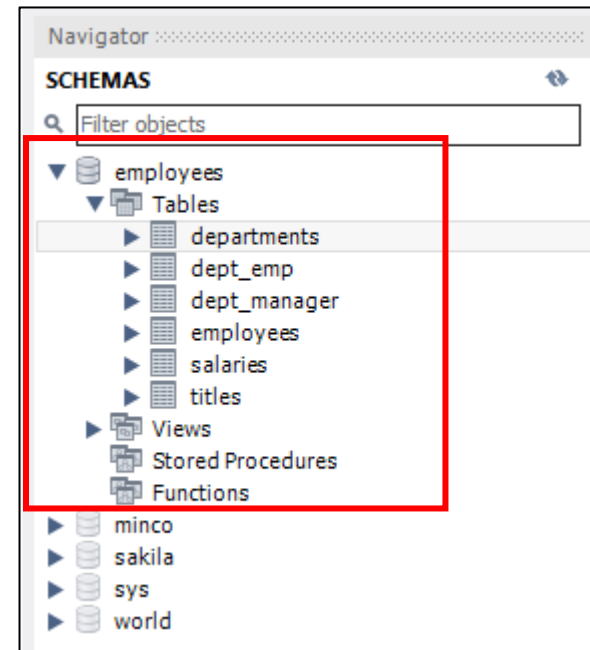
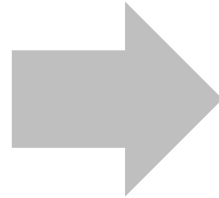
```
C:\Users\minco>cd C:\Users\minco\Downloads\test_db-master\test_db-master
C:\Users\minco\Downloads\test_db-master\test_db-master>
```

샘플 데이터 설치

root 계정으로 script를 수행한다.

- 약 1분 소요
- 환경변수가 정상적으로 등록된 경우에만 사용가능

```
$ mysql -u root -p < employees.sql  
Enter password: ****
```



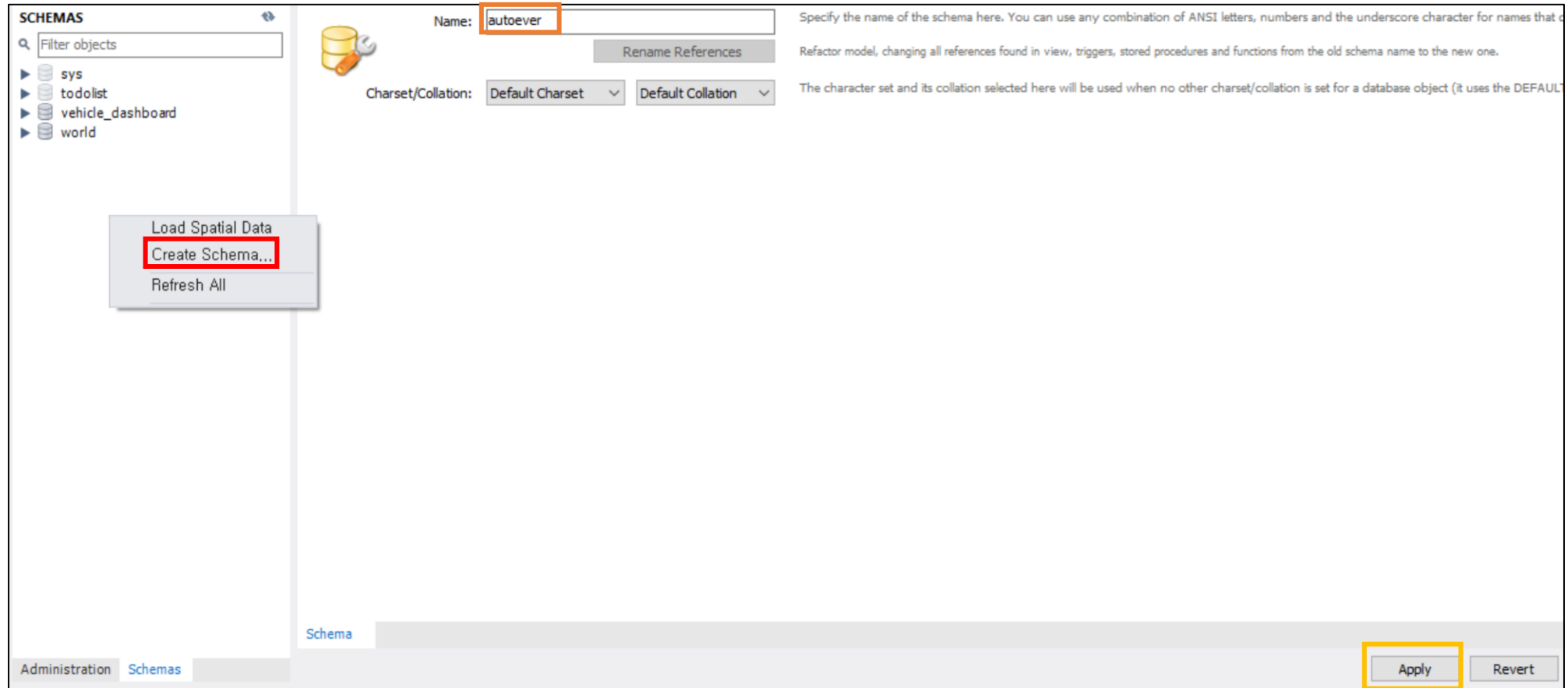
Chapter 3

SQL 실습

SQL 프로그래밍

schema 생성

✓ 마우스 우클릭 – CreateSchema... 클릭 -> autoever 입력 -> Apply 클릭



schema 생성

✓ Apply 클릭 -> Finish 클릭

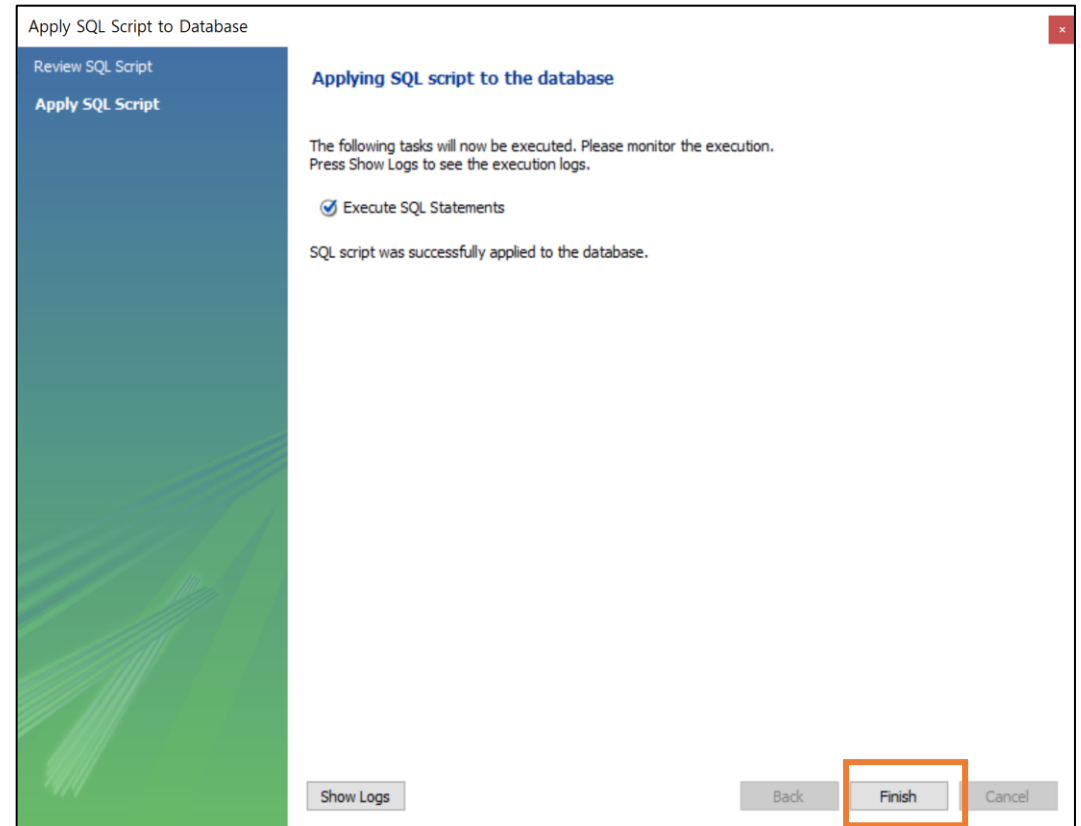
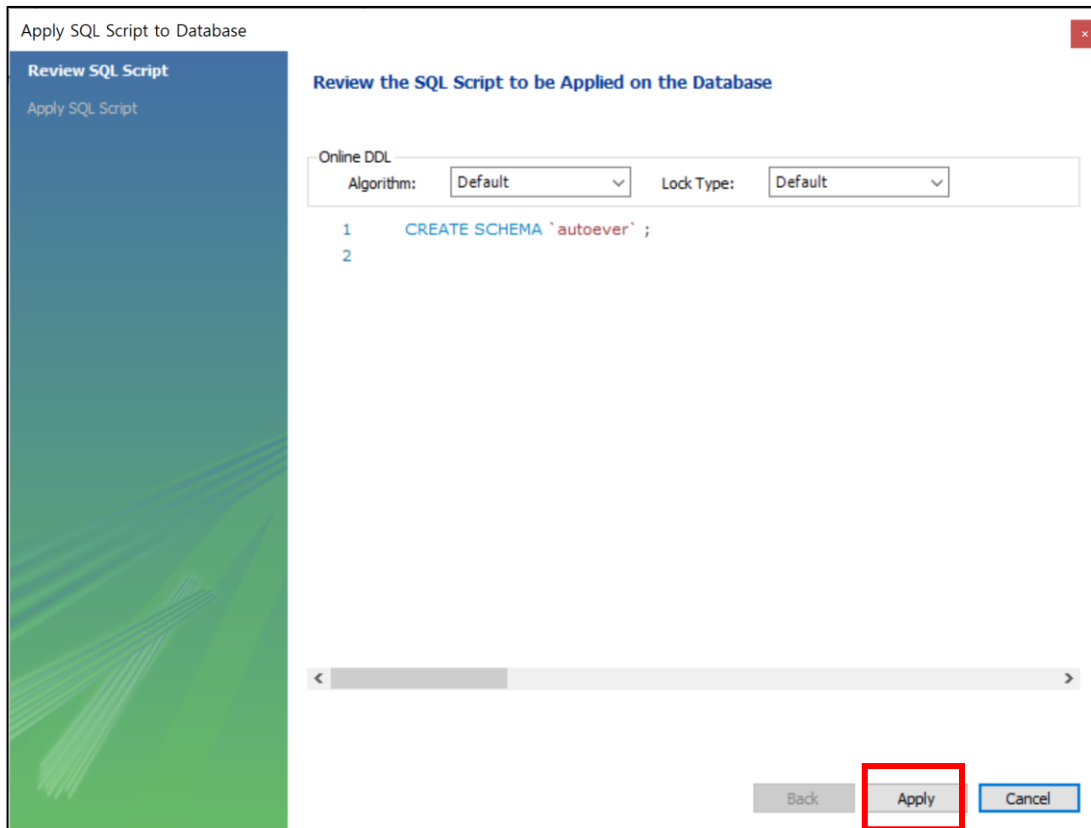


table 생성

- ✓ members 테이블을 생성
 - PK : Primary Key (주키)
 - NN : Not Null
 - AI : Auto Increment

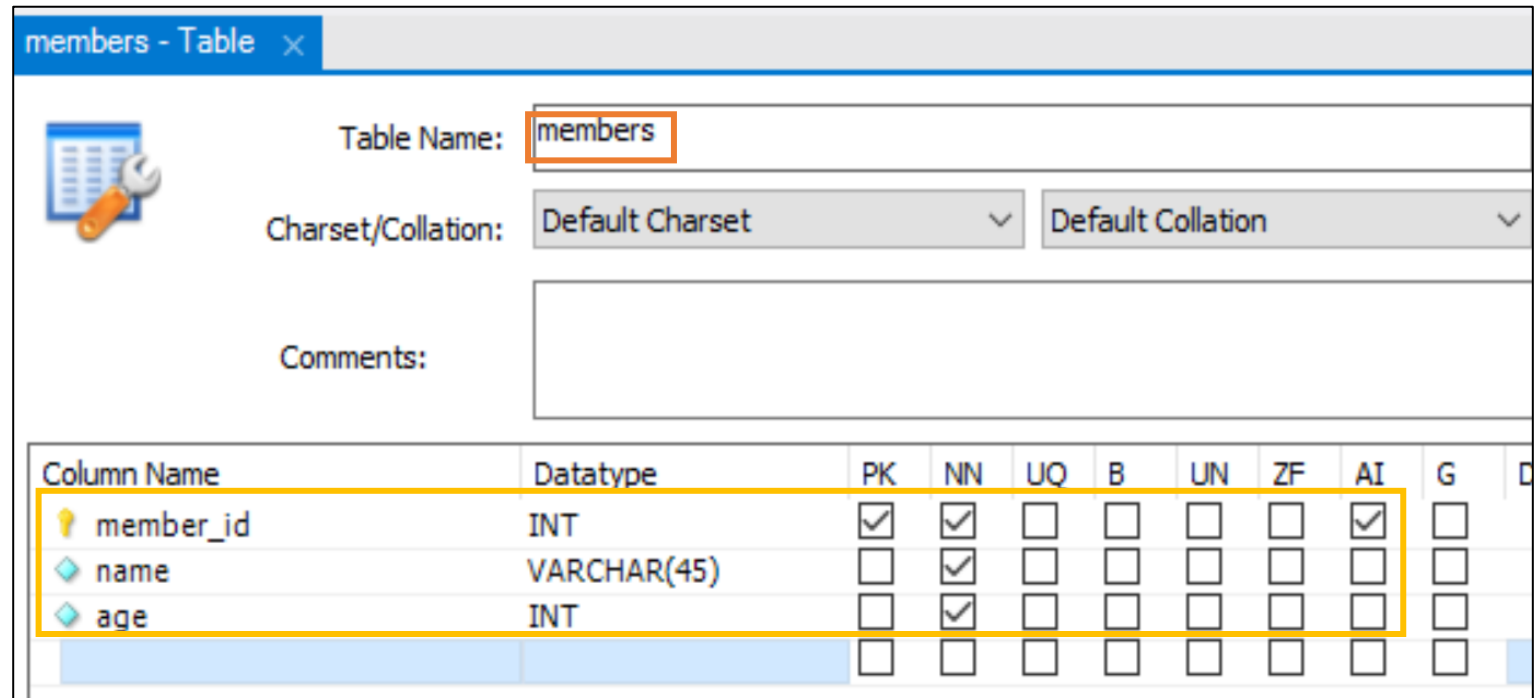
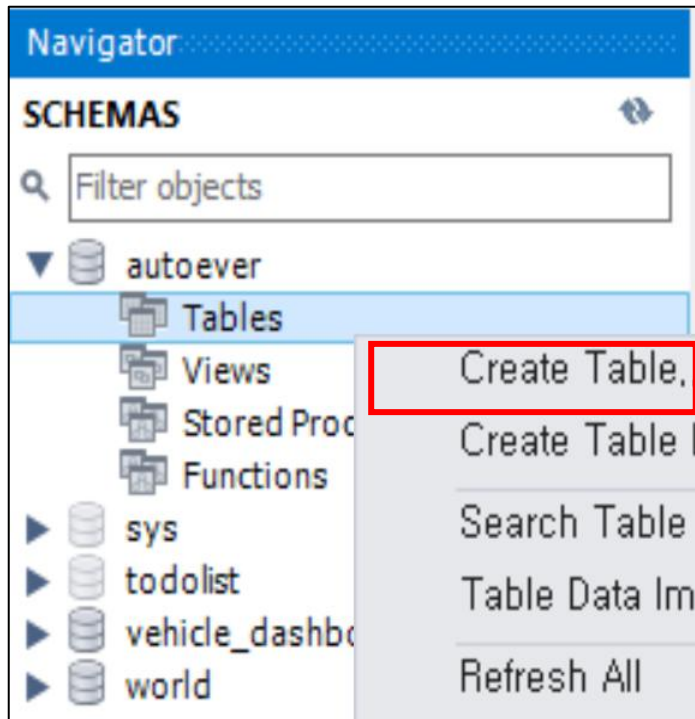


table 생성

✓ CREATE 쿼리문 확인

Apply SQL Script to Database

Review SQL Script
Apply SQL Script

Review the SQL Script to be Applied on the Database

Online DDL
Algorithm: Lock Type:

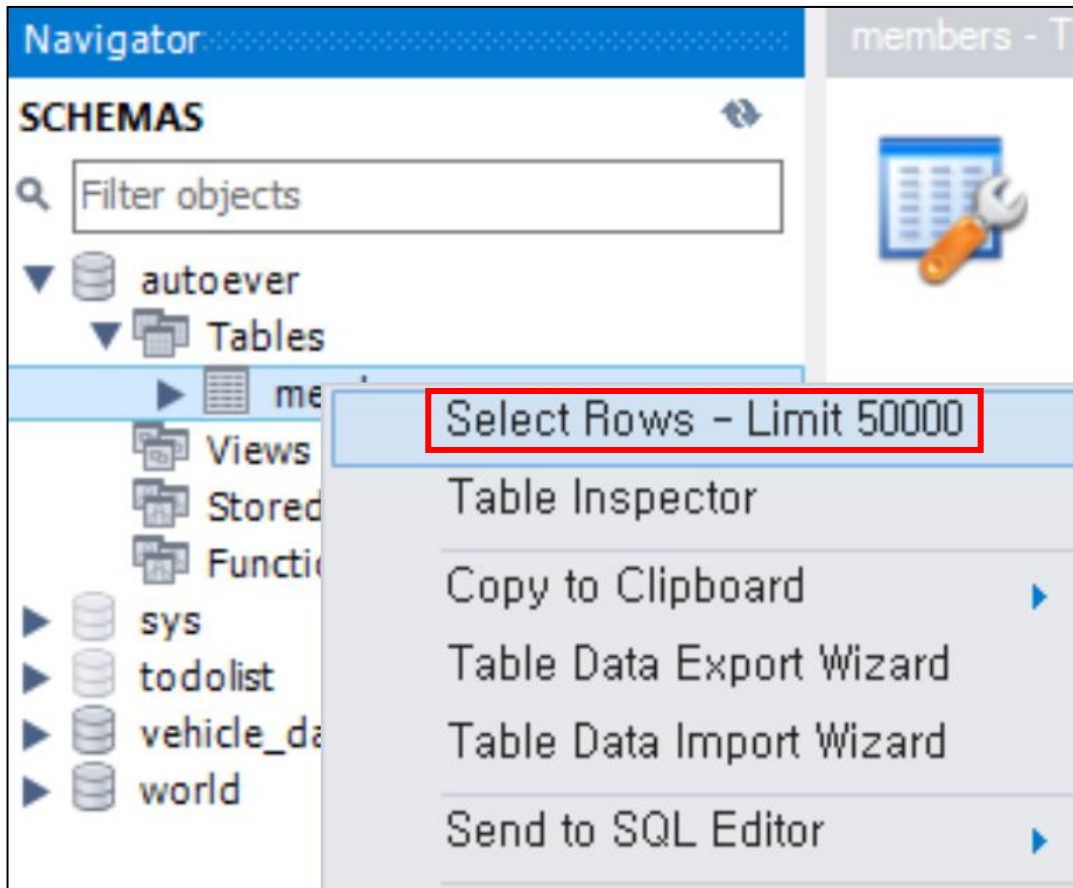
```
1 CREATE TABLE `autoever`.`members` (  
2   `member_id` INT NOT NULL AUTO_INCREMENT,  
3   `name` VARCHAR(45) NOT NULL,  
4   `age` INT NOT NULL,  
5   PRIMARY KEY (`member_id`));  
6
```

< >

Back Apply Cancel

생성된 table 확인

- ✓ Navigator 에서 생성된 member 테이블을 확인 해보자
 - Select Rows 클릭
 - 생성된 Table 확인 가능



```
SELECT * FROM autoever.members;
```

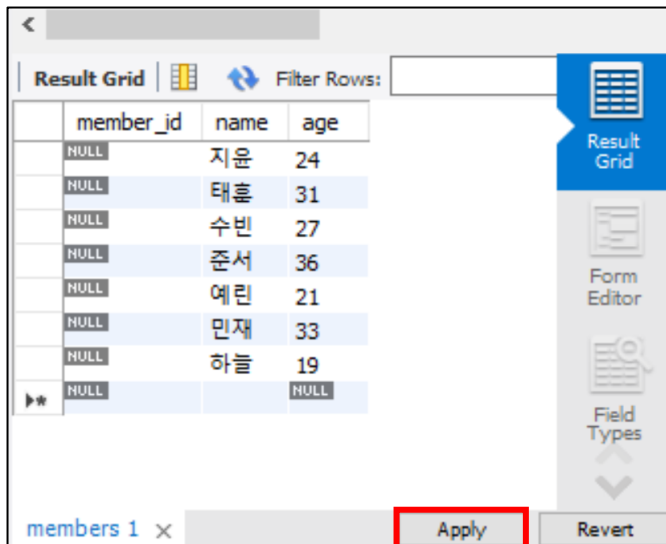
자동으로 작성된 쿼리문 확인

	member_id	name	age
*	NULL	NULL	NULL

샘플 데이터 채우기

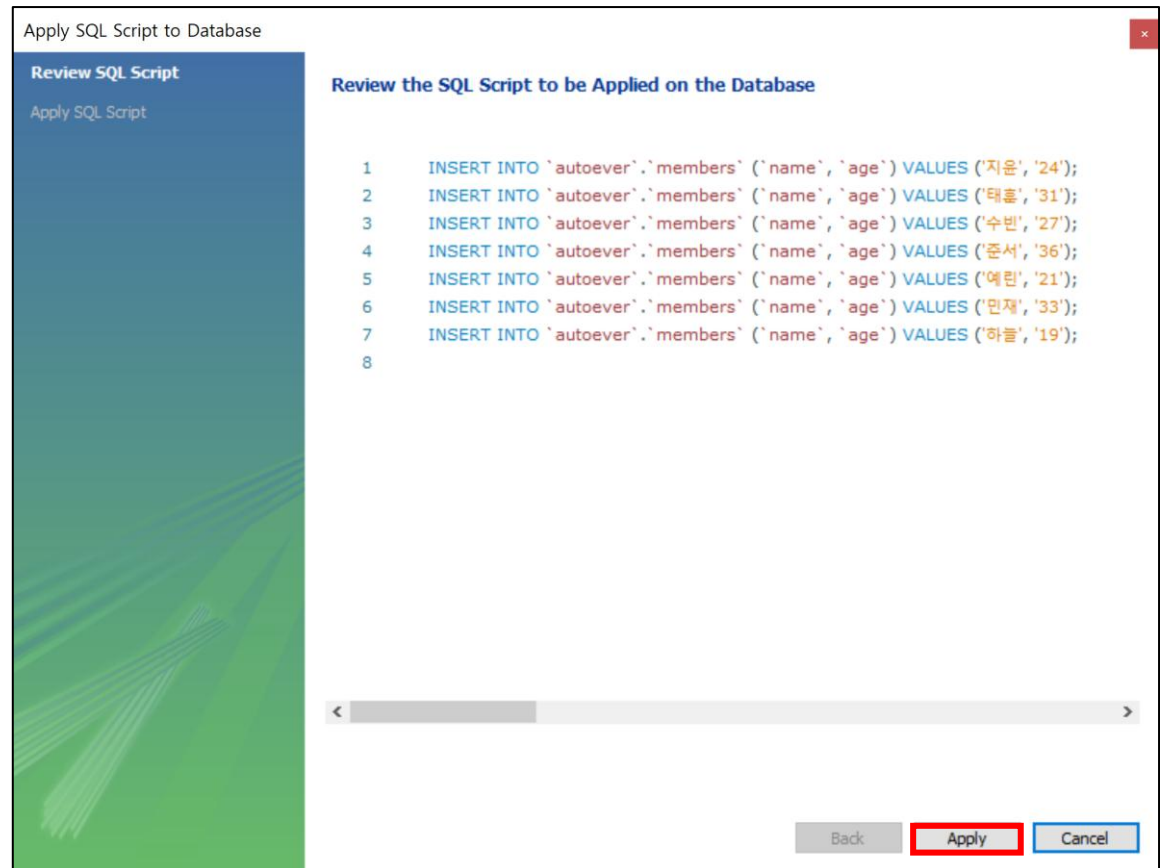
✓ 샘플 데이터를 입력 후 Apply 클릭

- member_id 는 Auto Increment 이므로 입력 안함
- 자동으로 생성된 INSERT INTO 쿼리문 확인



member_id	name	age
NULL	지윤	24
NULL	태훈	31
NULL	수빈	27
NULL	준서	36
NULL	예린	21
NULL	민재	33
NULL	하늘	19
NULL		NULL

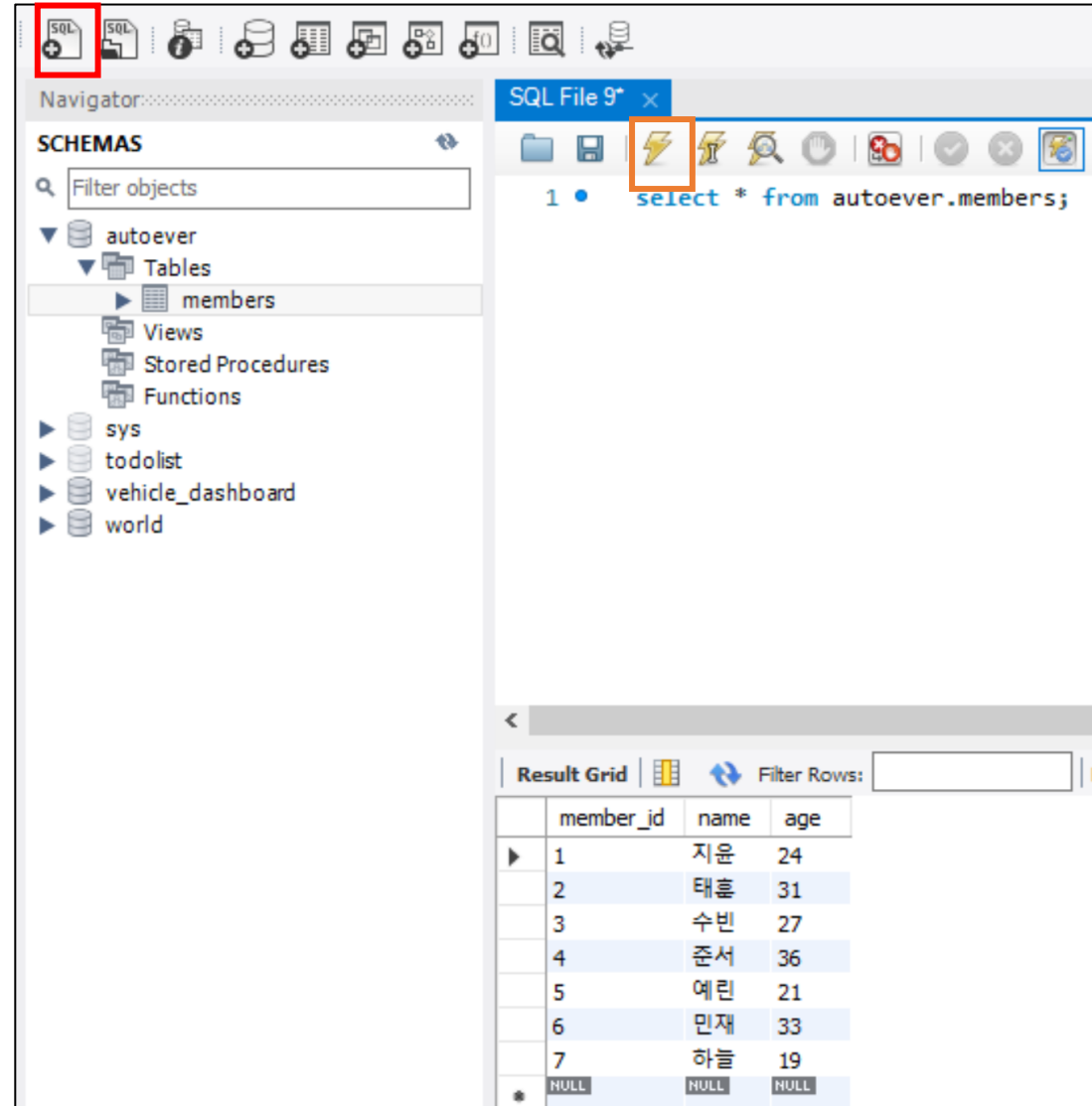
members 1 x Apply Revert



적용 확인하기

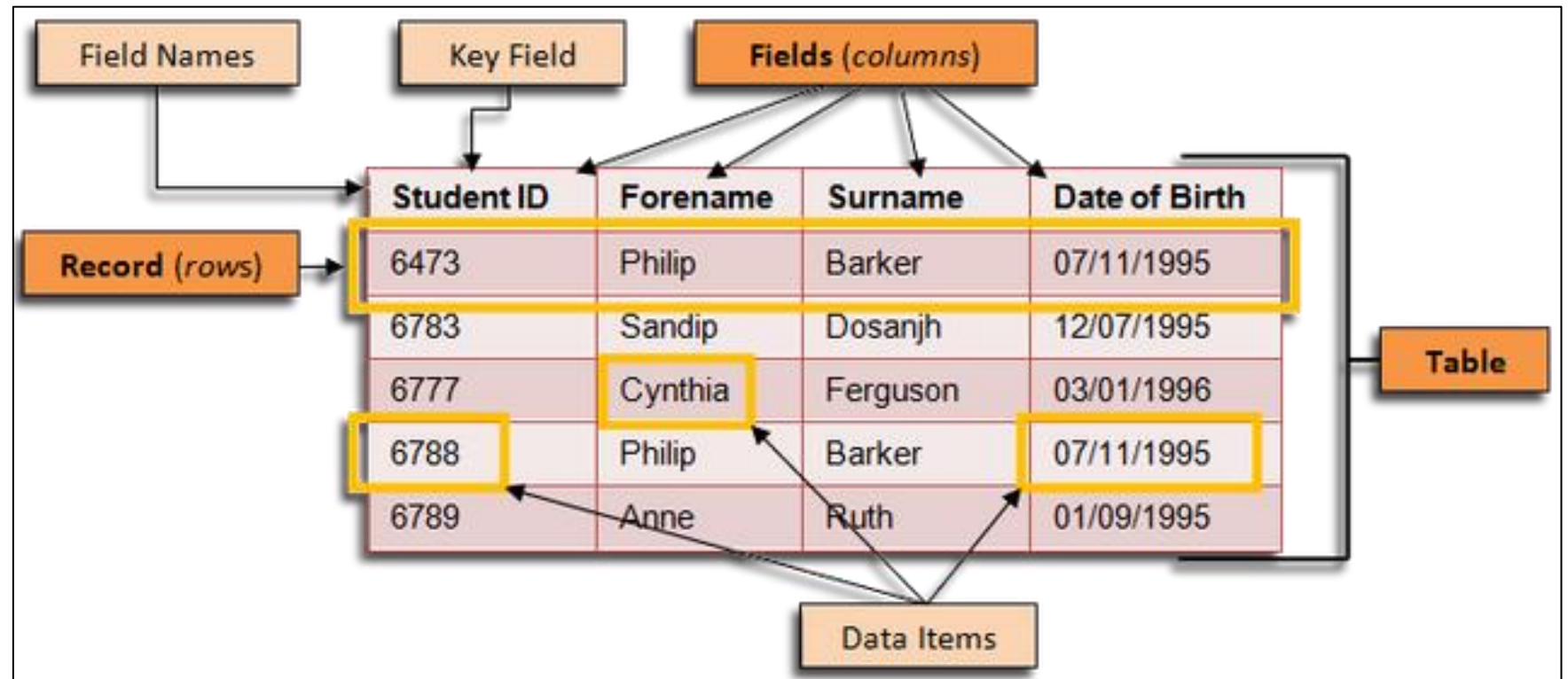
✓ SQL File을 생성

- select * from autoever.members 입력
- 실행 단축키 : Ctrl + Enter



용어 정리

- ✓ Field (columns)
- ✓ Record (rows)
- ✓ Items



SELECT

- ✓ SELECT 필드명 FROM 테이블

	member_id	name	age
▶	1	지운	24
	2	태훈	31
	3	수빈	27
	4	준서	36
	5	예린	21
	6	민재	33
	7	하늘	19

SELECT * FROM autoever.members;

	name	age
▶	지운	24
	태훈	31
	수빈	27
	준서	36
	예린	21
	민재	33
	하늘	19

SELECT name, age from autoever.members;

WHERE

- ✓ SELECT * FROM TABLE WHERE 조건1 AND 조건2 AND 조건3 ...
- 특정 조건에 해당하는 레코드만 출력할 때 사용
 - 특정 필드만 출력 : SELECT [필드] FROM TABLE
 - 특정 레코드만 출력 : SELECT * FROM TABLE WHERE [레코드]

	member_id	name	age
▶	3	수빈	27
	4	준서	36
	5	예린	21
★	NULL	NULL	NULL

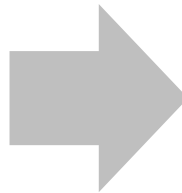
```
SELECT * FROM autoever.members  
WHERE member_id >= 3 and member_id <= 5;
```

	member_id	name	age
▶	2	태훈	31
★	NULL	NULL	NULL

```
SELECT * FROM autoever.members  
WHERE name='태훈';
```

- ✓ alias 지정하여 필드 이름 변경
 - SELECT 필드명 AS [별칭] FROM TABLE

Result Grid	
	name
▶	지윤
	태훈
	수빈
	준서
	예린
	민재
	하늘



Result Grid	
	이름
▶	지윤
	태훈
	수빈
	준서
	예린
	민재
	하늘

```
SELECT name FROM autoever.members;
```

```
SELECT name as '이름' FROM autoever.members;
```


ORDER BY

✓ 특정 필드를 정렬한다

- 오름차순 : `SELECT * FROM TABLE ORDER BY [필드명]`
- 내림차순 : `SELECT * FROM TABLE ORDER BY [필드명] DESC;`

	member_id	name	age
▶	7	하늘	19
	5	예린	21
	1	지윤	24
	3	수빈	27
	2	태훈	31
	6	민재	33
	4	준서	36
*	NULL	NULL	NULL

	member_id	name	age
▶	4	준서	36
	6	민재	33
	2	태훈	31
	3	수빈	27
	1	지윤	24
	5	예린	21
	7	하늘	19
*	NULL	NULL	NULL

`SELECT * FROM autoever.members ORDER BY age;` `SELECT * FROM autoever.members ORDER BY age DESC;`

LIMIT

- ✓ 원하는 데이터 개수만큼만 가져온다.
 - LIMIT 가져올 개수 / LIMIT 건너뛸 개수, 가져올 개수

```
SELECT * FROM autoever.members LIMIT 3;
```

```
SELECT * FROM autoever.members LIMIT 3, 3;
```

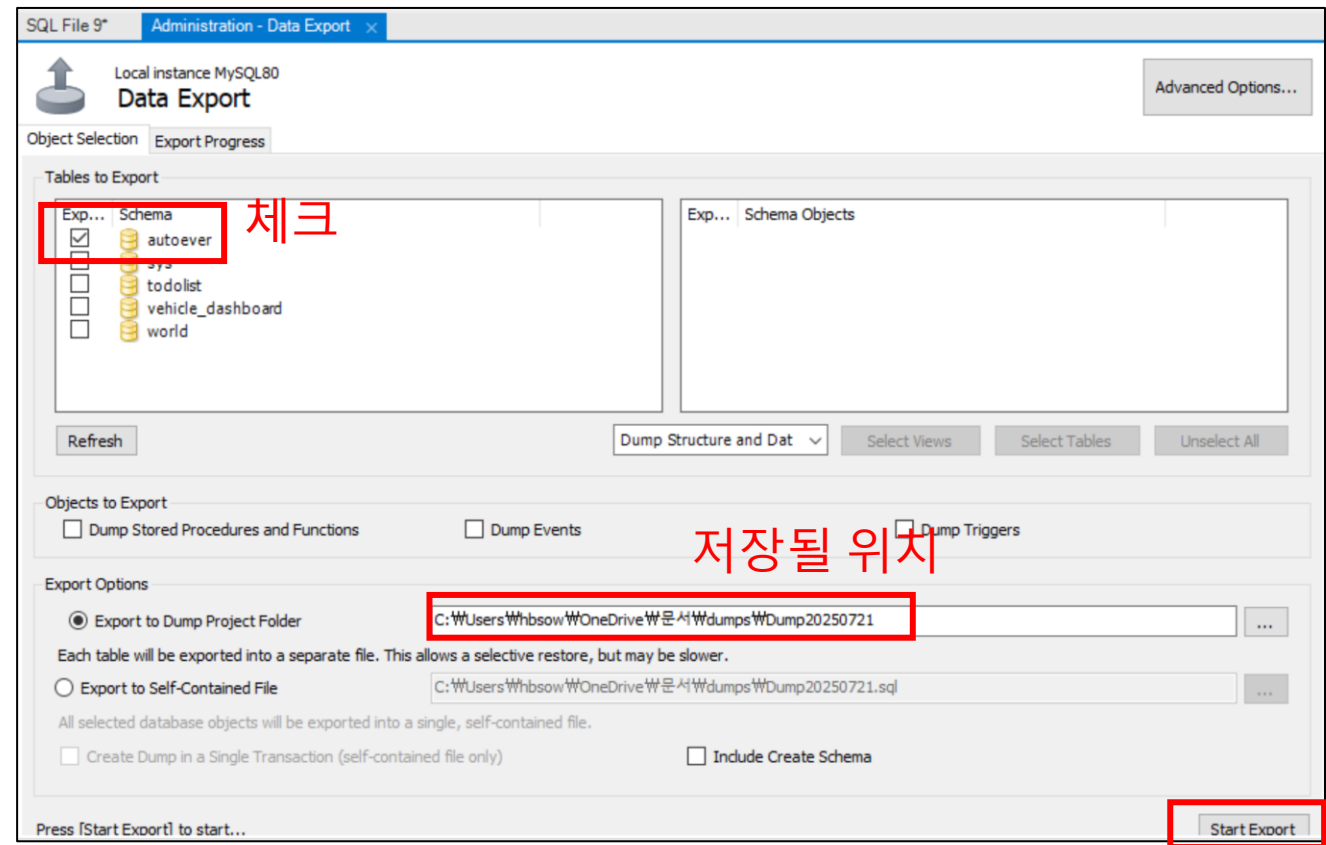
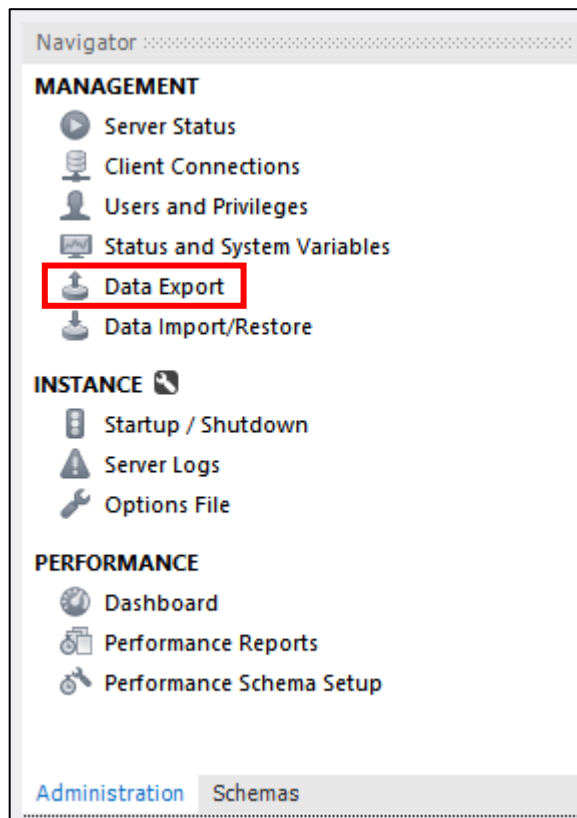
Result Grid  Filter Rows: 			
	member_id	name	age
▶	1	지윤	24
	2	태훈	31
	3	수빈	27
●	NULL	NULL	NULL

Result Grid  Filter Rows: 			
	member_id	name	age
▶	4	준서	36
	5	예린	21
	6	민재	33
●	NULL	NULL	NULL

DB backup

✓ Data Export

- 현재 Data를 외부 파일로 추출한다.



DB restore 준비

✓ 데이터 변경

- 데이터를 변경한 후, Restore 해보자
 - Members Table에서 레코드 추가 및 제거
 - 새로운 Table 추가하기

Result Grid

Filter Rows:

Edit:

Export/Import

	member_id	name	age
	1	지윤	24
	2	태훈	31
	3	수빈	27
	4	준서	36
	5	예린	21
	6	민재	33
	7	하늘	19
	NULL	NULL	NULL

Open Value in Editor

Set Field to NULL

Mark Field Value as a Function/Literal

Delete Row(s)

Load Value From File...

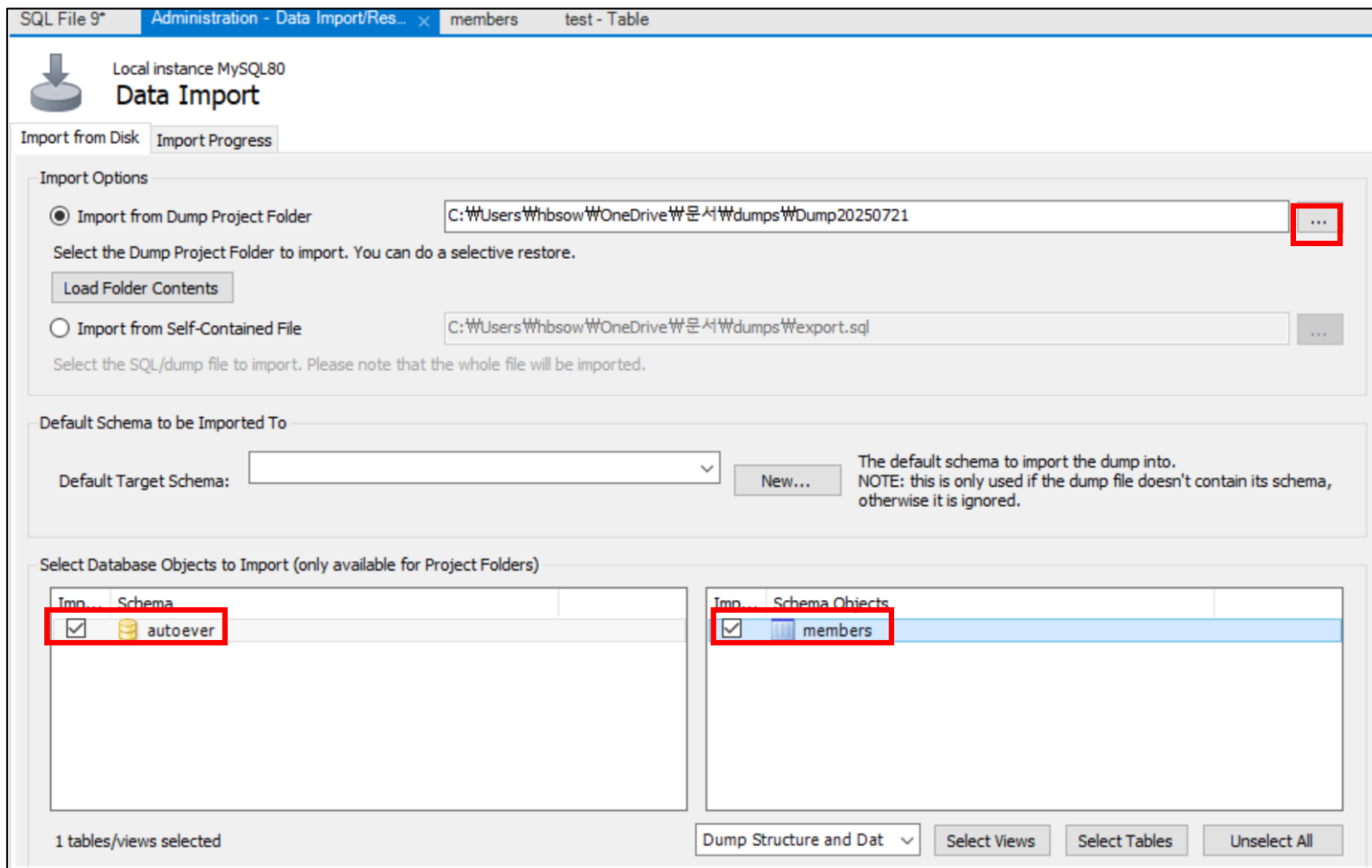
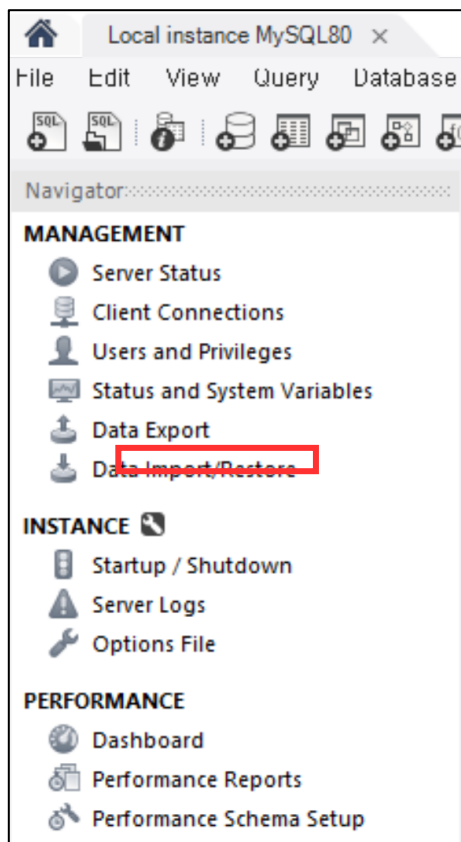
Save Value To File...

[illegible]

DB restore

✓ 데이터 복구 하기

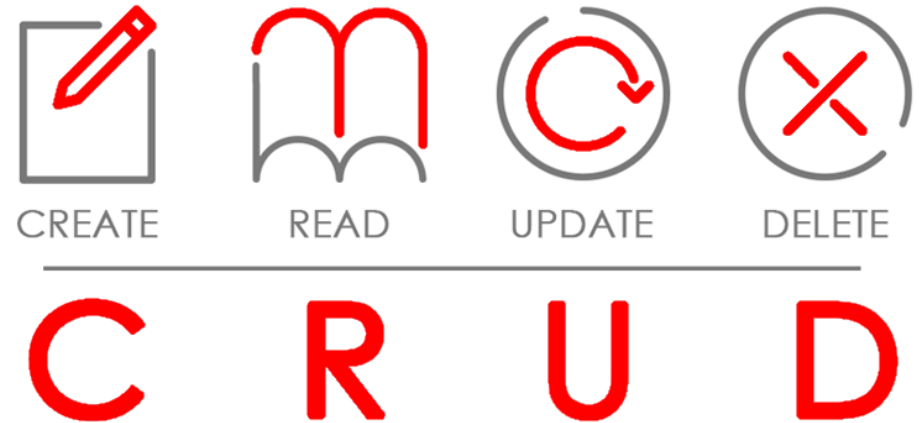
- 제거된 데이터는 복구가 된다.
- 복구 전 추가된 Table은 삭제하지 않는다.



CRUD

✓ CRUD : Create + Read + Update + Delete

- 데이터를 다루는 Software의 기본적인 인터페이스를 뜻함
- Database에서 CRUD
 - C : INSERT INTO ~ VALUES
 - R : SELECT
 - U : UPDATE ~ SET WHERE
 - D : DELETE FROM ~ WHERE



Web에서 CRUD

✓ 웹서비스에서 해당 Query를 쓰는 이유에 대해 생각해보자.

- 로그인 : SELECT
- 회원가입 : SELECT, INSERT, UPDATE
- 회원탈퇴 : DELETE
- 게시판은 CRUD 전체 사용

Insert

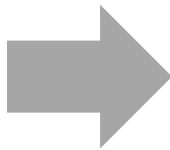
✓ INSERT INTO

- INSERT INTO 테이블명 (`컬럼명`, `컬럼명`, `컬럼명`) VALUES (값1, 값2, 값3)
- 값 ('김다운', 23세)를 추가
- 값 ('최태호', 38세)을 추가
- PK 는 기입하지 않음

INSERT INTO autoever.members (`name`, `age`) value('다운', 23);

INSERT INTO autoever.members (`name`, `age`) value('태호', 38);

	member_id	name	age
▶	1	지운	24
	2	태훈	31
	3	수빈	27
	4	준서	36
	5	예린	21
	6	민재	33
	7	하늘	19
*	NULL	NULL	NULL



Result Grid Filter Rows:			
	member_id	name	age
▶	1	지운	24
	2	태훈	31
	3	수빈	27
	4	준서	36
	5	예린	21
	6	민재	33
	7	하늘	19
	8	다운	23
	9	태호	38
*	NULL	NULL	NULL

Update

✓ UPDATE 테이블 SET 필드=값 WHERE 레코드 조건

- 수정, 삭제 시 WHERE 은 필수 기입!
- '태호' 나이를 37세로 수정하자.
- '다은'의 이름을 '가은' 으로 수정하자.

```
UPDATE autoever.members SET age=37 WHERE member_id=9;
```

```
UPDATE autoever.members SET name= '가은' WHERE member_id=8;
```

```
SELECT * FROM autoever.members;
```

Result Grid   Filter Rows			
	member_id	name	age
	1	지운	24
	2	태호	31
	3	수빈	27
	4	준서	36
	5	예린	21
	6	민재	33
	7	하늘	19
	8	가은	23
	9	태호	37
*	NULL	NULL	NULL

Delete

✓ DELETE FROM 테이블 WHERE 레코드 조건

- 8번 레코드 삭제
- 9번 레코드 삭제

```
DELETE FROM autoever.members WHERE member_id=8;  
DELETE FROM autoever.members WHERE member_id=9;  
SELECT * FROM autoever.members;
```

Result Grid  Filter Rows: 			
	member_id	name	age
▶	1	지윤	24
	2	태훈	31
	3	수빈	27
	4	준서	36
	5	예린	21
	6	민재	33
	7	하늘	19
✱	NULL	NULL	NULL

정리

✓ CRUD 명령어 Format을 기억 해두자

- INSERT INTO 테이블 (`컬럼명`, ...) VALUES(값, ...)
- UPDATE 테이블 SET 필드 WHERE
- DELETE FROM 테이블 WHERE

✓ DB 관리도구

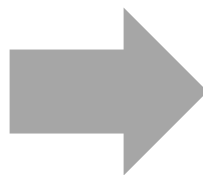
- GUI 환경 : MySQL Workbench
 - 대량 데이터에도, 구조를 쉽게 파악할 수 있다.
 - 사용하기 편리한 장점
- CLI 환경 : MySQL Console
 - Linux + DB Server + SSH 에서 DB 초기설정에 사용된다.
 - Workbench 없이 간단한 데이터 조회 시에도 사용

사용할 수 있는 DB 확인하기

✓ SHOW DATABASES / USE [DB명]

- 현 계정이 접근 권한이 있는 Database 목록을 확인한다.
- 사용할 Databases 선택 명령어 : use [DB이름]

```
mysql> show databases;  
+-----+  
| Database  
+-----+  
| autoever  
| information_schema  
| mysql  
| performance_schema  
| sys  
| todolist  
| vehicle_dashboard  
| world  
+-----+  
8 rows in set (0.01 sec)
```



```
mysql> use autoever;  
Database changed
```

테이블 정보 확인

- ✓ SHOW TABLES;
 - 테이블 목록 확인
- ✓ DESC [테이블명]
 - 테이블 구조 확인

```
mysql> show tables;
```

Tables_in_autoever
members
test

```
2 rows in set (0.01 sec)
```

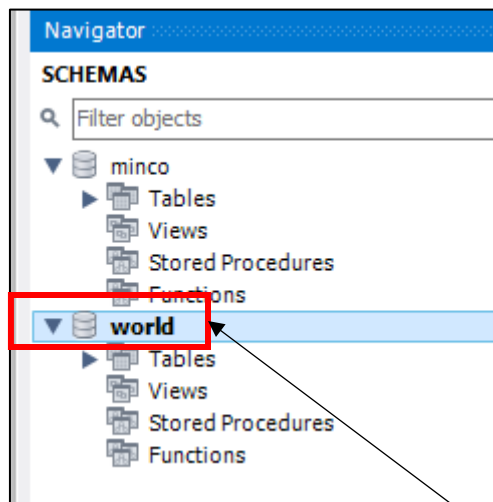
```
mysql> DESC members;
```

Field	Type	Null	Key	Default	Extra
member_id	int	NO	PRI	NULL	auto_increment
name	varchar(45)	NO		NULL	
age	int	NO		NULL	

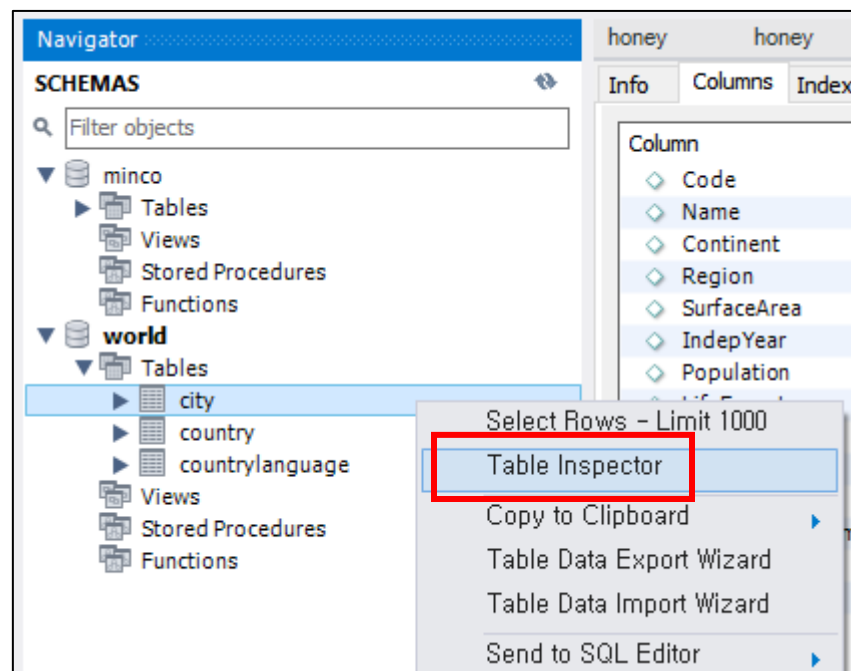
```
3 rows in set (0.01 sec)
```

Select 심화

- ✓ 샘플 스키마 구조 확인



더블 클릭시
자동으로 use world;
명령어가 수행된다.



Desc [테이블] 정보 확인

- ✓ Columns 구조를 확인할 수 있다.

world.city x						
Info Columns Indexes Triggers Foreign keys Partitions Grants DDL						
Column	Type	Default Value	Nullable	Character Set	Collation	Privileges
◇ ID	int		NO			select,insert,update,references
◇ Name	char(35)		NO	latin1	latin1_swedish_ci	select,insert,update,references
◇ CountryCode	char(3)		NO	latin1	latin1_swedish_ci	select,insert,update,references
◇ District	char(20)		NO	latin1	latin1_swedish_ci	select,insert,update,references
◇ Population	int	0	NO			select,insert,update,references

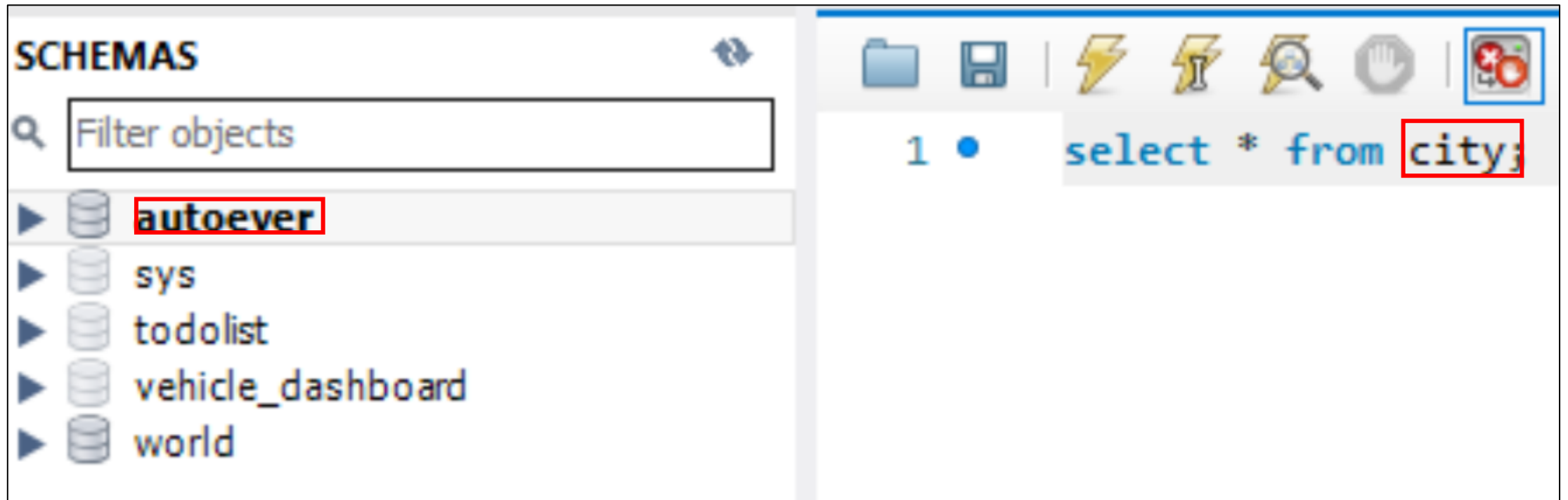
```
mysql> DESC city;
```

Field	Type	Null	Key	Default	Extra
ID	int	NO	PRI	NULL	auto_increment
Name	char(35)	NO			
CountryCode	char(3)	NO	MUL		
District	char(20)	NO			
Population	int	NO		0	

```
5 rows in set (0.01 sec)
```


다른 스키마 사용시 유의사항

- ✓ world 스키마의 city table을 query 하기 위해서 USE 필수
 - 아래 그림과 같이 world가 굵은 글씨가 아닌 상태에서, 에러가 발생한다.
 - "world" 에 더블클릭 해주면 된다.



DB KEY - PK

✓ PK(Primary Key)

한 테이블 내에서 중복되지 않는 값만 가질수 있는 키

유일한 값

중복 허용 X

NULL 값 X

menus			
id	name	description	image_src
1	카페라떼	라떼는.. 말이야	public/image1.jpg
2	카푸치노	언빌리 "버블"	public/image2.jpg
3	아메리카노	아메아메 좋아	public/image3.jpg
4	복숭아 아이스	상큼한 복숭아!	public/image4.jpg

DB KEY - FK

✓ FK (Foreign Key)

특정 테이블에 포함되어 있으면서 다른테이블의 기본키로 지정된 키
기본적으로는 다른 테이블의 기본 키를 참조한다.

NULL, 중복 허용 OK

menus			
id	name	description	image_src
1	카페라떼	라떼는.. 말이야	public/image1.jpg
2	카푸치노	언빌리 "버블"	public/image2.jpg
3	아메리카노	아메아메 좋아	public/image3.jpg
4	복숭아 아이스	상큼한 복숭아!	public/image4.jpg

orders			
id	quantity	request_detail	menus_id
1	1	얼음 많이많이 주세요.	1
2	3	시럽 넣지 말아주세요.	2
3	1	우유 많이 주세요.	2
4	2	복숭아 좋아요.	4

- ✓ 관계형 데이터 베이스의 핵심
데이터베이스들을 관계를 맺어 데이터를 검색하는 방법
PK, FK로 두 테이블을 연결한다.

테이블을 나눠야 하는 이유

✓ 왜 테이블을 둘로 나뉘었을까?

- 다음 상황을 가정해보자.
- 만약 100,000 개의 데이터를 저장한다면, 중복이 지나치게 많아 매우 비효율적
DB 용량 증가, 속도 저하
비용 증가: Oracle 등의 유료 데이터베이스를 사용한다면, 훨씬 많은 비용지불
만약, menu_description 을 바꾸고 싶다면? 100,000 개의 데이터에서 찾아 전부 바꿔야한다

id	name	description	quantity	request_detail
1	카페라떼	라떼는.. 말이야	1	얼음 많이많이 주세요.
2	카페라떼	라떼는.. 말이야	3	시럽 넣지 말아주세요.
3	아메리카노	아메아메 좋아	1	
4	복숭아 아이스티	상큼한 복숭아!	2	

테이블을 나눠야 하는 이유

✓ 관계형 데이터베이스 (Relational Database)

- 여러 테이블의 "관계" 로 이루어진다.
- 핵심은 FK (Foreign Key, 외래키) 와 JOIN
- 중복 제거

만약 100,000 개의 데이터를 저장하더라도,

DB 용량을 훨씬 효율적으로 사용

만약, menu_description 을 바꾸고 싶다면? menus 테이블에서 "단 하나만" 변경하면 끝

menus			
id	name	description	image_src
1	카페라떼	라떼는.. 말이야	public/image1.jpg
2	카푸치노	언빌리 "버블"	public/image2.jpg
3	아메리카노	아메아메 좋아	public/image3.jpg
4	복숭아 아이스	상큼한 복숭아!	public/image4.jpg

orders			
id	quantity	request_detail	menus_id
1	1	얼음 많이많이 주세요.	1
2	3	시럽 넣지 말아주세요.	2
3	1	우유 많이 주세요.	2
4	2	복숭아 좋아요.	4

DB 설계

✓ JOIN 문법을 사용하는 이유

- orders 에서, 주문마다 menus_id 보유
- 즉, id 만 알고 있다면 menus 에 접근해서
메뉴 이름, 메뉴 설명, 메뉴 이미지에 대한 정보를 가져올 수 있음
이를 위해 **JOIN 문법** 사용

menus			
id	name	description	image_src
1	카페라떼	라떼는.. 말이야	public/image1.jpg
2	카푸치노	언빌리 "버블"	public/image2.jpg
3	아메리카노	아메아메 좋아	public/image3.jpg
4	복숭아 아이스	상큼한 복숭아!	public/image4.jpg

orders			
id	quantity	request_detail	menus_id
1	1	얼음 많이많이 주세요.	1
2	3	시럽 넣지 말아주세요.	2
3	1	우유 많이 주세요.	2
4	2	복숭아 좋아요.	4

JOIN 사용

✓ JOIN

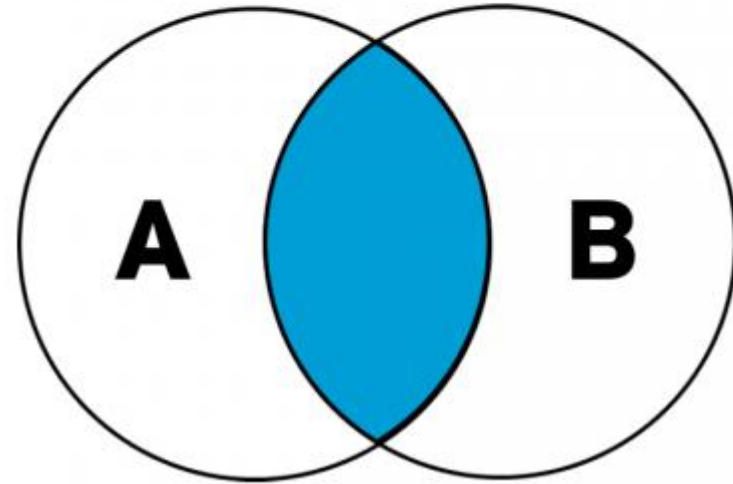
- SELECT <열 목록>
 - FROM <첫번째 테이블>
 <조인의 종류> <두번째 테이블>
ON <조인될 조건>
[WHERE 검색조건]

JOIN 종류 - INNER

✓ INNER JOIN

```
SELECT *  
FROM A a  
INNER JOIN B b  
ON a.id = b.id
```

INNER JOIN



JOIN 종류 - INNER

- ✓ 각 사원들의 사원번호, first_name, 현재 받고있는 급여액(salary) 가져오기

```
SELECT employees.emp_no, first_name, salary
FROM employees
INNER JOIN salaries
ON employees.emp_no = salaries.emp_no;
```

	emp_no	first_name	salary
▶	10001	Georgi	60117
	10001	Georgi	62102
	10001	Georgi	66074
	10001	Georgi	66596
	10001	Georgi	66961
	10001	Georgi	71046
	10001	Georgi	74333
	10001	Georgi	75286
	10001	Georgi	75994
	10001	Georgi	76884
	10001	Georgi	80013
	10001	Georgi	81025
	10001	Georgi	81097
	10001	Georgi	84917

BETWEEN

✓ 구간 사이 데이터 추출

- WHERE 절 다음 입력한다.
- `population >= 500 AND population <= 900` 과 동일하다.

```
SELECT * FROM city  
WHERE population BETWEEN 500 AND 900;
```

	ID	Name	CountryCode	District	Population
▶	62	The Valley	AIA	—	595
	1791	Flying Fish Cove	CXR	—	700
	2316	Bantam	CCK	Home Island	503
	2728	Yaren	NRU	—	559
	2805	Alofi	NIU	—	682
	2806	Kingston	NFK	—	800
*	NULL	NULL	NULL	NULL	NULL

IN

- ✓ 후보들 중 해당하는 row 찾기
 - WHERE 절 다음 입력한다.
 - 필드명 IN (후보 Item1, 후보 Item2, 후보 Item3 ..)

```
SELECT * FROM city  
WHERE name IN('Seoul', 'Sydney', 'Oxford');
```

	ID	Name	CountryCode	District	Population
▶	130	Sydney	AUS	New South Wales	3276207
	498	Oxford	GBR	England	144000
	2331	Seoul	KOR	Seoul	9981619
*	NULL	NULL	NULL	NULL	NULL

LIKE

✓ 문자열 검색

- WHERE 절 다음에 사용
- % : 다중 문자를 의미한다. (wildcard 문자)
- _ : 한 글자를 의미한다.

```
SELECT * FROM city  
WHERE name LIKE 'New%';
```

	ID	Name	CountryCode	District	Population
▶	137	Newcastle	AUS	New South Wales	270324
	482	Newcastle upon Tyne	GBR	England	189150
	502	Newport	GBR	Wales	139000
	734	Newcastle	ZAF	KwaZulu-Natal	222993
	1106	New Bombay	IND	Maharashtra	307297
	1109	New Delhi	IND	Delhi	301297
	3793	New York	USA	New York	8008278
	3823	New Orleans	USA	Louisiana	484674

```
SELECT * FROM city  
WHERE countrycode LIKE 'K_R';
```

	ID	Name	CountryCode	District	Population
▶	2255	Bikenibeu	KIR	South Tarawa	5055
	2256	Bairiki	KIR	South Tarawa	2226
	2331	Seoul	KOR	Seoul	9981619
	2332	Pusan	KOR	Pusan	3804522
	2333	Inchon	KOR	Inchon	2559424
	2334	Taegu	KOR	Taegu	2548568
	2335	Taejon	KOR	Taejon	1425835
	2336	Kwangju	KOR	Kwangju	1368341

GROUP BY / HAVING

✓ GROUP BY

- 그룹을 지어 데이터를 묶는다.
 - 집계 함수를 사용하기 위해 묶는다.
 - SUM() / AVG()
 - MIN() / MAX()
 - COUNT()

```
SELECT countrycode, sum(population) AS '총인구'  
FROM city  
GROUP BY countrycode;
```

	countrycode	총인구
▶	ABW	29034
	AFG	2332100
	AGO	2561600
	AIA	1556
	ALB	270000
	AND	21189
	ANT	2345
	ARE	1728336
	ARG	19996563
	ARM	1633100
	ASM	7523

```
SELECT countrycode, count(population) AS '도시수'  
FROM city  
GROUP BY countrycode;
```

	countrycode	도시수
▶	ABW	1
	AFG	4
	AGO	5
	AIA	2
	ALB	1
	AND	1
	ANT	1
	ARE	5
	ARG	57
	ARM	3

GROUP BY / HAVING

✓ HAVING

- GROUP BY을 썼을 때 쓸 수 있는 조건 절
 - 집계함수에 대해 조건을 걸 수 있음

```
SELECT countrycode, count(population) AS '도시수' FROM city  
GROUP BY countrycode  
HAVING count(population) > 200;
```

	CountryCode	도시수
▶	BRA	250
	CHN	363
	IND	341
	JPN	248
	USA	274

도전

1. SQL 연습 – “autoever.members” 라는 하나의 Table Query
2. SQL 연습 – “world” Database 의 다량의 Table Query
3. W3Schools 서비스를 이용한 SQL 연습
4. 프로그래머스 SQL 고득점 kit (solvesql)

도전1 Members - 1

✓ 다음 미션을 순서대로 수행하자

1. 모든 필드 출력하되 + id가 2, 4번 레코드만 출력하기
2. id, age 필드 출력하고, $1 \leq id \leq 3$ 번 레코드만 출력하기 (조건에 AND 사용)
3. 나이가 30대인 멤버들의 모든 컬럼 출력
4. id는 홀수이면서, age는 짝수인 멤버들의 이름만 출력
5. id 을 "번호", name을 "성함", age를 "나이" 로 출력하되 나이를 기준으로 오름차순

도전1 Members - 2

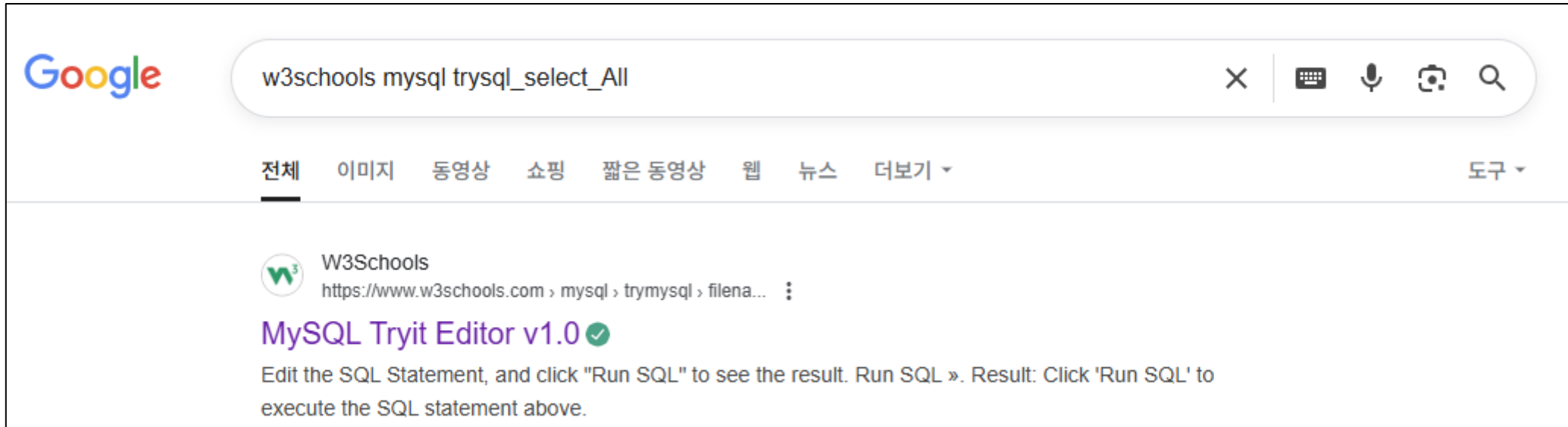
6. 이름을 가나다순으로 정렬하고, 이름과 나이 만 출력
7. id 1 번과 3 번 레코드의 나이를 모두 33로 수정
8. 5 번 레코드 삭제
9. 레코드 추가 (정선, 32)
10. 나이가 가장 많은 멤버의 이름과 나이를 출력

✓ 다음 미션을 순서대로 수행하자

1. 인구수가 1000명 미만이면서, A로 시작하는 도시 찾기
2. 일본 (JPN) 에서, 인구수가 100만명 ~ 200만명 인 도시를 찾아 도시 이름과, 국가 코드 (JPN) 필드만 출력하기
3. kim 이라는 글자가 포함된 도시명 찾기
4. world 스키마의 레코드 총 개수 출력
5. 하위 5개의 도시 코드, 인구수 출력
6. TOP 5개의 국가 코드, 인구수 출력

도전3

- ✓ 접속하기 https://www.w3schools.com/mysql/trymysql.asp?filename=trysql_select_all



도전3

- ✓ 실습 환경 테스트 (대소문자 구분 필수)
 - MySQL 버전에 따라서, 대소문자 지원여부가 달라짐
 - 우리가 AWS 에 설치한 버전은 대소문자 구분 안함

```
SELECT * FROM OrderDetails;
```

Number of Records: 2155

OrderDetailID	OrderID	ProductID	Quantity
1	10248	11	12
2	10248	42	10
3	10248	72	5
4	10249	14	9
5	10249	51	40
6	10250	41	10
7	10250	51	35
8	10250	65	15

다음 미션을 순서대로 수행하자

1. CustomerID 33초과이면서, EmployeeID 5 미만 출력
2. OrderID 10,000 미만 또는 ShipperID 5 미만 출력
1. OrderDetails의 ProductID가 14보다 큰 모든 필드 출력
2. CustomerName, city 필드 출력 $10 < \text{CustomerID} \leq 50$ 레코드출력 (Customers에 데이터 존재)
3. Orders에 CustomerId 가 30번대만 출력 (모든 필드 출력, and 사용)
4. ProductID가 짝수이면서, SupplierId 홀수인 ProductName과 ProductID 출력 (Products)

7. Customers에 Address를 "주소", city를 "도시", PostalCode를 "우편번호" 로 출력, 우편번호 내림차순으로 정렬
8. OrderDetails에 OrderDetailID를 기준으로 내림차순으로 정렬하고, 5개만 출력하기 (모든 컬럼)
9. Orders에 모든 컬럼을 OrderID를 기준으로 내림차순 정렬후, 3개를 건너뛰고 7개만큼 출력하기
10. OrderDetail 에서 다음 alias 지정
OrderDetailID => 상세주문번호
OrderID => 주문번호
ProductID => 상품번호
Quantity => 수량

OrderID 10,000 이하 또는 ProductID 50 이상을 출력하되,
OrderDetailID 기준 내림차순 정렬하며,
8개만 출력

11. OrderDetails 의 모든 컬럼을 출력하되, ProductID 가 1 로 시작하는 모든 레코드 출력
12. OrderDetails 의 모든 컬럼을 출력하되, ProductID 의 십의자리가 1인 모든 레코드 출력

도전4

✓ 브라우저에 프로그래머스 검색



교육데브코스 **부트캠프**코딩테스트인증시험리눅스 자격증

블로그기업서비스로그인

코딩테스트 문제

스킬체크

모든 문제

기초 문제

입문 문제

알고리즘 고득점 Kit

SQL 고득점 Kit

과제테스트

Q&A



나의 코딩테스트 연습 활동 내역을 살펴보세요!

매일 매일 도전하며 실력을 쌓아나가요!

코딩테스트 활동 내역 보기 >

SQL 고득점 Kit

코딩 테스트에서 SQL문제를 만날 확률이 25%나 된다는 사실, 알고 계셨나요?

이제 프로그래머스에서 SQL 쿼리도 연습하세요!

기초적인 SELECT문부터 어려운 JOIN문까지. 실습 문제를 통해 차근차근 SQL 기본기를 다져보세요.



SELECT

조건에 맞는 레코드를, 원하는 순서대로. 기본기를 처음부터 탄탄히 다져보세요.

출제 빈도

낮음 ↘

평균 점수

높음 ↗

문제 세트

0 / 33

SUM, MAX, MIN

나이가 가장 많은 사람은 누구일까? 테이블을 뒤져 통계를 내어봅시다.

출제 빈도

보통 →

평균 점수

높음 ↗

문제 세트

0 / 10

GROUP BY

나랑 이름이 같은 사람은 몇 명일까요? 데이터를 묶고 평균값을 계산해보세요.

출제 빈도

보통 →

평균 점수

보통 →

문제 세트

0 / 24

IS NULL

값이 없음을 의미하는 NULL. NULL을 처리하는 방법을 배워봅시다.

출제 빈도

낮음 ↘

평균 점수

보통 →

문제 세트

0 / 8

JOIN

테이블 사이의 복잡한 관계를 파악해보아요. 이제부터 어렵습니다.

출제 빈도

높음 ↗

평균 점수

낮음 ↘

문제 세트

0 / 12

String, Date

숫자는 어떻게 쓰는 건지 알겠는데, 글자와 날짜는 어떻게 다루지?

출제 빈도

낮음 ↘

평균 점수

낮음 ↘

문제 세트

0 / 19

Chapter 04

Spring boot 개요

SQL 프로그래밍

교육 내용 및 학습 목표

- ✓ Spring Boot Framework 이해
 - Spring Boot project 구현

Spring Framework

- ✓ Java 를 사용하는 웹 애플리케이션 프레임워크
- ✓ Java 진영에서 많이 사용되는 Backend 프레임워크



Spring Framework

✓ 전자정부 표준프레임워크(eGovFrame)

제조 대기업과 금융권(은행,증권사) -> 안정성 / 트랜잭션 관리 우선

IT 빅테크(네이버,카카오,쿠팡,배달의민족,당근마켓,토스) -> MSA의 엔진은 스프링 부트



Spring의 어려운 점

- ✓ XML 언어를 사용해, 어려운 설정 파일을 개발자가 직접 작성
- ✓ Spring Application 실행을 위해, HTTP 서버 설치 필요
- ✓ 프로젝트 디렉터리 구조를 직접 만들어야 함
- ✓ 필요한 라이브러리 의존성을 개발자가 직접 관리해야 함

Spring의 보다 더 쉬운 Spring boot

✓ Spring 을 더 쉽게 사용하도록 만든 도구

- 자동 설정 : 복잡한 설정 파일을 개발자가 작성할 필요가 없음
- 내장 서버 : Tomcat 서버가 내장되어 있기 때문에, 즉시 Spring Application 실행 가능
- 기본 구조 : 프로젝트 디렉터리 구조를 제공함(JAR,WAR)
- 의존성 관리 : 자동으로 해주기 때문에, 개발자는 필요한 라이브러리를 설치해서 사용하기만 하면 됨
- 생산성 증가 : 개발자는 오로지 개발에만 집중



Chapter 05

개발 환경 세팅

Spring Boot를 활용한 백엔드 개발

Spring initializr

<https://start.spring.io>

- ✓ Project : Maven
- ✓ Language : Java
- ✓ Spring Boot : 3.5.11 (SNAPSHOT x)
- ✓ Packaging : Jar
- ✓ Configuration : Properties
- ✓ Java : 17

26년 2월기준

Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT)

☐ 4.1.0 (M2)

☐ 4.0.4 (SNAPSHOT)

☐ 4.0.3

☐ 3.5.12 (SNAPSHOT)

☒ 3.5.11

Project Metadata

Group

Artifact

Name

Description

Package name

Packaging

☒ Jar

☐ War

Configuration

☒ Properties

☐ YAML

Java

☐ 25

☐ 21

☒ 17

Spring initializr

Dependencies(의존성) 필요시 설치

- Spring web
- Spring Boot DevTools
- MySQL Driver
- MyBatis Framework
- **H2 Database(임시)**
- **Thymeleaf(임시)**

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Boot DevTools DEVELOPER TOOLS

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

MySQL Driver SQL

MySQL JDBC driver.

MyBatis Framework SQL

Persistence framework with support for custom SQL, stored procedures and advanced mappings. MyBatis couples objects with stored procedures or SQL statements using a XML descriptor or annotations.

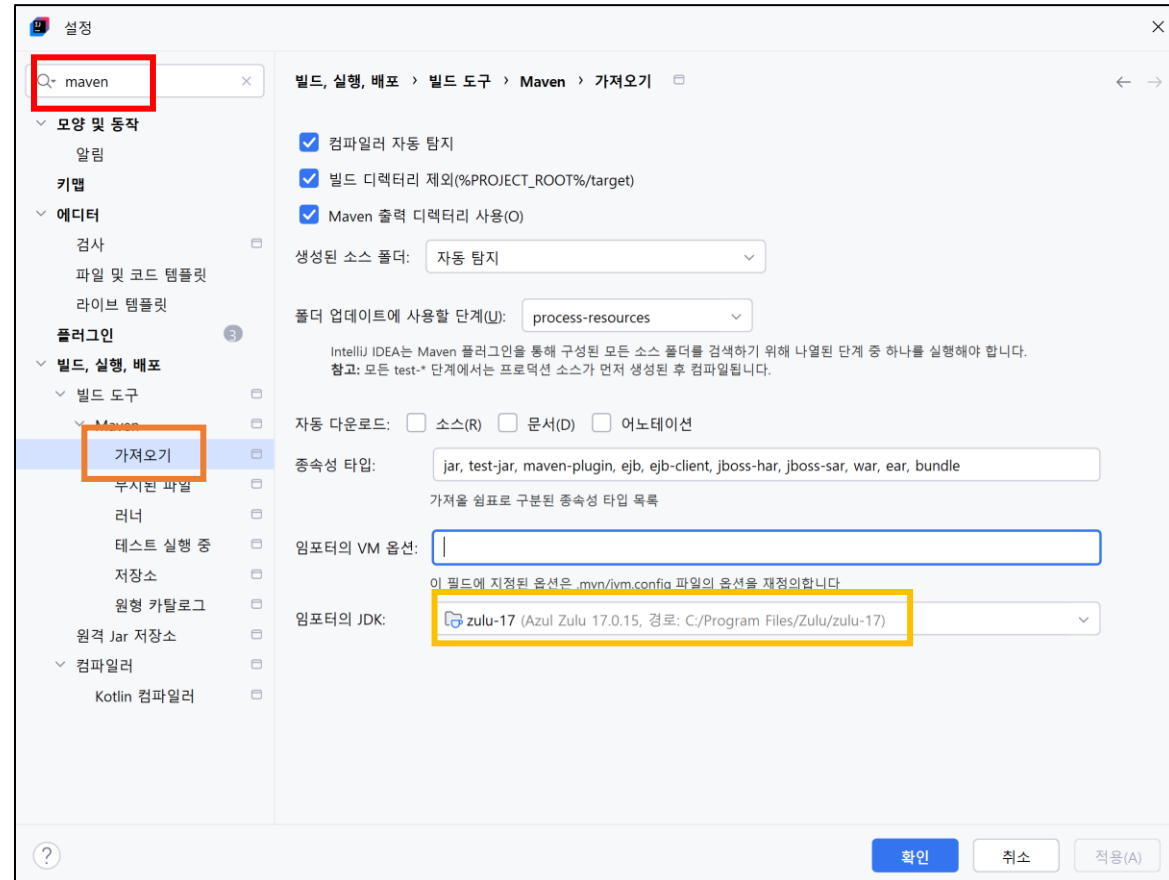
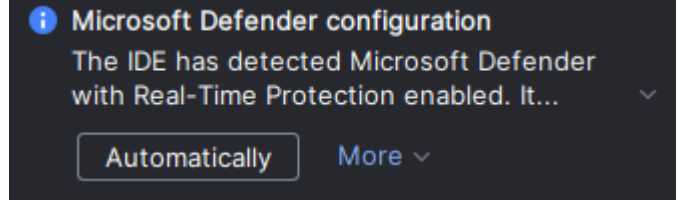
프로젝트 열기

- IntelliJ 화면에서 Open 클릭
- Open as Project – 체크박스 클릭한 후 , Trust Project



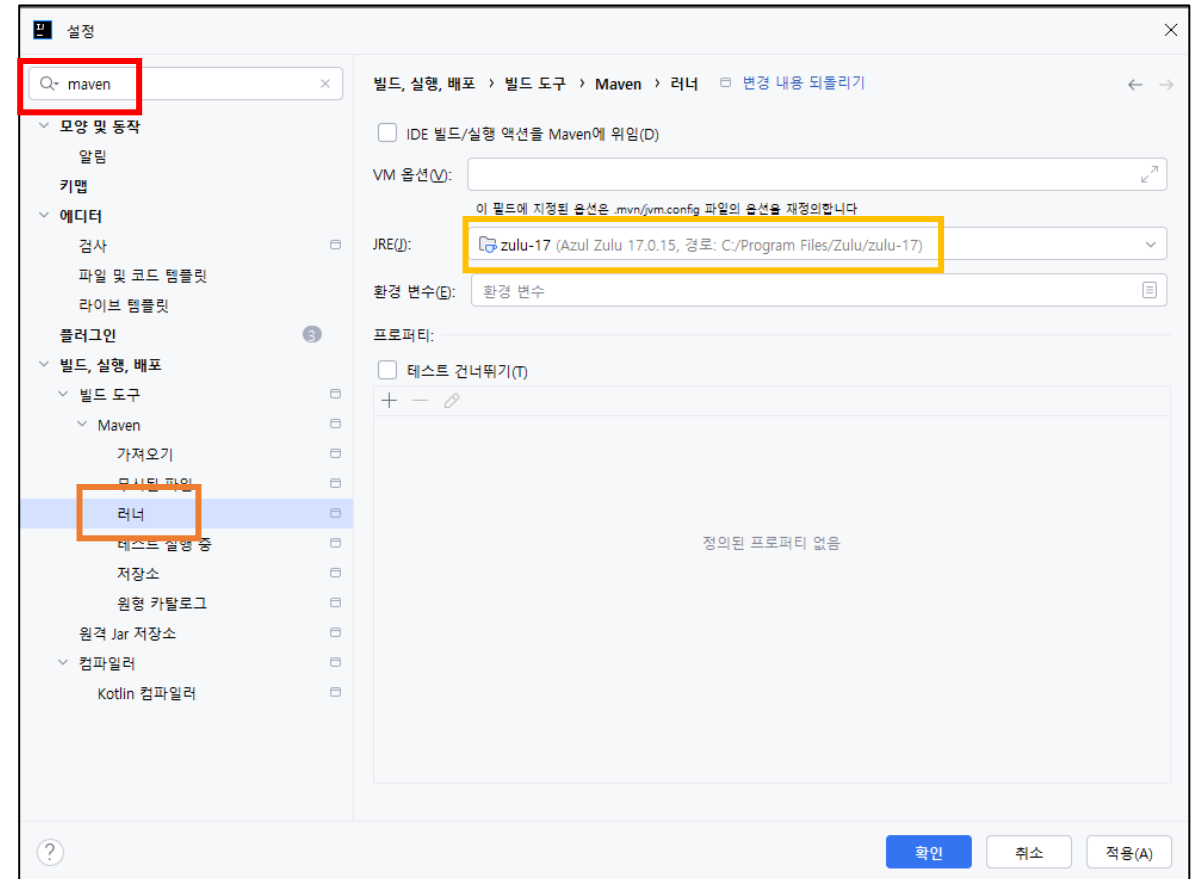
설정 변경

- ✓ file – settings...
- ✓ 검색창에 maven 입력
- ✓ 빌드, 실행, 배포 – 빌드 도구 – Maven
- ✓ 가져오기 -> JDK zulu 정상설치시 출력



설정 변경

- ✓ file – settings...
- ✓ 검색창에 maven 입력
- ✓ 빌드, 실행, 배포 – 빌드 도구 – Maven
- ✓ runner - JRE -> zulu 17 정상 설치시 출력



설정 변경

- ✓ file – Project Structure ...
- ✓ SDK – zulu 확인
- ✓ Language level : 17 - Sealed types, always-strict floating-point semantics



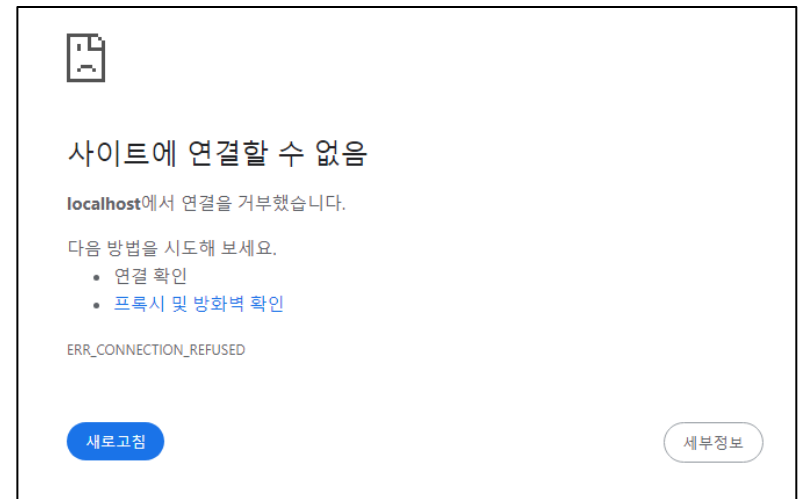
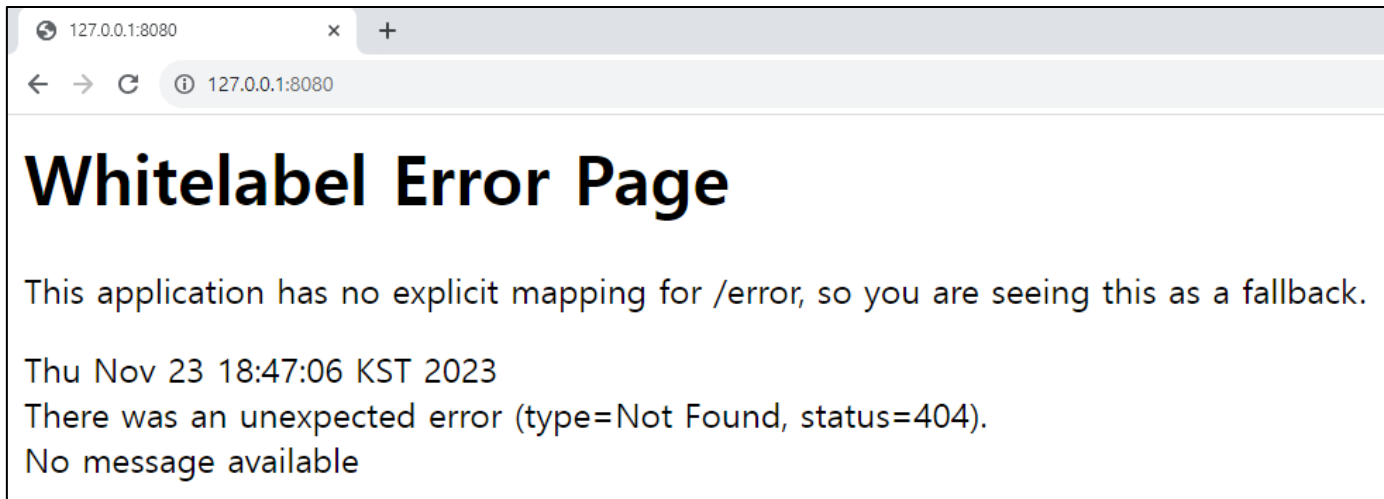
템플릿에 포함된 패키지

- ✓ pom.xml에서 확인 가능
- ✓ 새로고침 클릭하면 변경된 목록으로 갱신(최초 한번)



서버 실행

- ✓ 서버 작동
- ✓ 127.0.0.1:8080 접속
- ✓ 아래와 같은 화면이 나오면 정상 동작
- ✓ 상단 빨간색 중단 버튼 누르고 다시 접속
- ✓ 앞으로 서버 작동이라고 하면, main 메서드 실행을 뜻함
- ✓ 새로고침 클릭하면 변경된 목록으로 갱신(최초 한번)



Chapter 06

Hello Spring!

SQL 프로그래밍

Controller 생성

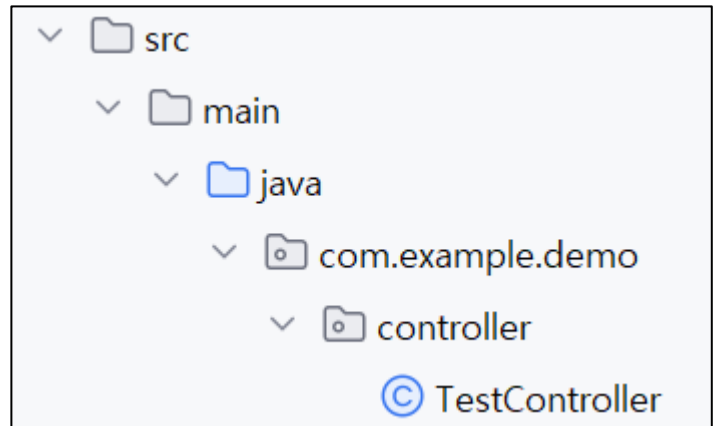
- ✓ HTTP 요청이 들어왔을 때, 작동할 메서드를 모은 클래스
- ✓ controller 패키지 생성 후, TestController 클래스 생성
- ✓ 다음과 같이 입력 후 , 서버 동작 테스트

```
package com.example.demo.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

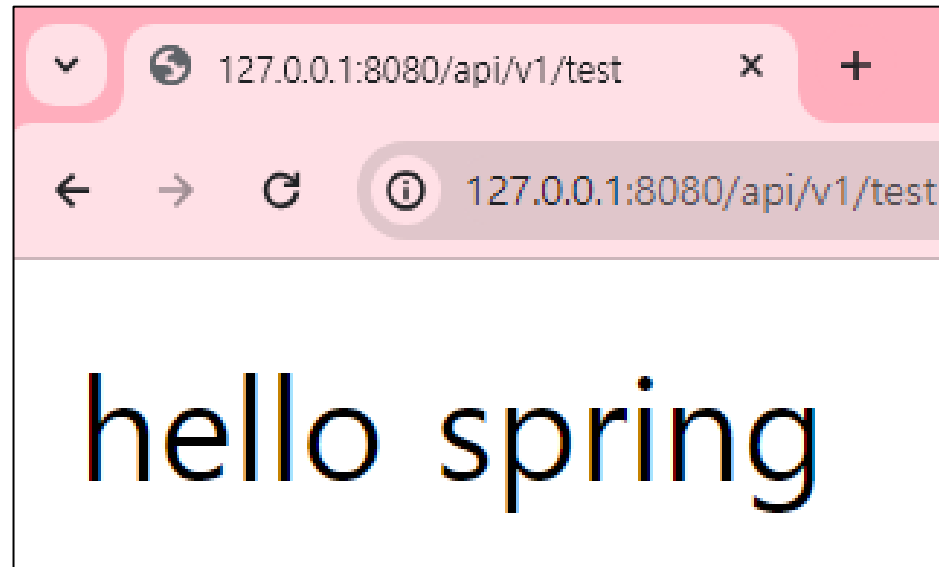
@RestController
@RequestMapping("/api/v1")
public class TestController {

    @GetMapping("/test")
    public String test() {
        return "hello spring";
    }
}
```



Browser 에서 요청

✓ `http://localhost:8080/api/v1/test` 접근



Browser 에서 요청

- ✓ HTTP 요청이 들어왔을 때 작동할 메서드를 모은 클래스
 - HTTP 요청 처리
 - JSON 전송, service 호출

@ 가 앞에 붙은 것을 "어노테이션" 이라고 함
어노테이션이 있다면, Spring 이 해당 부분을 인식

- 부분 : 클래스, 메서드, 필드
- @RestController
 - 해당 클래스가 Rest 컨트롤러임을 나타냄
- @RequestMapping("경로") : 기준 경로
- @GetMapping("경로")
 - 해당 경로로 HTTP GET 요청이 왔을 때 실행시킬 메서드
 - "/api/v1/test" - 127.0.0.1:8080/api/v1/test
- return : 반환값

```
package com.example.demo.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v1")
public class TestController {

    @GetMapping("/test")
    public String test() {
        return "hello spring";
    }
}
```

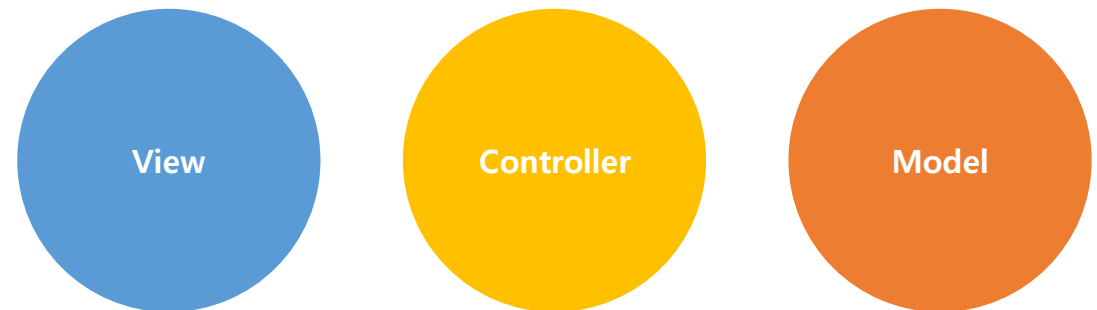
Chapter 07

MVC

SQL 프로그래밍

MVC 패턴

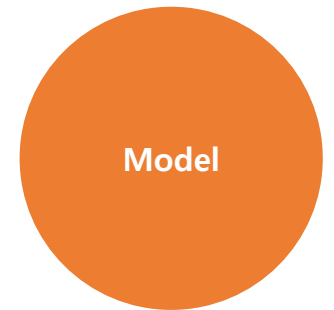
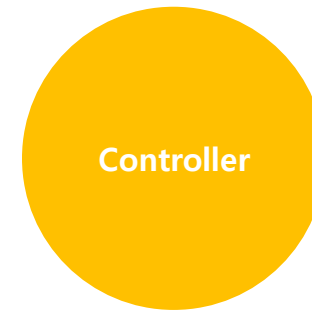
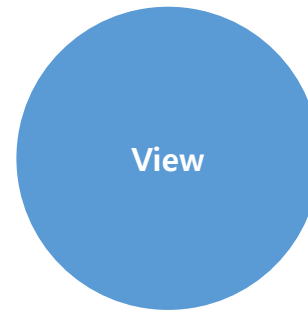
- ✓ 애플리케이션을 세 부분으로 나누어 개발하는 패턴
 - ✓ 하나의 부분으로 개발할 때보다, 협업과 유지보수에 도움이 됨
- ✓ Model : 데이터 조회, 저장, 수정, 삭제
ex) 사용자, 상품, 주문 데이터...
- ✓ View : 사용자에게 데이터를 보여주는 페이지
ex) 로그인, 회원가입, 게시판 페이지...
- ✓ Controller : 사용자의 데이터 요청에 대한 처리
ex) 로그인, 회원가입, 게시물 조회, 사용자 정보 수정 요청



MVC 패턴

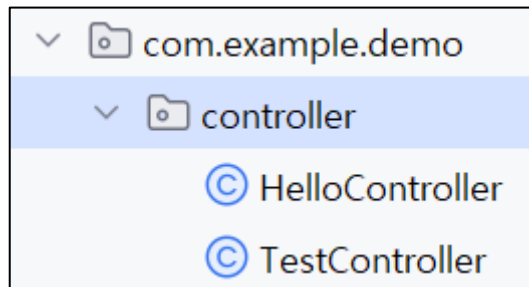
✓ MVC 패턴 예시: 로그인

- 사용자가 View 에서 아이디와 비밀번호를 기입 후 엔터 입력해 Controller 호출
- Controller 에서 아이디와 비밀번호를 Model 에 전달
- Model 은 DB 에 접근해, 해당 사용자의 아이디와 비밀번호가 올바른지 확인
- Model 은 로그인 성공/실패 결과를 Controller 에 전달
- Controller 는 View 에게 결과 전달해 화면에 결과 출력



컨트롤러 만들기

- ✓ Package 추가 : controller
 - ✓ 해당 패키지 내부에 파일 추가 : HelloController



컨트롤러 추가

- ✓ Model
 - MVC 에서 데이터에 해당되는 객체
 - Spring 에 Model 에 모델 객체를 넣어준다.
 - 모델 안에 데이터를 넣어둔다. (addAttribute)
- ✓ @RequestParam
 - Query Param 받을 때 사용
?name="김오토"
 - defaultValue : name 을 제시하지 않았을 경우의 기본값
- ✓ return "hello"
템플릿 폴더에 있는 파일을 지정해준다.
(hello.html)

```
package com.example.demo.controller;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;

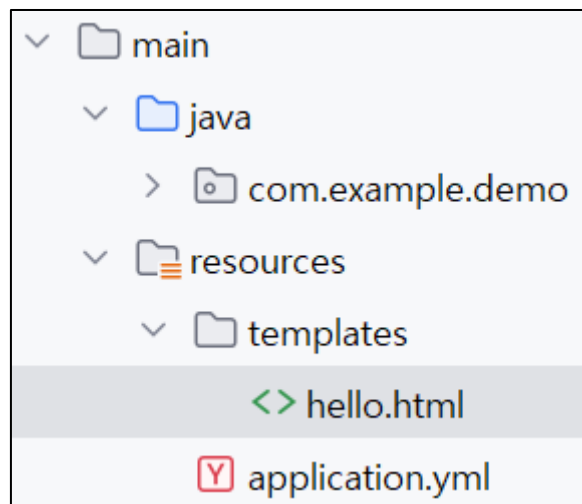
@Controller 0개의 사용위치
public class HelloController {

    @GetMapping("/hello") 0개의 사용위치
    public String hello(
        @RequestParam(value = "name", defaultValue = "김현대") String name,
        Model model
    ) {
        model.addAttribute(attributeName: "name", name);
        return "hello";
    }
}
```

✓ Class 선언부 위에 @RestController 가 아닌 @Controller 를 사용함에 유의!

View 추가

- ✓ resources 내부에 templates 디렉터리 생성
 - ✓ hello.html 파일 추가



template 작성

✓ 제시된 코드 입력 후, 서버 재구동

```
<!DOCTYPE html>
<html
    lang="en"
    xmlns:th="www.thymeleaf.org"
>
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
<p>제 이름은 <span th:text="${name}">000</span>입니다.</p>
</body>
</html>
```

결과 확인

- ✓ /hello
 - ✓ 제 이름은 김현대입니다.
- ✓ /hello?name=김오토
 - ✓ 제 이름은 김오토입니다.

/hello

제 이름은 김현대입니다.

/hello?name=이영훈

제 이름은 김오토입니다.

Thymeleaf 장점

- ✓ 서버 구동 없이, 출력될 모습을 미리 확인 가능

```
<!DOCTYPE html>
<html
    lang="en"
    xmlns:th="www.thymeleaf.org"
>
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
<p>제 이름은 <span th:text="${name}">000</span>입니다.</p>
</body>
</html>
```

제 이름은 000입니다.

Chapter 08

IOC (Inversion of Control)

SQL 프로그래밍

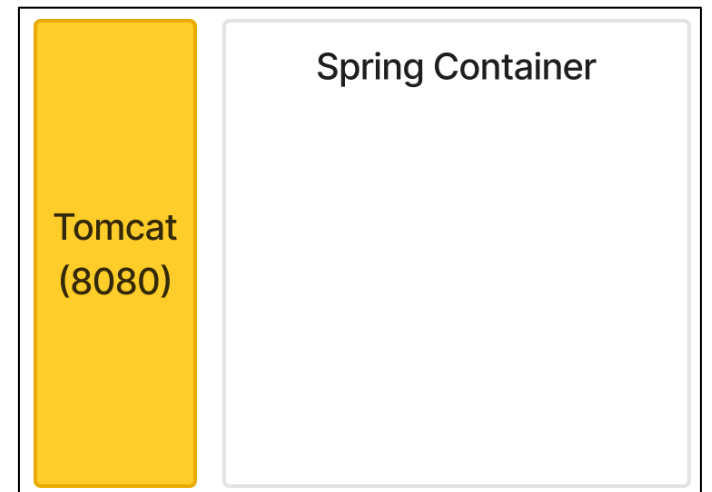
Spring 작동원리

- ✓ 두 가지 방식으로 설명
 - ✓ 어떻게 서비스 준비를 마치는가
 - ✓ 어떻게 서비스하는가

서비스 준비

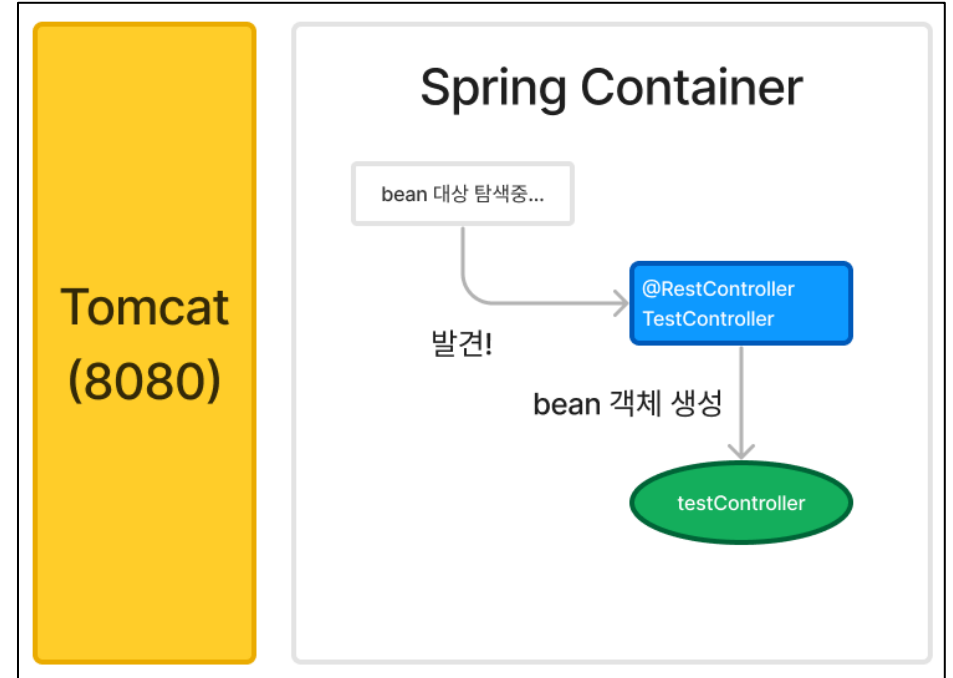
- ✓ Spring 에서 사용하는 서버 : Tomcat
- ✓ 127.0.0.1:8080 에서 서비스 시작 후, HTTP 요청 대기중
- ✓ 이후, Spring Container 생성됨
 - ✓ Spring Container : bean 을 관리하는 저장소
 - ✓ bean : Spring 에서 쓰이는 **전역 객체**
- ✓ 어노테이션이 있다면,
 - ✓ Spring 컨테이너가 해당 부분을 인식
 - ✓ 즉, 어노테이션이 없을 경우, Spring 은 해당 부분을 인식할 수 없음
 - ✓ 상황에 따라 객체로 만들어 bean 으로 관리

```
@RestController  
@RequestMapping("/api/v1")  
public class TestController {
```



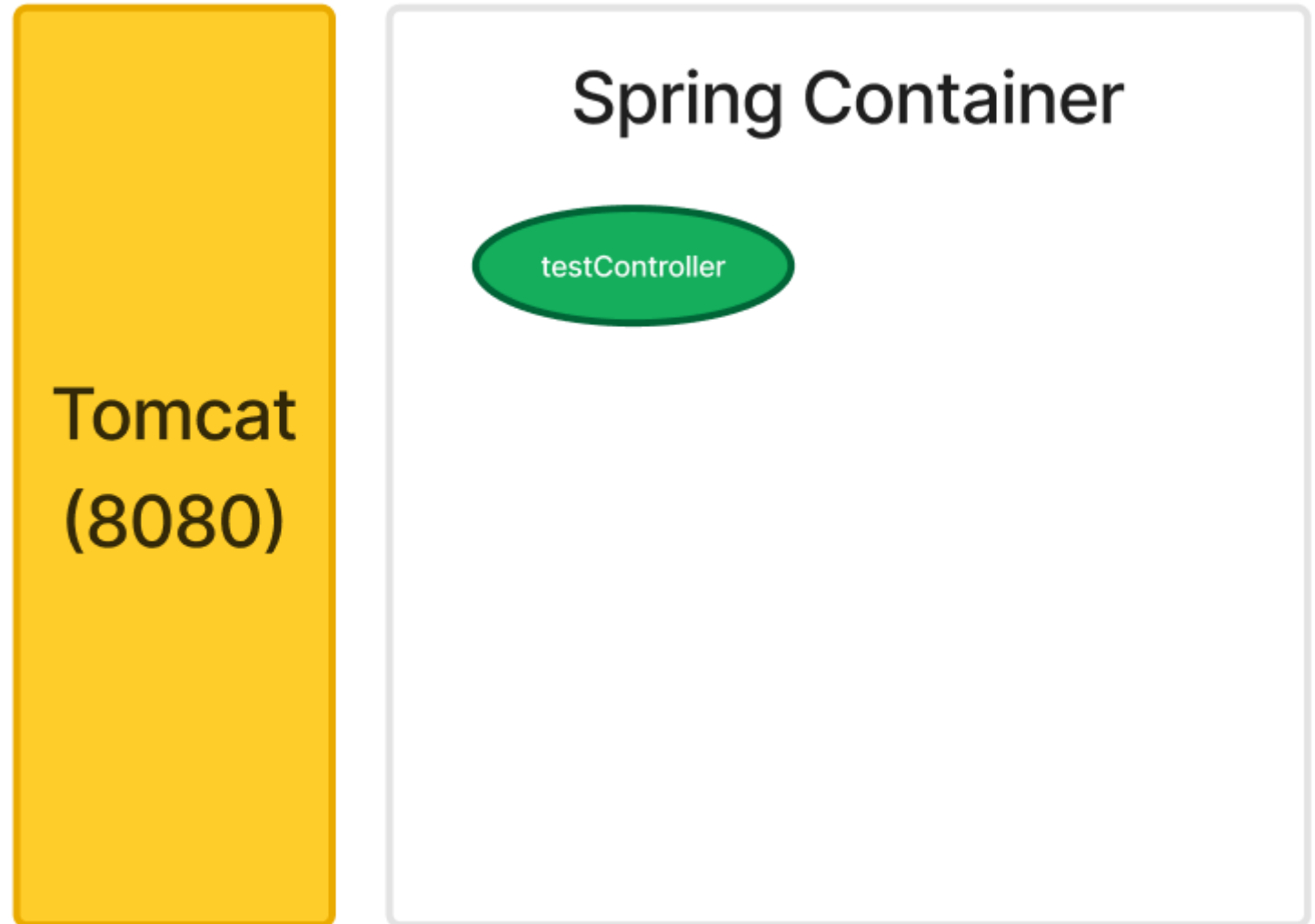
서비스 준비

- ✓ bean 의 정의를 다시 보면,
 - ✓ Spring 에서 쓰이는 **전역 객체**
 - ✓ 즉, 클래스가 아니다!
- ✓ 도식과 같은 과정을 거치게 됨
 - ✓ TestController (클래스) 와,
 - ✓ testController (객체) 가 구분되어 있음
- ✓ 각각의 어노테이션 특성에 따라, bean 객체로 만들 수도 있음
 - ✓ @RestController 어노테이션은 bean 객체로 만든 후,
 - ✓ Spring Container 에서 쓰이는 "전역 객체" 로 저장
- ✓ 도식에서 초록색 찌그러진 원이 보이면,
- ✓ 콩처럼 생겼으므로 bean 이라고 생각하자!



서비스 준비

- ✓ Spring Container 가 준비를 마치면,
- ✓ 다음과 같은 그림이 된다

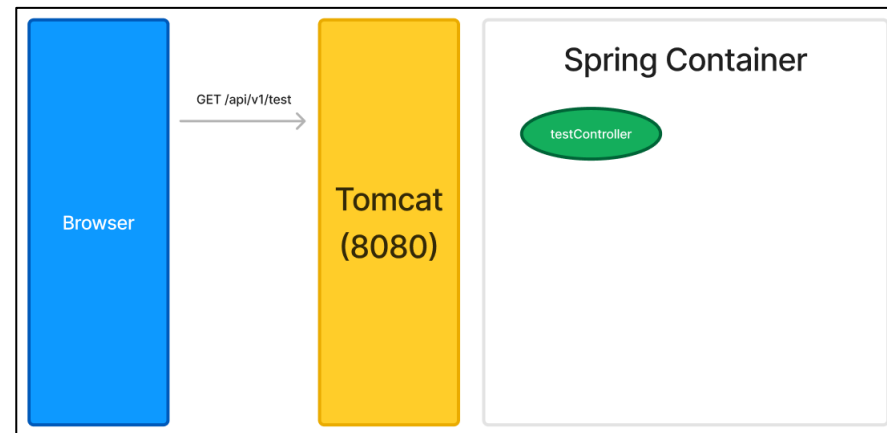


서비스

- ✓ 준비가 끝났으면, 실제 서비스를 어떻게 처리하는지 알아보자.
- ✓ 브라우저에서 127.0.0.1:8080/api/v1/test 으로 접속을 시도한다.
 - ✓ 즉, "GET /api/v1/test " 요청을 보낸다.
- ✓ 이 때, 무엇을 할지 결정하는 게 컨트롤러!
 - ✓ HTTP 요청이 들어왔을 때 작동할 메서드를 모은 클래스
- ✓ 요청이 왔을 때, Spring 은 누가 "GET /api/v1/test " 를 처리할 수 있는지 찾음
 - ✓ 일할 자격 조건: @RestController 어노테이션이 붙어서, bean 객체가 된 것
 - ✓ 찾다보니, testController 에서 해결 가능!
 - ✓ @GetMapping("/test") 어노테이션이 붙은 test 메서드가 있기 때문

```
@RestController
@RequestMapping("/api/v1")
public class TestController {

    @GetMapping("/test")
    public String test() {
        return "hello spring";
    }
}
```

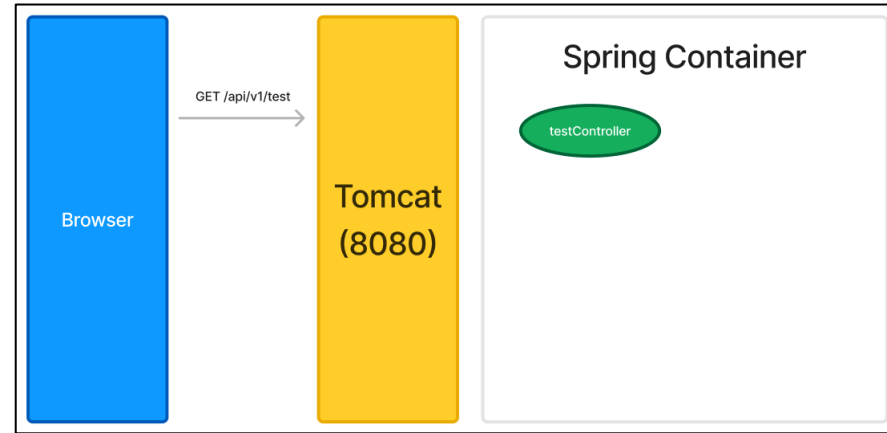


서비스

- ✓ 리턴값은 반환할 문자열
- ✓ 실제 개발 시엔, DTO 전달

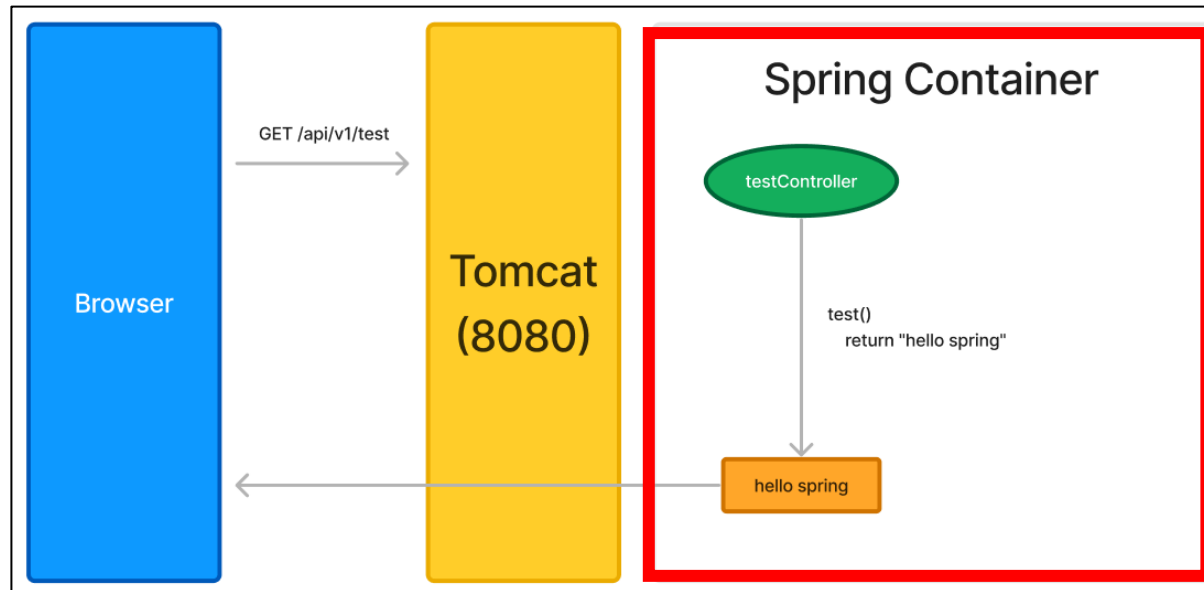
```
@RestController
@RequestMapping("/api/v1")
public class TestController {

    @GetMapping("/test")
    public String test() {
        return "hello spring";
    }
}
```



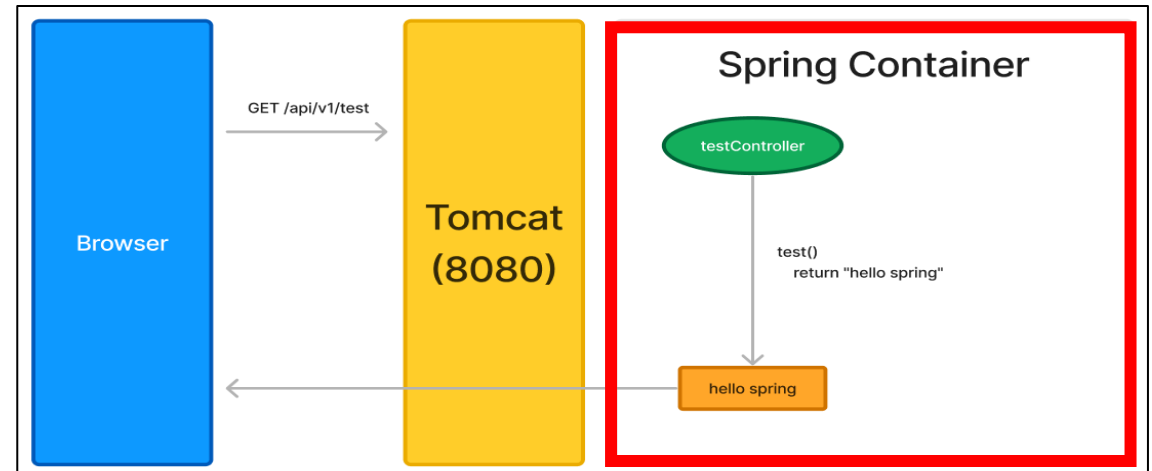
서비스

- ✓ IoC Container : Spring Container 를 의미함
- ✓ IoC (Inversion of Control, 제어권 역전)
 - 객체 생성 및 제어를 App 개발자가 직접 하지 않음
 - 대신 "IoC Container" 가 직접 생성 및 관리
 - App 개발자는 객체 필요 시 참조를 가져옴



총정리

1. Tomcat 이 먼저 실행되어 8080 서버에서 서비스를 시작하고, HTTP 요청을 기다릴 준비를 마칩
2. 이후, 스프링 컨테이너라는 것이 생성되어 bean 이 될 대상을 탐색
3. @RestController 어노테이션의 경우, 해당 클래스는 bean 객체가 되어 스프링 컨테이너에서 쓰이는 "전역 객체" 가 됨
4. 사용자는 브라우저를 켜고 "GET /api/v1/test " 요청을 보냄
5. Spring 은 컨트롤러 bean 들 중에 누가 "GET /api/v1/test " 요청을 처리할 수 있을지를 찾음
6. @GetMapping("/test") 어노테이션이 사용된 test 메서드를 발견해 실행하고, "hello spring" 를 리턴
7. 브라우저로 hello spring 을 보냄



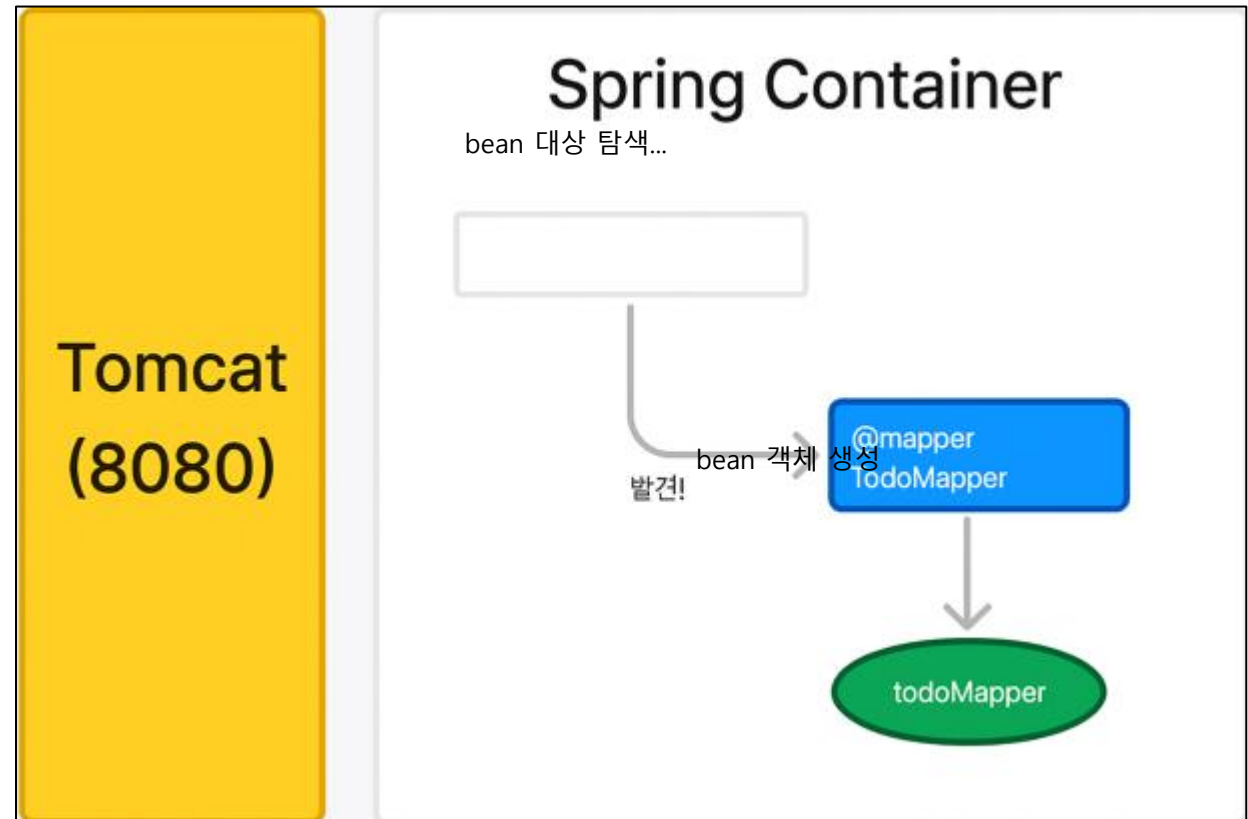
Chapter 09

DI (Dependency Injection)

SQL 프로그래밍

서비스 준비

- ✓ 우리 예제에 사용될 어노테이션 중 bean 객체를 만드는 것들
 - ✓ 클래스 단위 : 기본적으로 @Component 를 포함
 - ✓ @Mapper : MyBatis 매퍼 인터페이스에 사용, Spring이 Bean 등록
 - ✓ @Service
 - ✓ @Controller, @RestController
 - ✓ @Repository : Mybatis에서는 매퍼에 주로 @Mapper
 - ✓ @Repository 필수아님



서버 구동

- ✓ JPA는 Entity를 기반으로 테이블을 생성하지만, MyBatis는 XML로 SQL을 직접 작성
- ✓ Spring은 @Mapper 어노테이션을 통해 해당 인터페이스를 Bean으로 등록
- ✓ Service에서는 Mapper를 주입받아 DB에 접근하며, Controller는 Service를 통해 기능을 호출

DI

- ✓ DI : todoService - todoMapper
- ✓ private : 클래스 외부로 노출 안함
- ✓ final : 한 번 초기화되면 값 변경 불가
- ✓ 생성자 사용
- ✓ @Autowired : Spring 이 자동으로 bean 찾아서 연결
- ✓ 생성자를 사용해 DI 를 적용하는 기법을
- ✓ "생성자 주입" (Constructor Injection) 이라고 함

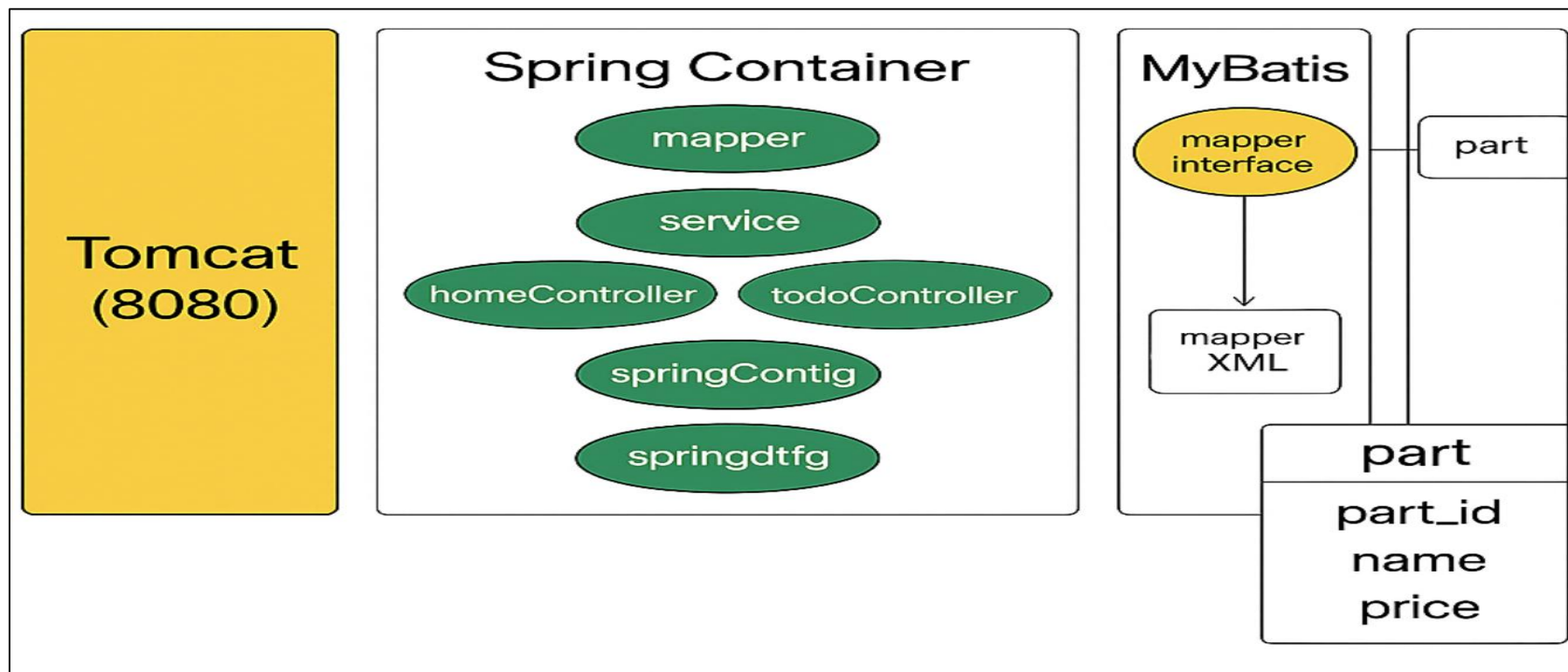
```
@Service
public class TodoService {

    private final TodoMapper todoMapper;

    @Autowired
    public TodoService(TodoMapper todoMapper) {
        this.todoMapper = todoMapper;
    }
}
```

DI

✓ 이로서, 서비스 준비를 마치게 됨



Chapter 10

프로젝트

SQL 프로그래밍

서비스 준비

- ✓ 별도의 인증 없이 누구나 글을 작성·조회·수정·삭제(CRUD)할 수 있는 최소한의 게시판 API 서버 구현
 - 글 작성(Create)
 - 글 목록/단건 조회(Read)
 - 글 수정(Update)
 - 글 삭제>Delete)

DB 모델링

필드명	타입	설명
id	Long (PK)	게시글 고유 ID
title	String	게시글 제목
content	Text or String	게시글 내용
created_at	Timestamp	생성 시각
updated_at	Timestamp	수정 시각

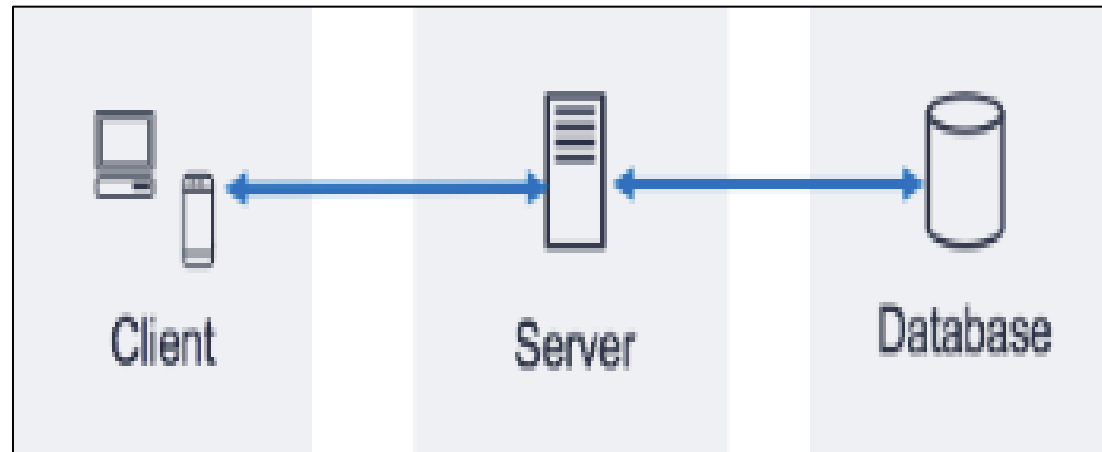
API 설계

메서드	경로	설명
POST	/posts	게시글 작성
GET	/posts	게시글 전체 목록 조회
GET	/posts/{id}	게시글 단건 조회
PUT	/posts/{id}	게시글 수정
DELETE	/posts/{id}	게시글 삭제

Chapter 11

DB 제어 방법

SQL 프로그래밍



Spring DB 제어방법

1. JDBC & Spring Data JDBC
2. Spring Data JPA
3. MyBatis

✓ JDBC

- 자바에서 데이터베이스에 연결하고, SQL을 실행하며, 결과를 주고받을 수 있도록 해주는 **표준** API
- 데이터베이스별 드라이버를 통해 다양한 DBMS(Oracle, MySQL, PostgreSQL 등)와 연결
- 주요 객체: Connection, Statement, PreparedStatement, ResultSet
- 모든 자바 기반의 데이터베이스 연동 기술(JPA, MyBatis 포함)은 내부적으로 JDBC API를 사용
- 모든 과정을 개발자가 수동으로 처리

```
import java.sql.*;

public class JDBCExample {

    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/testDB";
        String user = "your_username";
        String password = "your_password";

        try (Connection conn = DriverManager.getConnection(url, user, password);
            Statement stmt = conn.createStatement()) {

            // 데이터베이스 스키마 생성 (선택 사항)
            stmt.execute(sql: "CREATE TABLE IF NOT EXISTS users (id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(255));");

            // 데이터 삽입
            stmt.executeUpdate(sql: "INSERT INTO users (name) VALUES ('John Doe'), ('Jane Doe')");

            // 데이터 조회
            ResultSet rs = stmt.executeQuery(sql: "SELECT * FROM users");
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt(columnLabel: "id") + ", Name: " + rs.getString(columnLabel: "name"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

✓ Spring Data JDBC

- Spring에서 제공하는 JDBC 기반의 데이터 접근을 지원하는 프레임워크
- ORM을 최소화하고, 직접 SQL을 사용하여 자바 객체와 테이블을 매핑
- 복잡한 JPA/Hibernate 기능 없이, 단순하고 예측 가능한 데이터 접근을 제공
- JPA와 유사하게 Entity 클래스 기반으로 Repository를 자동 생성
- 내부는 SQL로 작동하고 복잡한 ORM 기능은 없음
- JPA보다 단순하지만, 순수 SQL보다 생산성이 높음

"JPA는 너무 복잡하고, MyBatis는 SQL 작성이 귀찮지만, JDBC는 너무 로우 레벨이야" 라는 생각을 할 때 고려해볼 수 있는 대안

Spring Data JDBC 예

```
import org.springframework.data.annotation.Id;
import org.springframework.data.relational.core.mapping.Table;
import org.springframework.stereotype.Repository;
import org.springframework.data.repository.CrudRepository;
import org.springframework.beans.factory.annotation.Autowired;
import java.util.List;

@Table("users") 3개 사용 위치
public class User {
    @Id 0개의 사용위치
    private Long id;
    private String name; 0개의 사용위치
    // getter/setter
}

@Repository 1개 사용 위치
public interface UserRepository extends CrudRepository<User, Long> {
    List<User> findByName(String name); 1개 사용 위치
}

// 사용 예시 (서비스 등)
@Autowired 1개 사용 위치
private UserRepository userRepository;

List<User> users = userRepository.findByName("홍길동"); 0개의 사용위치
```

- ✓ SQL Mapper 프레임워크로, 개발자가 **직접** 작성한 SQL과 자바 객체를 매핑해주는 오픈소스 프레임워크
- ✓ SQL을 XML 파일이나 어노테이션으로 **분리** 관리하며, JDBC의 반복적인 코드를 자동화
- ✓ 직접 SQL을 제어할 수 있어, 복잡한 쿼리나 최적화가 필요한 상황에 유리
- ✓ 스프링과 연동 시 @Mapper 인터페이스를 통해 간단하게 사용
- ✓ 반자동 ORM이라 불림 (SQL은 수동, 매핑은 자동)

MyBatis 예

```
@Data 0개의 사용위치
public class User {
    private Long id; 0개의 사용위치
    private String name; 0개의 사용위치
}
```

```
@Mapper 0개의 사용위치
public interface UserMapper {
    @Select("SELECT * FROM users") 0개의 사용위치
    List<User> getAllUsers();
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper
    PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
    "http://mybatis.org/dtd/mybatis-3-mapper.dtd">

<mapper namespace="com.example.mapper.UserMapper">
    <select id="getAllUsers" resultType="com.example.domain.User">
        SELECT * FROM users
    </select>
</mapper>
```

- ✓ JPA(Java Persistence API)는 자바 애플리케이션에서 객체와 관계형 데이터베이스 간의 매핑
 - ✓ (ORM, Object-Relational Mapping)을 관리하는 자바 ORM 기술의 표준 인터페이스
- ✓ 개발자는 SQL을 직접 작성하지 않고, 자바 객체를 조작함으로써 데이터베이스 연산.
- ✓ JPA는 데이터베이스 접근 방식을 추상화하여, 객체지향적으로 데이터를 다룸
- ✓ 대표적인 구현체로는 Hibernate가 있으며, Spring에서는 Spring Data JPA를 통해 더욱 간편하게 사용
- ✓ JPA를 사용하면 객체와 테이블을 자동 매핑, 메서드 호출만으로도 CRUD 등 데이터베이스 작업이 가능
- ✓ 반복적인 SQL 작성이 줄어들고, 객체지향적인 코드 작성과 유지보수가 쉬움

@Entity 0개의 사용위치

```
public class User {
```

@Id 2개 사용 위치

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;
```

```
private String name; 3개 사용 위치
```

```
public User() {} 1개 사용 위치
```

```
public User(String name) { 1개 사용 위치
```

```
    this.name = name;
```

```
}
```

@Component 0개의 사용위치

```
public class UserService implements CommandLineRunner {
```

```
    private final UserRepository userRepository; 4개 사용 위치
```

```
    public UserService(UserRepository userRepository) { 0개의 사용위치
```

```
        this.userRepository = userRepository;
```

```
}
```

@Override

```
public void run(String... args) {
```

```
    // 저장 예시
```

```
    userRepository.save(new User("홍길동"));
```

```
    userRepository.save(new User("이순신"));
```

```
    // 모든 유저 조회
```

```
    List<User> users = userRepository.findAll();
```

```
    for (User user : users) {
```

```
        System.out.println("ID: " + user.getId() + ", 이름: " + user.getName());
```

```
}
```

```
}
```

```
}
```

- ✓ 치킨을 먹는 것으로 비유하자면,
 - JDBC → 닭을 직접 잡고, 손질하고, 양념해서, 기름에 튀겨서 치킨을 만드는 것
(모든 과정을 개발자가 직접 처리)
 - Spring Data JDBC → 정육점에서 손질된 닭을 사 와서, 직접 양념하고 튀기는 것
(닭 손질 등 반복적이고 번거로운 작업은 프레임워크가 대신 해주고, 개발자는 필요한 부분(양념, 튀김 등)만 신경)
- ✓ MyBatis -> 치킨 전문점에서 반조리 치킨(양념, 튀김옷 입힘)을 사 와서 집에서 튀기는 것
(SQL 작성 등 핵심 조리법(레시피)은 개발자가 직접 만들지만, 반복적이고 번거로운 준비 작업은 프레임워크)
- ✓ JPA (Spring Data JPA, Hibernate) -> 프랜차이즈 치킨집에 치킨 한 마리 주문하는 것
치킨이 완성되어 소스와 함께 배달까지
(복잡한 과정은 모두 프레임워크가 자동 처리)

Chapter 12

MyBatis & MySQL 연동

SQL 프로그래밍

연동 과정 단계

- ✓ MySQL 데이터베이스 준비
- ✓ 프로젝트 의존성 추가(pom.xml)
- ✓ 데이터베이스 연결 정보 설정(application.properties)
 - db 생성
- ✓ 모델 클래스 생성
- ✓ Mapper 인터페이스 및 매퍼 XML
 - db 테이블 생성
- ✓ 서비스 및 컨트롤러 구현

1.MySQL 데이터베이스 준비

Welcome to MySQL Workbench

MySQL Workbench is the official graphical user interface (GUI) tool for MySQL. It allows you to design, create and browse your database schemas, work with database objects and insert data as well as design and run SQL queries to work with stored data. You can also migrate schemas and data from other database vendors to your MySQL database.

[Browse Documentation >](#)

[Read the Blog >](#)

[Discuss on the Forums >](#)

MySQL Connections

Local instance MySQL80

 root

 localhost:3306

2.프로젝트 의존성 추가

```
<dependency>
  <groupId>org.mybatis.spring.boot</groupId>
  <artifactId>mybatis-spring-boot-starter</artifactId>
  <version>3.0.4</version>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>

<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```


3.데이터 베이스 연결 정보 설정

```
# MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/demo?useSSL=false&serverTimezone=UTC&characterEncoding=UTF-8
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# MyBatis
mybatis.mapper-locations=classpath:mapper/*.xml
mybatis.type-aliases-package=com.example.demo.model
mybatis.configuration.map-underscore-to-camel-case=true
```

3.모델 클래스 생성

```
package com.example.demo.model;

public class User { 20개 사용 위치
    private Long id; 3개 사용 위치
    private String name; 4개 사용 위치
    private String email; 4개 사용 위치

    // 기본 생성자
    public User() {} 0개의 사용위치

    // 생성자
    public User(String name, String email) { 0개의 사용위치
        this.name = name;
        this.email = email;
    }
}
```

4.Mapper 인터페이스

```
@Mapper 3개 사용 위치
public interface UserMapper {

    // 모든 사용자 조회
    List<User> findAll(); 1개 사용 위치

    // ID로 사용자 조회
    User findById(@Param("id") Long id); 1개 사용 위치

    // 사용자 등록
    void insert(User user); 1개 사용 위치

    // 사용자 수정
    void update(User user); 1개 사용 위치

    // 사용자 삭제
    void deleteById(@Param("id") Long id); 1개 사용 위치
}
```

4.Mapper XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN" "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="com.example.demo.mapper.UserMapper">

    <!-- 결과 매핑 -->
    <resultMap id="UserResultMap" type="com.example.demo.model.User">
        <id column="id" property="id"/>
        <result column="name" property="name"/>
        <result column="email" property="email"/>
    </resultMap>

    <!-- 모든 사용자 조회 -->
    <select id="findAll" resultMap="UserResultMap">
        SELECT id, name, email
        FROM users
        ORDER BY id
    </select>

    <!-- ID로 사용자 조회 -->
    <select id="findById" parameterType="long" resultMap="UserResultMap">
        SELECT id, name, email
        FROM users
        WHERE id = #{id}
    </select>

    <!-- 사용자 등록 -->
    <insert id="insert" parameterType="com.example.demo.model.User" useGeneratedKeys="true" keyProperty="id">
        INSERT INTO users (name, email)
        VALUES (#{name}, #{email})
    </insert>

    <!-- 사용자 수정 -->
    <update id="update" parameterType="com.example.demo.model.User">
        UPDATE users
        SET name = #{name}, email = #{email}
        WHERE id = #{id}
    </update>

    <!-- 사용자 삭제 -->
    <delete id="deleteById" parameterType="long">
        DELETE FROM users
        WHERE id = #{id}
    </delete>

</mapper>
```

5.서비스

```
@Service 2개 사용 위치
public class UserService {

    @Autowired 5개 사용 위치
    private UserMapper userMapper;

    // 모든 사용자 조회
    public List<User> getAllUsers() { return userMapper.findAll(); }

    // ID로 사용자 조회
    public User getUserById(Long id) { return userMapper.findById(id); }

    // 사용자 등록
    public void createUser(User user) { userMapper.insert(user); }

    // 사용자 수정
    public void updateUser(User user) { userMapper.update(user); }

    // 사용자 삭제
    public void deleteUser(Long id) { userMapper.deleteById(id); }
}
```

6.컨트롤러

```
@RestController 0개의 사용위치
@RequestMapping("/api/users")
public class UserController {

    @Autowired 5개 사용 위치
    private UserService userService;

    // 모든 사용자 조회
    @GetMapping 0개의 사용위치
    public ResponseEntity<List<User>> getAllUsers() {
        List<User> users = userService.getAllUsers();
        return ResponseEntity.ok(users);
    }

    // ID로 사용자 조회
    @GetMapping("/{id}") 0개의 사용위치
    public ResponseEntity<User> getUserById(@PathVariable Long id) {
        User user = userService.getUserById(id);
        if (user != null) {
            return ResponseEntity.ok(user);
        } else {
            return ResponseEntity.notFound().build();
        }
    }

    // 사용자 등록
    @PostMapping 0개의 사용위치
    public ResponseEntity<String> createUser(@RequestBody User user) {
        try {
            userService.createUser(user);
            return ResponseEntity.ok( body: "사용자가 성공적으로 등록되었습니다.");
        } catch (Exception e) {
            return ResponseEntity.badRequest().body("사용자 등록에 실패했습니다: " + e.getMessage());
        }
    }

    // 사용자 수정
    @PutMapping("/{id}") 0개의 사용위치
    public ResponseEntity<String> updateUser(@PathVariable Long id, @RequestBody User user) {
        try {
            user.setId(id);
            userService.updateUser(user);
            return ResponseEntity.ok( body: "사용자 정보가 성공적으로 수정되었습니다.");
        } catch (Exception e) {
            return ResponseEntity.badRequest().body("사용자 수정에 실패했습니다: " + e.getMessage());
        }
    }

    // 사용자 삭제
    @DeleteMapping("/{id}") 0개의 사용위치
    public ResponseEntity<String> deleteUser(@PathVariable Long id) {
        try {
            userService.deleteUser(id);
            return ResponseEntity.ok( body: "사용자가 성공적으로 삭제되었습니다.");
        } catch (Exception e) {
            return ResponseEntity.badRequest().body("사용자 삭제에 실패했습니다: " + e.getMessage());
        }
    }
}
```

Chapter 13

프로젝트

SQL 프로그래밍

서비스 준비

✓ 기존의 게시판에 댓글 기능 추가구현

- 댓글 작성(Create)
- 댓글 목록/댓글 단건 조회(Read)
- 댓글 수정(Update)
- 댓글 삭제>Delete)

필드 이름	타입	설명
id	Long (PK)	댓글 고유 ID (기본키)
post_id	Long (FK)	연결된 게시글의 ID (외래키)
content	String/Text	댓글 내용
created_at	Timestamp	댓글 작성 시각
updated_at	Timestamp	댓글 수정 시각

메서드	경로	설명
POST	/posts/{postId}/comments	특정 게시물에 댓글 작성
GET	/posts/{postId}/comments	특정 게시물의 댓글 목록 조회
GET	/comments/{id}	댓글 단건 조회
PUT	/comments/{id}	댓글 수정
DELETE	/comments/{id}	댓글 삭제

추가 구현 아이디어

- ✓ 조회수 출력
- ✓ 좋아요 / 싫어요
- ✓ 페이지네이션 / 게시물 상세조회(조회수, 좋아요, 댓글 많은순)
- ✓ 게시물 검색(작성자, 게시물 제목 일부)
- ✓ 파일 첨부